
计算机图形学 实验指导书

电子科技大学计算机科学与工程学院

2024 年 12 月

目录

实验环境.....	1
OpenGL 简介.....	1
实验材料.....	1
配置步骤.....	1
环境验证.....	3
OpenGL 窗口程序编程框架.....	5
实验一 直线段绘制算法.....	8
实验目的.....	8
实验内容.....	8
实验结果参考.....	10
实验二 圆生成算法.....	11
实验目的.....	11
实验内容.....	11
实验结果参考.....	14
实验三 填充算法.....	15
实验目的.....	15
实验内容.....	15
实验结果参考.....	24
实验四 立方体绘制.....	25
实验目的.....	25
实验内容.....	25
实验结果参考.....	31
实验五 图形几何变换.....	32
实验目的.....	32
实验内容.....	32
实验结果参考.....	36
实验六 图形用户交互.....	37
实验目的.....	37
实验内容.....	37
实验结果参考.....	44
实验七 图形观察变换.....	45
实验目的.....	45

实验内容.....	45
实验结果参考.....	52
实验八 贝塞尔曲线.....	53
实验目的.....	53
实验内容.....	53
实验结果参考.....	66
实验九 分形图形.....	67
实验目的.....	67
实验内容.....	67
实验结果参考.....	71
实验十 导入模型文件.....	73
实验目的.....	73
实验内容.....	73
实验结果参考.....	95
实验十一 绘图软件.....	96
实验目的.....	96
实验内容.....	96
实验结果参考.....	99
附录 1 实验三 填充算法的 C++实现.....	100
实验目的.....	100
实验内容.....	100
实验结果参考.....	109
附录 2 实验八 贝塞尔曲线的 C++实现.....	110
实验目的.....	110
实验内容.....	110
实验结果参考.....	123
附录 3 实验十 C 语言实现.....	124
实验目的.....	124
实验内容.....	124
附录 4 实验十一 C 语言实现.....	146
实验目的.....	146
实验内容.....	146

实验环境

OpenGL 简介

OpenGL (Open Graphics Library, 开放图形库) 是用于渲染 2D、3D 矢量图形的跨语言、跨平台的应用程序编程接口。OpenGL 标准最初由 OpenGL 架构评审委员会 (ARB) 维护, 之后由 Khronos Group 更新。相似的实现包括由微软公司开发的仅用于 Microsoft Windows 系统的 Direct3D, 以及高性能跨平台图形计算接口 Vulkan。

实验材料

- Visual Studio 2017 及其他版本
- OpenGL related files
 - opengl32.lib, opengl32.dll, gl.h
 - glu32.lib, glu32.dll, glu.h
 - glut32.lib, glut32.dll, glut.h

配置步骤

下面提供了 3 种配置方式, 实质都是相同的, 都是让编译器找到正确的文件位置。

最通用方法

1. 将 glut32.dll 复制到 c:\windows\system32 或 c:\windows\SysWOW64 (for Win10 and later)
2. 在项目根目录创建 include 和 lib 文件夹, include 文件夹放入 gl.h, glu.h, glut.h, lib 文件夹放入 opengl32.lib, glu32.lib, glut32.lib
3. 在 项目-属性-VC++ 目录 设置 包含目录 和 库目录, 包含目录添加 include 文件夹路径, 库目录添加 lib 文件夹路径
4. 配置项目属性中的 链接-输入-附加依赖项, 添加 opengl32.lib, glu32.lib, glut32.lib 到依赖项

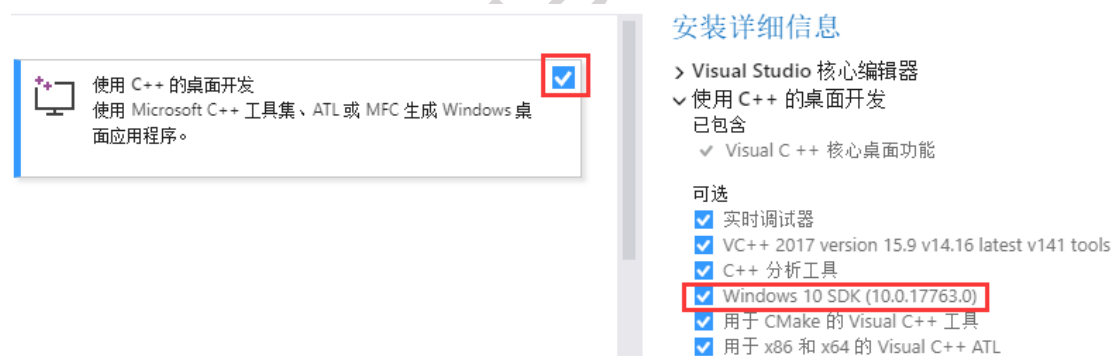
上面的方法每次创建新项目都需要将库文件拷贝进项目根目录中，下面的 2 种方法将库文件添加进编译器或系统 SDK 文件，避免了上述麻烦。

添加进 Visual Studio

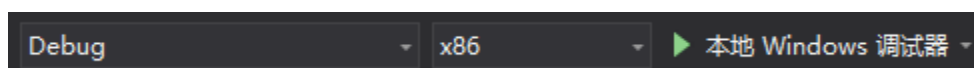
1. 将 glut32.dll 复制到 c:\windows\system32 或 c:\windows\SysWOW64 (for Win10 and later)
2. 定位 VS 的安装路径
3. 在*/include 文件夹下建子目录 GL，并复制三个头文件 gl.h, glu.h, glut.h 到其中
4. 在*/lib 中复制三个库文件 opengl32.lib, glu32.lib, glut32.lib
5. 配置项目属性中的 链接-输入-附加依赖项，添加 opengl32.lib, glu32.lib, glut32.lib 到依赖项

添加进 Win10 SDK

1. 将 glut32.dll 复制到 c:\windows\system32 或 c:\windows\SysWOW64 (for Win10 and later)
2. 首先确认已经在 VS Installer 中安装了 Windows 10 SDK，如图所示：



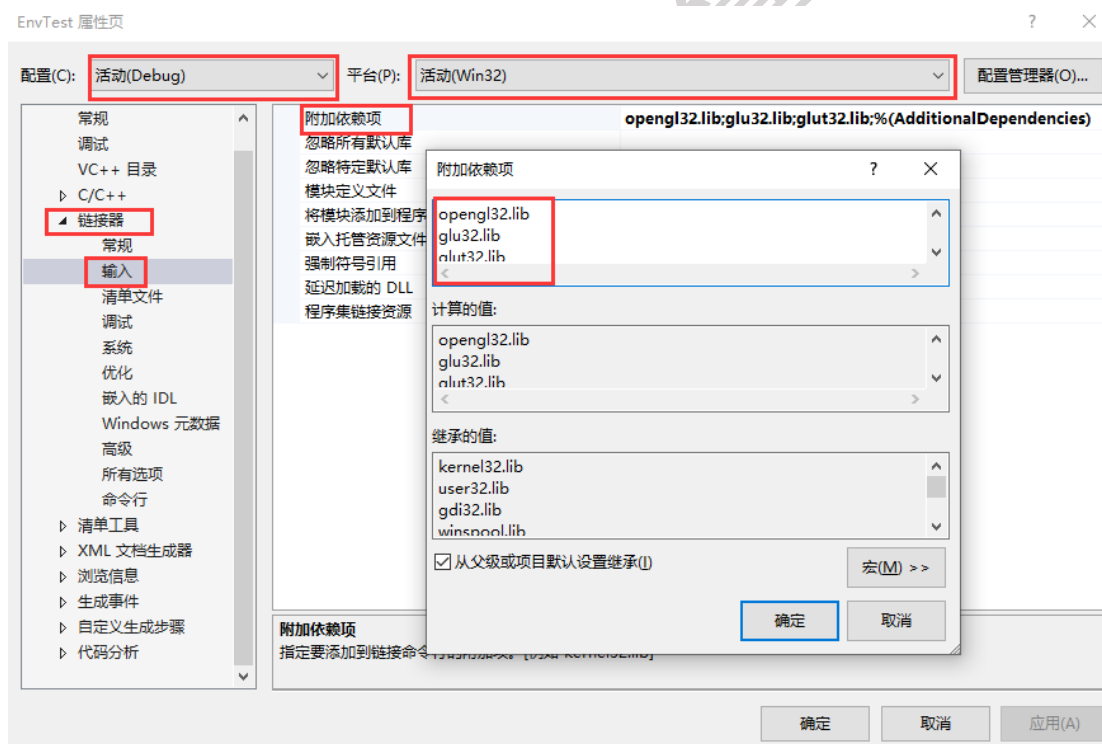
3. 在 SDK 安装位置找到 Windows Kit 文件夹
4. 找到 include 和 lib 文件夹，将对应的文件复制进去
如：在 D:\Windows Kits\10\Include\10.0.17763.0\um\gl 中放入*.h 文件
在 D:\Windows Kits\10\Lib\10.0.17763.0\um\x86 中放入*.lib 文件
5. 配置项目属性中的 链接-输入-附加依赖项，添加 opengl32.lib, glu32.lib, glut32.lib 到依赖项
6. 注意查看编译配置如下（x86 对应 32 位程序，默认使用 32 位引用目录和库目录，即 um\x86\ 中查找文件，如果用 64 位编译，放入位置为 \um\x64\ ）



此阶段产生的绝大部分问题都是路径索引错误，请详细检查所需文件是否可以被项目正确引用。

环境验证

1. 使用 VS 新建 Visual C++ 空项目，添加 C++ 源文件
2. 右键项目，配置属性
3. 在链接器→输入→附加依赖项中，添加 `opengl32.lib;glu32.lib;glut32.lib`，注意保证对应当前活动配置，即默认的 Debug Win32



4. 在 C++ 源文件中添加如下代码，详见《计算机图形学第四版》3.5 节：

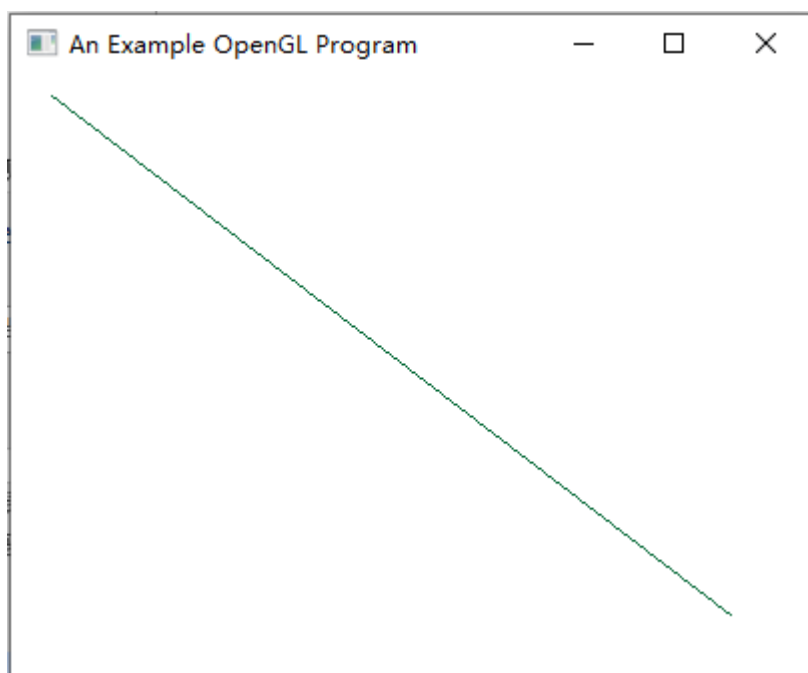
```
1.  #include <GL/glut.h>
2.
3.  void init (void)
4.  {
```

```

5.      glClearColor (1.0, 1.0, 1.0, 0.0); // Set display-
window color to white.
6.      glMatrixMode (GL_PROJECTION);      // Set projection parameters.
7.      gluOrtho2D (0.0, 200.0, 0.0, 150.0);
8.  }
9.
10. void lineSegment (void)
11. {
12.     glClear (GL_COLOR_BUFFER_BIT); // Clear display window.
13.
14.     glColor3f (0.0, 0.4, 0.2);      // Set line segment color to green.
15.
16.     glBegin (GL_LINES);
17.         glVertex2i (180, 15);      // Specify line-segment geometry.
18.         glVertex2i (10, 145);
19.     glEnd ( );
20.
21.     glFlush ( ); // Process all OpenGL routines as quickly as possi-
ble.
22.
23. void main (int argc, char** argv)
24. {
25.     glutInit (&argc, argv);          // Initialize GLUT.
26.     glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB); // Set display mode
.
27.     glutInitWindowPosition (50, 100); // Set top-left display-
window position.
28.     glutInitWindowSize (400, 300);    // Set display-
window width and height.
29.     glutCreateWindow ("An Example OpenGL Program"); // Create display w
indow.
30.
31.     init ( );                          // Execute initialization proced
ure.
32.     glutDisplayFunc (lineSegment);    // Send graphics to display wi
ndow.
33.     glutMainLoop ( );                  // Display everything and wait.
34. }

```


5. 开始调试，程序会正确输出一条斜线



OpenGL 窗口程序编程框架

基础结构

1. 在源文件中引入 `#include <gl/glut.h>` 头文件
2. 在 `main` 函数中使用 `glutInit` 函数初始化 GLUT
3. 使用 `glutInitDisplayMode` 设置显示模式
4. 使用 `glutInitWindowPosition` 设置窗口相对屏幕左上角的位置
5. 使用 `glutInitWindowSize` 设置窗口大小
6. 使用 `glutCreateWindow` 创建显示窗口，并设置标题
7. 在 `init` 自定义函数中，设置背景颜色，投影模式，裁剪窗口
8. 使用 `glutDisplayFunc` 将 `renderScene` 自定义显示函数绑定到窗口
9. 进入 `glutMainLoop` 主循环
10. 在窗口绘制时调用 `renderScene` 函数，清除缓冲区，绘制图元，刷新窗口

```
1.  #include <GL/glut.h>
2.
3.  void init (void)
4.  {
5.      glClearColor (1.0, 1.0, 1.0, 0.0);
```

```

6.      glMatrixMode (GL_PROJECTION);      // Set projection parameters.
7.      gluOrtho2D (0.0, 200.0, 0.0, 150.0);
8.  }
9.
10. void lineSegment (void)
11. {
12.     glClear (GL_COLOR_BUFFER_BIT); // Clear display window.
13.
14.     // ...
15.
16.     glFlush ( );    // Process all OpenGL routines as quickly as possible.
17. }
18.
19. void main (int argc, char** argv)
20. {
21.     glutInit (&argc, argv);            // Initialize GLUT.
22.     glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB); // Set display mode
23.
24.     glutInitWindowPosition (50, 100);
25.     glutInitWindowSize (400, 300);
26.     glutCreateWindow ("An Example OpenGL Program");
27.
28.     init ( );
29.     glutDisplayFunc (lineSegment);
30.     glutMainLoop ( );
31. }

```

部分函数说明

glutMainLoop

该函数使程序进入事件循环，是 OpenGL 窗口程序的函数入口。它将根据需要调用回调函数，以处理图像刷新，鼠标按键输入等事件。

glutDisplayFunc

该函数接受一个无参数无返回值的函数作为事件绑定，用来自定义显示方法，该方法在需要绘制画面时被调用。

glMatrixMode

指定一个矩阵为当前矩阵模式。设置为 GL_PROJECTION 后的所有矩阵操作即为对投影矩阵的变换。

gluOrtho2D

设置正交裁剪窗口大小。

glClear

清除缓冲区数据。设置为 GL_COLOR_BUFFER_BIT 清除颜色缓冲区。

glClearColor

使用指定颜色清除缓冲区。

glFlush

强制刷新缓冲，保证绘图命令将被执行，而不是存储在缓冲区中等待其他的 OpenGL 命令。

glColor

设置绘制颜色。

参考文档

《计算机图形学第四版》附录“OpenGL 函数索引”提供书中所有用到的 OpenGL 函数的章节索引。

OpenGL 标准文档: <https://www.khronos.org/registry/OpenGL-Refpages/gl4/>,
OpenGL Wiki: https://www.khronos.org/opengl/wiki/Category:Core_API_Reference,
包含 gl 开头的函数和常用函数的规范。

FreeGLUT 接口文档: <http://freeglut.sourceforge.net/docs/api.php>, 包含 glut 开头的函数接口说明。FreeGLUT 与 GLUT 不同，课程所有试验都基于 GLUT 实现。实验提供的 OpenGL related files 即为 GLUT 版本，虽然 GLUT 版本代码较旧，且不再更新维护，但仍可以满足实验需求。

实验一 直线段绘制算法

实验目的

熟悉 OpenGL 图形绘制管线，使用 OpenGL 画点函数实现 DDA 算法和 Bresenham 算法。

实验内容

在教材¹的 6.1 节画线算法中，提供了 DDA 算法和 Bresenham 算法，包含计算步骤和代码实现。在此基础上，添加 main 函数及相关代码。

Main 函数²即为“环境验证”部分的代码，其功能为初始化 GLUT 窗口，绑定显示函数，进入主循环。

```
1. void main (int argc, char** argv)
2. {
3.     glutInit (&argc, argv);
4.     glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
5.     glutInitWindowPosition (50, 100);
6.     glutInitWindowSize (400, 300);
7.     glutCreateWindow ("DrawLine");
8.
9.     init ( );
10.    glutDisplayFunc (renderScene);
11.    glutMainLoop ( );
12. }
```

Init 函数为自定义的初始化函数，设置背景色和裁剪空间。

```
1. void init (void)
2. {
3.     glClearColor (0.0, 0.0, 0.0, 0.0);
4.     gluOrtho2D (0.0, 200.0, 0.0, 150.0);
5. }
```

¹ 教材统一指《计算机图形学第四版》

² C99 标准后 main 函数应当返回 int，否则编译器提示 warning，此处不做更改，保留示例代码原样

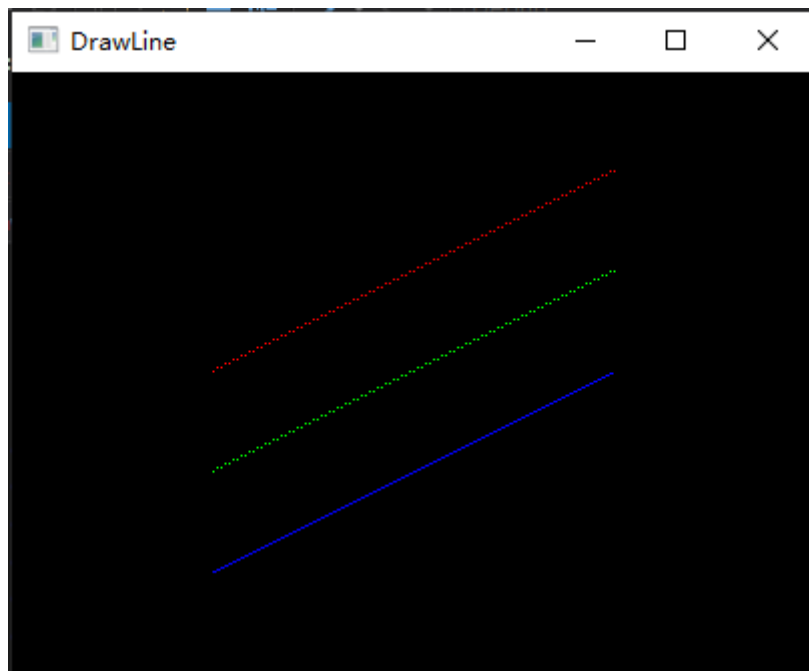
与环境验证部分不同的是，将显示绑定函数替换为 `renderScene` 函数，绘制两种算法的直线段，并与 OpenGL 的画线函数做对比。这里使用 `glColor3f` 设置三种区分颜色，`glFlush` 刷新缓冲区。OpenGL 提供的内置画线方法从 `glBegin( GL_LINES )` 开始，给出直线端点并绘制。

```
1.  void renderScene(void)
2.  {
3.      glClear (GL_COLOR_BUFFER_BIT); // Clear display window.
4.
5.      glColor3f (1, 0, 0);
6.      lineDDA(50, 75, 150, 125); // ( StartX, StartY, EndX, EndY )
7.
8.      glColor3f (0, 1, 0);
9.      lineBres(50, 50, 150, 100);
10.
11.     glColor3f (0, 0, 1);
12.     glBegin(GL_LINES);
13.         glVertex2i(50, 25);
14.         glVertex2i(150, 75);
15.     glEnd();
16.
17.     glFlush ( );
18. }
```

显示函数中的 `lineDDA` 函数和 `lineBres` 函数，参考教材自行实现。
这两个函数都使用 `setPixel` 函数画点。

```
1.  void setPixel (GLint xCoord, GLint yCoord)
2.  {
3.      glBegin (GL_POINTS);
4.          glVertex2i (xCoord, yCoord);
5.      glEnd ( );
6.  }
```

实验结果参考



思考：为什么两种画线算法和内置方法效果不同？

提示：坐标偏移和像素偏移不同。

实验二 圆生成算法

实验目的

熟悉 OpenGL 图形绘制管线，使用 OpenGL 画点函数实现中点画圆算法。

实验内容

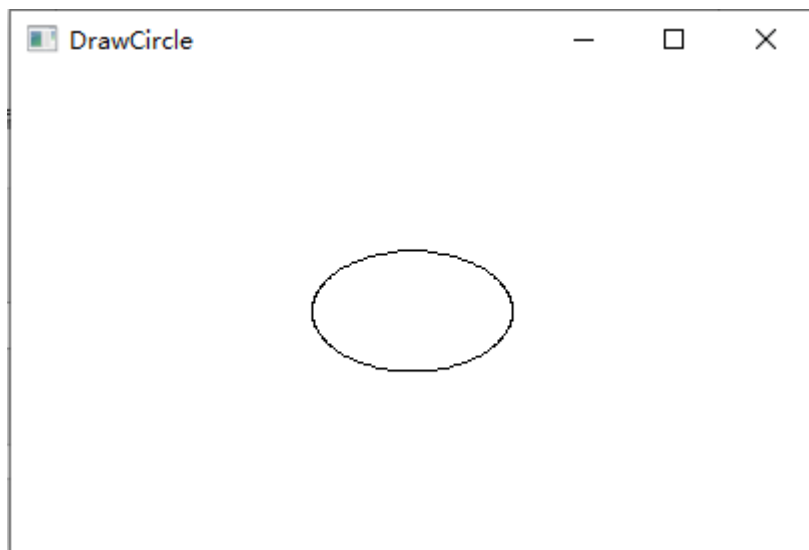
参考教材 6.4 节，实现 `screenPt` 屏幕点类和 `circleMidpoint` 中点画圆函数。基于前面的实验，我们不需要做太多更改，就能将图形绘制出来。在世界坐标系的原点（或其他位置），以给定半径绘制圆形。

```
1. void renderScene(void)
2. {
3.     glClear(GL_COLOR_BUFFER_BIT);
4.
5.     glColor3f(0, 0, 0);
6.     circleMidpoint(0, 0, 50);
7.
8.     glFlush();
9. }
```

为了居中显示圆形，将背景色调为白色，视图对准圆心，即世界坐标原点。

```
1. void init(void)
2. {
3.     glClearColor(1.0, 1.0, 1.0, 0.0);
4.     gluOrtho2D(-200.0, 200.0, -200.0, 200.0);
5. }
```

程序此时就可以正确绘制出圆形。但是在调整程序窗口时，由于窗口大小变化，而视图大小被硬编码，导致比例失准，圆形变成椭圆。



GLUT 提供 `glutReshapeFunc` 接口用于绑定窗口发生改变时的回调函数。在窗口大小发生改变时，将窗口宽度和高度传入绑定函数，进行自定义处理。

接口调用方式如下图所示。

```
1.  void myReshape(GLsizei w, GLsizei h)
2.  {
3.      // ...
4.  }
5.
6.  int main(int argc, char** argv)
7.  {
8.      // ...
9.      init();
10.     glutDisplayFunc(renderScene);
11.     glutReshapeFunc(myReshape);
12.     glutMainLoop();
13.     return 0;
14. }
```

该接口绑定的 `myReshape` 函数定义如下图所示。

```
1.  void myReshape(GLsizei w, GLsizei h)
2.  {
3.      glViewport(0, 0, w, h);
4.      glMatrixMode(GL_PROJECTION);
5.      glLoadIdentity();
6.  }
```



```
7.         if (w <= h)
8.             gluOrtho2D(-200.0, 200.0, -
200.0*(GLfloat)h / (GLfloat)w, 200.0*(GLfloat)h / (GLfloat)w);
9.         else
10.            gluOrtho2D(-
200.0*(GLfloat)w / (GLfloat)h, 200.0*(GLfloat)w / (GLfloat)h, -
200.0, 200.0);
11.
12.         glMatrixMode(GL_MODELVIEW);
13.         glLoadIdentity();
14.     }
```

函数根据新窗口的大小，重设视口大小（glViewport），然后对准焦点。

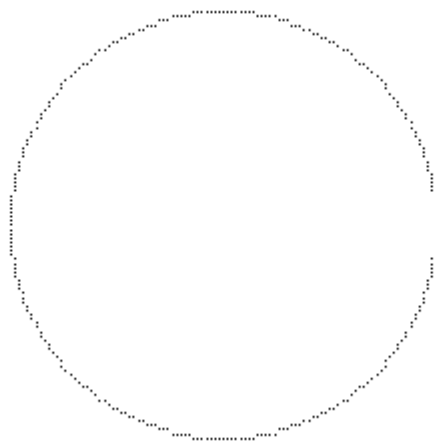
GL_PROJECTION 矩阵模式是将当前投影矩阵加载为待操作矩阵，后续的矩阵变换都是在对投影矩阵进行变换。函数 glLoadIdentity 是加载单位矩阵，对投影矩阵进行复位操作。

后面的对宽度和高度比较之后的选择分支，用来保证裁剪空间长宽比和视口长宽比相同，使图像不会变形（裁剪空间内的图像会缩放适应视口来显示）。

此时，init 函数只需要设置背景色。

```
1.     void init(void)
2.     {
3.         glClearColor(1.0, 1.0, 1.0, 0.0);
4.     }
```

实验结果参考



扩展：如何保证窗口扩大后，圆形曲线不会变成像素点？

提示：

- 使用 `glVertex2f` 画点，更改偏移量。
- 使用 `GL_LINE_STRIP` / `GL_LINE_LOOP` 画线逼近。³

³ 曲线绘制参考教材 4.5 节。此方法不同于圆的高速生成算法，需连续绘制。

实验三 填充算法

实验目的

熟悉 OpenGL 图形绘制管线，使用 OpenGL 画点函数实现边界填充算法和泛滥填充算法。

实验内容

图形填充

参考教材 6.13 节，实现对不规则边界区域的两种填充算法。

由于书中并没有规定“颜色”的数据结构和操作方法，在这里给出一种实现方案。

```
1.  struct Color
2.  {
3.      GLfloat r, g, b;
4.
5.      Color() = default;
6.      Color(GLfloat inR, GLfloat inG, GLfloat inB)
7.          : r(inR), g(inG), b(inB)
8.      {}
9.
10.     bool equal(Color otherColor) {
11.         if (r == otherColor.r && g == otherColor.g && b ==
otherColor.b)return true;
12.         return false;
13.     }
14. };
15.
16. void getPixel(GLint x, GLint y, Color& ptColor)
17. {
18.     glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, (GLvoid*)&ptColor);
19. }
```

Color 结构保存 RGB 三个浮点数，并提供构造函数和比较方法。

使用 `getPixel` 封装的 `glReadPixels` 获取某一像素位置的颜色，并将数据写回需要保存的位置。

后续改进此函数可能会导致栈溢出错误，问题根源在于

1. 世界坐标偏移和屏幕像素偏移不同，计算连通时容易越过填充边界。
2. 连通递归算法本身消耗大量栈空间，即使算法逻辑正确，在大面积填充时容易超出程序保留的可用栈空间⁴。

然后应当修改两种算法函数中的数据类型。

```
1. void boundaryFill4 (GLint x, GLint y, Color fillColor, Color borderColor)
2. {
3.     Color interiorColor;
4. }
```

为了暂时保证 `getPixel` 函数能正常运行，需要让裁剪空间坐标系和窗口屏幕像素坐标系重合，且程序窗口不支持缩放。

```
1. void init (void)
2. {
3.     glClearColor (0.0, 0.0, 0.0, 0.0);
4.     gluOrtho2D (0.0, 600.0, 0.0, 300.0);
5. }
6.
7. void main (int argc, char** argv)
8. {
9.     glutInitWindowSize (600, 300);
10. }
```

然后设置自定义的形状用于算法的颜色填充，这部分放在 `renderScene` 函数中。

边界填充算法只需要绘制出填充边界，然后给出边界内一点。

```
1. void renderScene(void)
2. {
3.     glColor3f (1, 1, 1);
```

⁴ 在 VS 项目属性 - 链接器 - 系统 - 堆栈保留大小设定了程序的可用栈空间，默认为 1MB。

```
4.      glBegin(GL_LINE_LOOP);
5.          glVertex2i(50, 100);
6.          glVertex2i(150, 100);
7.          glVertex2i(75, 200);
8.          glVertex2i(175, 200);
9.      glEnd();
10.     glFlush();
11.
12.     glColor3f (1, 0, 0);
13.     boundaryFill4(100, 101, Color(1, 0, 0), Color(1, 1, 1));
14.     glFlush();
15. }
```

泛滥填充算法可以使用多种不同颜色的边界，对内部单一颜色重新上色。示例代码绘制了三个不同颜色的三角形，中心围出的区域用白色填充。

```
1.  void renderScene(void)
2.  {
3.      glColor3f (1, 0, 0);
4.      glBegin(GL_POLYGON);
5.          glVertex2i(225, 100);
6.          glVertex2i(360, 100);
7.          glVertex2i(300, 70);
8.      glEnd();
9.      glColor3f (0, 1, 0);
10.     glBegin(GL_POLYGON);
11.         glVertex2i(310, 175);
12.         glVertex2i(250, 100);
13.         glVertex2i(200, 125);
14.     glEnd();
15.     glColor3f (0, 0, 1);
16.     glBegin(GL_POLYGON);
17.         glVertex2i(280, 150);
18.         glVertex2i(350, 100);
19.         glVertex2i(375, 125);
20.     glEnd();
21.     glFlush();
22.
23.     glColor3f (1, 1, 1);
24.     floodFill4(300, 101, Color(1, 1, 1), Color(0, 0, 0));
25.     glFlush();
```

```
26. }
```

可以额外设置一个 OpenGL 自动填充的多边形图元作为对比。

```
1. void renderScene(void)
2. {
3.     glColor3f (1, 1, 1);
4.     glBegin(GL_POLYGON);
5.         glVertex2i(450, 100);
6.         glVertex2i(550, 100);
7.         glVertex2i(475, 200);
8.         glVertex2i(575, 200);
9.     glEnd();
10. }
```

在填充算法内部，还可以添加 `glFlush` 调用，每次绘制点后刷新缓冲区，可以看到动态的填充过程。

栈空间分析

为了说明教材 6.13 节中关于填充算法对栈空间的大量需求，对上述程序代码作如下改动。

已知 VC++ 项目程序的默认栈保留空间为 1MB，如图所示，程序在装入内存时会声明自己需求的栈空间大小，由系统进行分配。

添加全局变量：

- `stackMemSize` 储存程序实际占用的最大栈内存空间
- `stackMemBasePtr` 储存程序的栈底基址
- `memInfo` 是调用 `VirtualQuery` 函数所需的储存结构，函数调用结束时内存信息被写入，该部分需引入 `Windows.h` 头文件⁵

在 `main` 函数中添加一个局部变量 `stackBase`，以此作为虚拟内存查询函数 `VirtualQuery` 的寻址目标，将结果中的基址写入 `stackMemBasePtr`。

```
1.  int stackMemSize = 0;
2.  void* stackMemBasePtr = nullptr;
3.  MEMORY_BASIC_INFORMATION memInfo;
4.
5.  void main (int argc, char** argv)
6.  {
7.      int stackBase = 0;
8.      if (VirtualQuery(&stackBase, &memInfo, sizeof(memInfo))) {
9.          stackMemBasePtr = memInfo.BaseAddress;
10.         printf("StackBase : 0x%p\n", memInfo.BaseAddress);
11.     }
12.     // ...
13. }
```

在算法内部借助 `Color` 结构进行寻址，使用内存栈底的高地址减去低地址，获取最大栈深度，将计算结果储存到 `stackMemSize`。

```
1.  void boundaryFill4 (GLint x, GLint y, Color fillColor, Color borderColor)
2.  {
3.      Color interiorColor;
4.
5.      if (VirtualQuery((LPCVOID)&interiorColor, &memInfo, sizeof(memInfo))) {
6.          int stackDeep = (int)stackMemBasePtr - (int)memInfo.BaseAddress;
7.          if (stackDeep > stackMemSize) {
8.              stackMemSize = stackDeep;
9.          }
10.     }
```

⁵ 也可以直接使用取地址符号 `&` 来获取地址并计算，最终计算结果相近。


```

11.      // ...
12.  }

```

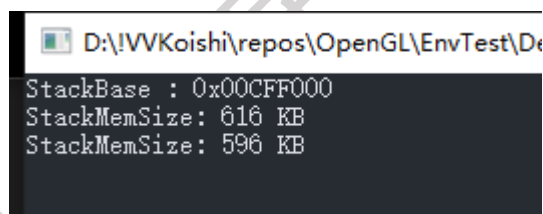
在填充算法调用前后，对 `stackMemSize` 进行初始化，并打印实际使用的栈空间大小。

```

1.      stackMemSize = 0;
2.
3.      glColor3f (1, 0, 0);
4.      boundaryFill4(100, 101, Color(1, 0, 0), Color(1, 1, 1));
5.      glFlush();
6.
7.      printf("StackMemSize: %d KB \n", stackMemSize / 1024);

```

在命令行窗口可以看到，在示例代码的条件下，两种填充算法对栈空间的实际需求分别为 616KB 和 596KB，而程序默认栈大小为 1MB，因此当需要填充的面积较大时，算法很容易栈溢出（比如填充一个平行四边形，而不是交叉后形成的小三角形）。



```

D:\!VVKoishi\repos\OpenGL\EnvTest\De
StackBase : 0x00CFF000
StackMemSize: 616 KB
StackMemSize: 596 KB

```

扫描线 4 连通算法

为了对区域填充算法进行栈空间优化，使用扫描线方式填充水平像素区段，详见教材 6.13 节。

在扫描线方式的 4 连通区域填充算法中，维护 `pointStack` 填充起点栈，对栈尾元素点位置进行线段填充，在每次线段填充完成后，对该条线段上下连通的待填充区域的起点压入栈中，直到栈为空，填充算法结束。

```

1.      struct Point
2.      {
3.          GLdouble x;
4.          GLdouble y;

```

```

5.     };
6.
7.     void scanLineFilling(GLdouble x, GLdouble y, Color fillColor, Color interiorColor)
8.     {
9.         std::vector<Point> pointStack; //使用 vector 需要
#include<vector>
10.
11.         fillLine(x, y, fillColor, interiorColor, pointStack);
12.
13.         while (!pointStack.empty())
14.         {
15.             GLdouble popX = pointStack.back().x;
16.             GLdouble popY = pointStack.back().y;
17.             pointStack.pop_back();
18.             fillLine(popX, popY, fillColor, interiorColor, pointStack)
;
19.         }
20.     }

```

函数 `fillLine` 负责填充该位置所在的线段，并且在填充完成后，将连通的待填充区域压入栈中。

```

1.     void fillLine(GLdouble x, GLdouble y, Color fillColor, Color interiorColor, std::vector<Point>& pointStack)
2.     {
3.         GLdouble startX = x;
4.         GLdouble startY = y;
5.
6.         GLdouble leftX = x;
7.         GLdouble rightX = x;
8.
9.         // 先向右后向左填充
10.        Color oldColor;
11.        getPixel(x, y, oldColor);
12.        while (oldColor.equal(interiorColor) && !(oldColor.equal(fillColor)))
13.        {
14.            setPixel(x, y);
15.            x = x + dx;
16.            getPixel(x, y, oldColor);
17.        }

```

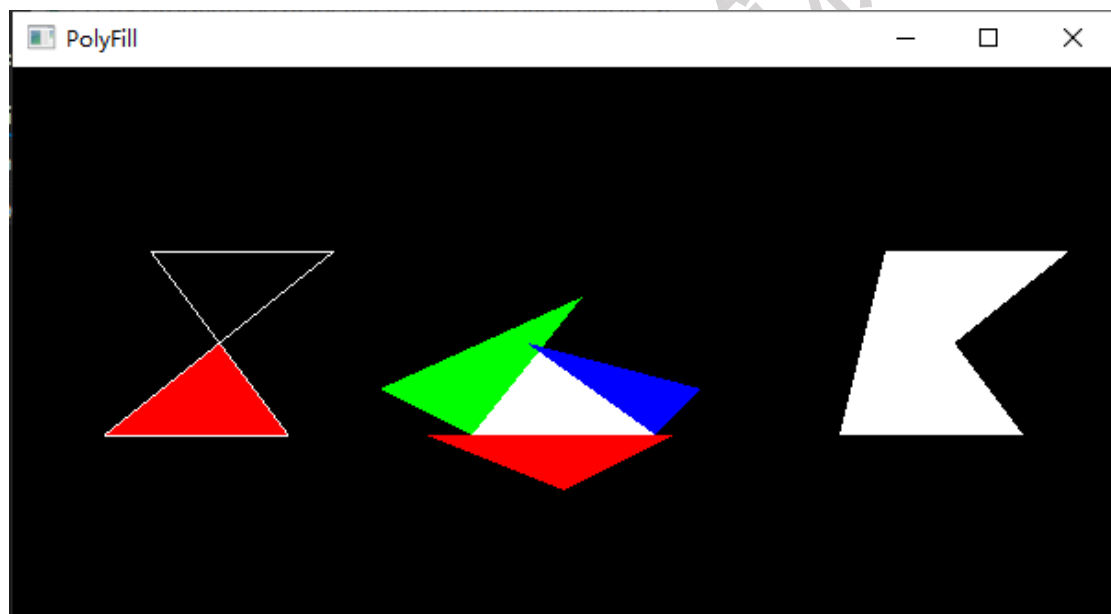
```

18.     rightX = x - dx;
19.     x = startX - dx;
20.     getPixel(x, y, oldColor);
21.     while (oldColor.equal(interiorColor) && !(oldColor.equal(fillC
olor)))
22.     {
23.         setPixel(x, y);
24.         x = x - dx;
25.         getPixel(x, y, oldColor);
26.     }
27.     leftX = x + dx;
28.
29.     // 从左至右,先下后上, 读取 4 连通
30.     x = leftX;
31.     y = startY - dy;
32.     bool startContinue = false;
33.     getPixel(x, y, oldColor);
34.     while (x <= rightX)
35.     {
36.         if (!startContinue) {
37.             if (oldColor.equal(interiorColor) && !(oldColor.equal(fi
llColor)){
38.                 pointStack.push_back({ x,y });
39.                 startContinue = true;
40.             }
41.         }
42.         else {
43.             if (!(oldColor.equal(interiorColor)) || oldColor.equal
(fillColor)) startContinue = false;
44.         }
45.         x = x + dx;
46.         getPixel(x, y, oldColor);
47.     }
48.
49.     x = leftX;
50.     y = startY + dy;
51.     startContinue = false;
52.     getPixel(x, y, oldColor);
53.     while (x <= rightX)
54.     {
55.         if (!startContinue) {
56.             if (oldColor.equal(interiorColor) && !(oldColor.equal(
fillColor))) {
57.                 pointStack.push_back({ x,y });

```

```
58.         startContinue = true;
59.     }
60. }
61. else {
62.     if (!(oldColor.equal(interiorColor)) || oldColor.equal
(fillColor)) startContinue = false;
63. }
64. x = x + dx;
65. getPixel(x, y, oldColor);
66. }
67. }
```

实验结果参考



实验四 立方体绘制

实验目的

熟悉 OpenGL 三维图形绘制，使用 OpenGL 函数实现绘制立方体。

实验内容

参考教材 4.7.6 节理解常用三维对象表示法，教材 4.9 节学习绘制简单三维图形，教材 13.2 节了解 GLUT 预定义的多面体函数，实践三维图形几何建模原理，为后续几何变换等实验打下基础。

绘制立方体

绘制立方体只需要定义模型的 8 个顶点（vertex）数据，然后由这些顶点构建 6 个面片（face）。

顶点数组 vertices 储存了顶点坐标数据，polygon 函数使用顶点下标绘制一个四边形线框，drawcube 函数借助其绘制立方体的 6 个面。

```
1. GLfloat vertices[8][3] = {
2.     {-1.0, -1.0, -1.0}, {1.0, -1.0, -1.0}, {1.0, 1.0, -1.0}, {-1.0, 1.0, -1.0},
3.     {-1.0, -1.0, 1.0}, {1.0, -1.0, 1.0}, {1.0, 1.0, 1.0}, {-1.0, 1.0, 1.0}
4. };
5.
6. void polygon(int a, int b, int c, int d)
7. {
8.     /* draw a polygon via list of vertices */
9.     glColor3f(1.0, 1.0, 1.0);
10.    glBegin(GL_LINE_LOOP);
11.        glVertex3fv(vertices[a]);
12.        glVertex3fv(vertices[b]);
13.        glVertex3fv(vertices[c]);
14.        glVertex3fv(vertices[d]);
15.    glEnd();
16. }
17.
18. void drawcube(void)
19. {
```

```

20.  /* map vertices to faces */
21.  polygon(1, 0, 3, 2);
22.  polygon(3, 7, 6, 2);
23.  polygon(7, 3, 0, 4);
24.  polygon(2, 6, 5, 1);
25.  polygon(4, 5, 6, 7);
26.  polygon(5, 4, 0, 1);
27. }

```

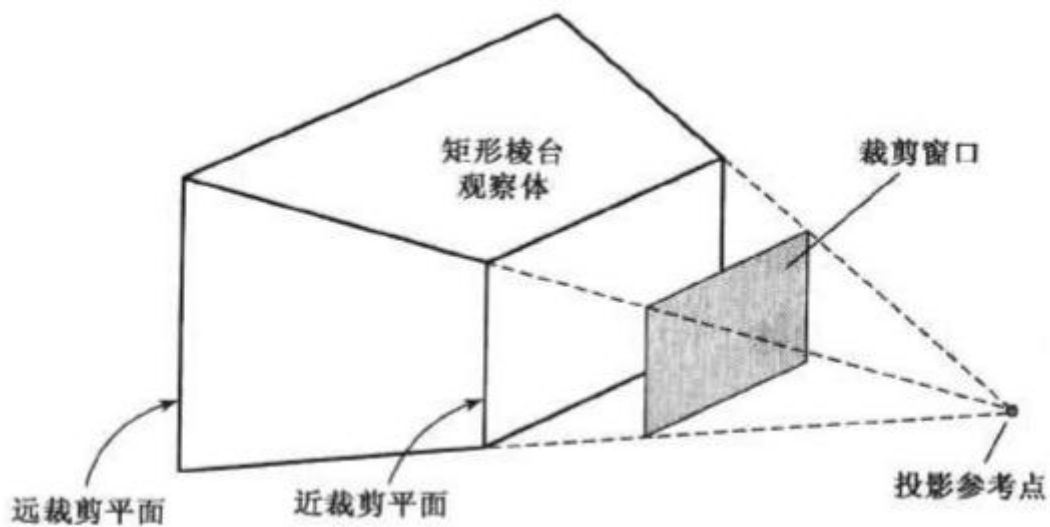
为了显示该三维对象，需要合理设置透视投影参数。`glFrustum` 函数⁶创建了一个平截头视椎体，其函数签名如下。

```

1. void glFrustum (GLdouble left, GLdouble right, GLdouble bottom, GLdouble top
    , GLdouble zNear, GLdouble zFar);

```

如下图所示⁷，前 4 项分别对应平截锥体在近平面的四条边界到近平面中心的距离，`zNear` 和 `zFar` 分别对应近平面中心和远平面中心到投影参考点的距离。超出视椎体的部分会被裁剪，不会显示在窗口。



投影参考点默认为世界原点，从原点看向负 z 轴方向⁸。本实验将立方体平

⁶ 参考教材 10.10 节，也可以使用 `glOrtho` 正交投影和 `gluPerspective` 对称透视投影。

⁷ 参考教材 10.8.4 节。

⁸ OpenGL 世界坐标系为右手坐标系，而归一化设备坐标是左手系，可以看作对 z 轴正方向取反。

移到了 (0,0,viewxform_z) 的位置进行绘制,可以保证立方体位于视锥体中。

```
1. float mynear = 3.0;    /* near clipping plane in eye coords */
2. float myfar = 7.0;    /* far clipping plane in eye coords */
3. float viewxform_z = -5.0;
4.
5. void display(void)
6. {
7.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
8.
9.     /* view transform */
10.    glLoadIdentity();
11.    glTranslatef(0.0, 0.0, viewxform_z);
12.
13.    drawcube();
14.
15.    glutSwapBuffers();
16.    glutPostRedisplay();
17. }
18.
19. void myReshape(int w, int h)
20. {
21.    glViewport(0, 0, w, h);
22.    glMatrixMode(GL_PROJECTION);
23.    glLoadIdentity();
24.
25.    glFrustum(-2.5, 2.5, -2.5, 2.5, mynear, myfar);
26.
27.    glMatrixMode(GL_MODELVIEW);
28. }
```

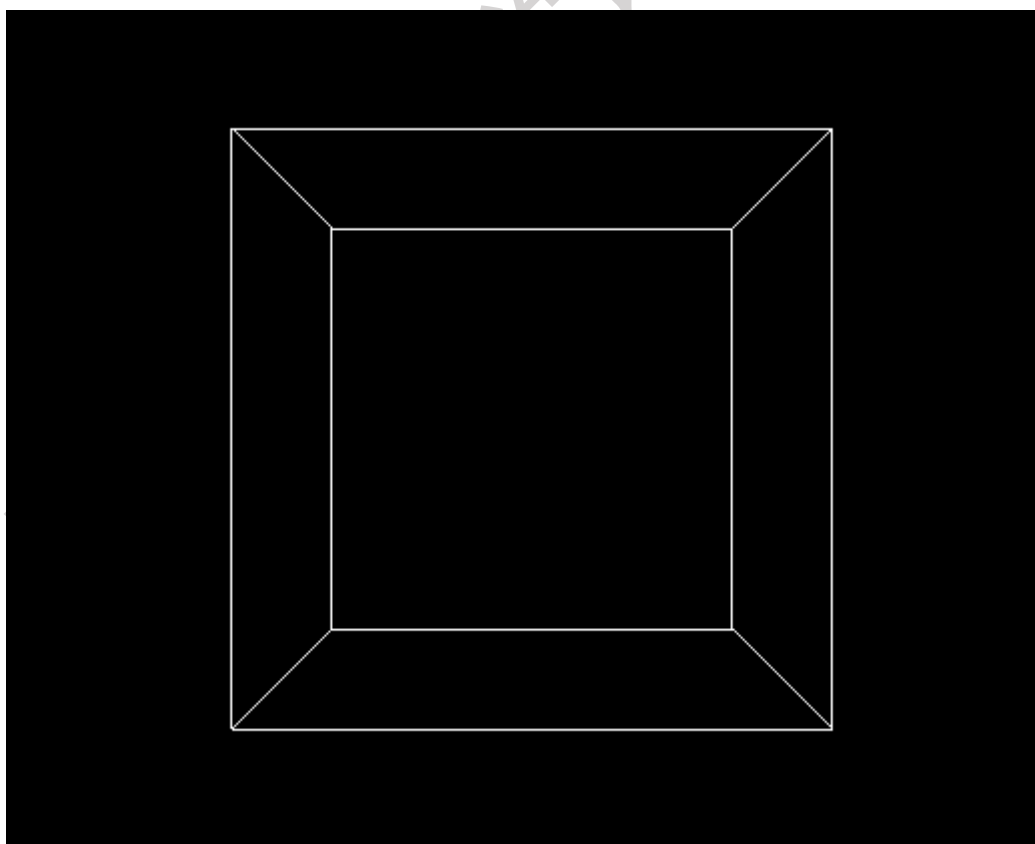
最后在 main 函数中绑定相关函数。因为需要使用深度缓冲区,所以设置了 GLUT_DEPTH。GLUT_DOUBLE 用于设置双缓冲区,相对于单缓冲区,双缓冲使用额外一倍空间的缓冲区存储下一帧将要绘制的图像,在显示时交换两个缓冲区,因此原来的 glFlush 函数只能刷新当前缓冲区显示,需改用 glutSwapBuffers 函数交换两个缓冲区内容。

因为使用了 glTranslatef 函数平移对象,需要保证在后续渲染时对象不会累积平移量,因此使用 glLoadIdentity 复位。在 display 函数末尾使用 glutPostRedisplay 函数,该函数作用为通知 GLUT 程序重新绘制窗口(即重新调

用 `display` 函数)，其在后续实验中制作动画部分被经常使用。

```
1. int main(int argc, char **argv)
2. {
3.     glutInit(&argc, argv);
4.     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
5.     glutInitWindowSize(1000, 1000);
6.     glutCreateWindow("Example");
7.
8.     //init();
9.     glutReshapeFunc(myReshape);
10.    glutDisplayFunc(display);
11.    glutMainLoop();
12.    return 0;
13. }
```

此时，程序可以生成如下立方体透视线框，这种由多个网格（grid）组成的模型对象称为网格体（mesh）。



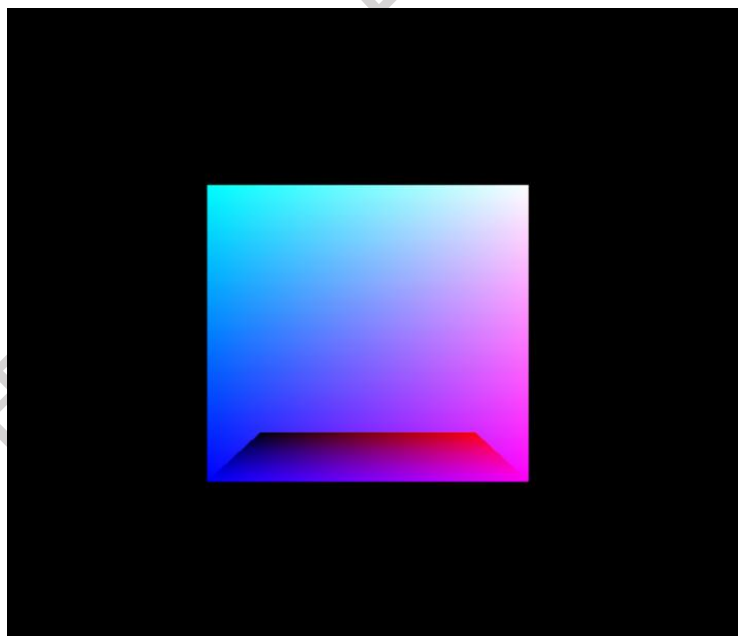
在三维世界中，仅仅有线框图是不够的，网格体需要绘制面片表面，而不是只绘制面片边缘。我们只需要修改 `polygon` 函数的绘制方法即可。

将 `GL_LINE_LOOP` 改为 `GL_POLYGON`，可以绘制多边形表面。

```
1. void polygon(int a, int b, int c, int d)
2. {
3.     /* draw a polygon via list of vertices */
4.     glBegin(GL_POLYGON);
5.         glVertex3fv(vertices[a]);
6.         glVertex3fv(vertices[b]);
7.         glVertex3fv(vertices[c]);
8.         glVertex3fv(vertices[d]);
9.     glEnd();
10. }
```

但这样绘制出的表面默认为白色的。在三维对象表示法中，颜色通常是顶点属性的一部分，而面片颜色由组成面片的顶点和着色模式确定。

分别定义 8 个顶点的颜色，并且在绘制顶点前，设置顶点颜色覆盖默认值，立方体就会使用默认的 `GL_SMOOTH` 平滑着色模式对表面颜色求解双线性插值。



定义顶点颜色并且进行绘制后得出的效果如上图所示，可以看到立方体的被遮挡面也被显示了出来。为了正确剔除被遮挡面，需要启用深度测试 `GL_DEPTH_TEST`，写在 `init` 函数中。

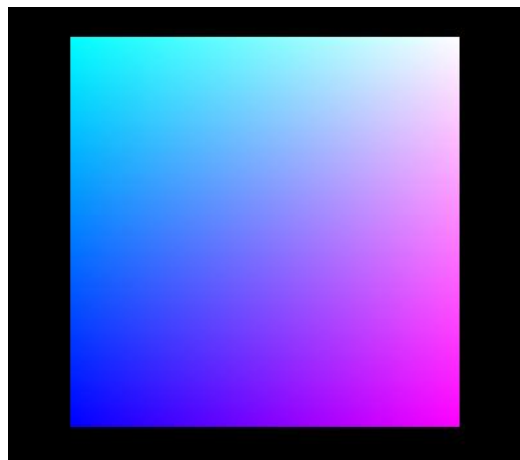
```
1. GLfloat colors[8][3] = {
2.     {0.0,0.0,0.0}, {1.0,0.0,0.0}, {1.0,1.0,0.0}, {0.0,1.0,0.0},
3.     {0.0,0.0,1.0}, {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}
4. };
5.
6. void polygon(int a, int b, int c, int d)
7. {
8.     /* draw a polygon via list of vertices */
9.     glBegin(GL_POLYGON);
10.
11.     glColor3fv(colors[a]);
12.     glVertex3fv(vertices[a]);
13.
14.     glColor3fv(colors[b]);
15.     glVertex3fv(vertices[b]);
16.
17.     glColor3fv(colors[c]);
18.     glVertex3fv(vertices[c]);
19.
20.     glColor3fv(colors[d]);
21.     glVertex3fv(vertices[d]);
22.
23.     glEnd();
24. }
25.
26. void init(void)
27. {
28.     glEnable(GL_DEPTH_TEST); /* Enable hidden--surface--removal */
29. }
30.
31. int main(int argc, char **argv)
32. {
33.     // ...
34.     init();
35.     // ...
36. }
```

重新编译运行，可以看到实验结果参考中的图像，一个五彩斑斓的立方体。

读者可以尝试将默认的 GL_SMOOTH 着色模式修改为 GL_FLAT，查看立方体变化。

```
1. void init(void)
2. {
3.     glShadeModel(GL_FLAT);
4.     //glShadeModel(GL_SMOOTH);
5.     glEnable(GL_DEPTH_TEST); /* Enable hidden--surface--removal */
6. }
```

实验结果参考



实验五 图形几何变换

实验目的

熟悉 OpenGL 几何变换函数，使用 OpenGL 函数实现图形的平移、旋转和缩放效果。

实验内容

在上一节的基础上，参考教材 7.9 节和 9.8 节，学习使用 OpenGL 几何变换函数和矩阵操作。

为了显示 3D 图形，需要启用 GLUT 窗口程序的深度缓冲区(GLUT_DEPTH)，深度测试遮挡剔除 (GL_DEPTH_TEST)，并设置观察裁剪空间。

```
1.  float mynear = 3.0;      /* near clipping plane in eye coords */
2.  float myfar = 7.0;      /* far clipping plane in eye coords */
3.  float viewxform_z = -5.0;
4.
5.  void init(void)
6.  {
7.      glLineWidth(3.0);
8.      glEnable(GL_DEPTH_TEST); /* Enable hidden--surface--removal */
9.  }
10.
11. void myReshape(int w, int h)
12. {
13.     glViewport(0, 0, w, h);
14.     glMatrixMode(GL_PROJECTION);
15.     glLoadIdentity();
16.     glOrtho(-5.0, 5.0, -5.0, 5.0, mynear, myfar);
17.
18.     glMatrixMode(GL_MODELVIEW);
19. }
20.
21. int main(int argc, char **argv)
22. {
23.     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
24.     glutInitWindowSize(1000, 1000);
25.     // ...
```

26. }

创建一个正方体对象，用于演示图形变换。该对象储存正方体的所有顶点坐标，顶点颜色，以及正方体的变换矩阵。变换矩阵描述模型局部坐标系相对于世界坐标系的变换操作，用于储存阶段性变换，实现变换的累积。

```

1.  GLfloat vertices[][3] = {
2.      {-1.0,-1.0,-1.0}, {1.0,-1.0,-1.0}, {1.0,1.0,-1.0}, {-1.0,1.0,-
3.      1.0},
4.      {-1.0,-1.0,1.0}, {1.0,-1.0,1.0}, {1.0,1.0,1.0}, {-1.0,1.0,1.0}
5.  };
6.  GLfloat colors[][3] = {
7.      {0.0,0.0,0.0}, {1.0,0.0,0.0}, {1.0,1.0,0.0}, {0.0,1.0,0.0},
8.      {0.0,0.0,1.0}, {1.0,0.0,1.0}, {1.0,1.0,1.0}, {0.0,1.0,1.0}
9.  };
10.
11. GLfloat objectXform[4][4] = {
12.     {1.0, 0.0, 0.0, 0.0},
13.     {0.0, 1.0, 0.0, 0.0},
14.     {0.0, 0.0, 1.0, 0.0},
15.     {0.0, 0.0, 0.0, 1.0}
16. };

```

使用 `colorcube` 函数初始化正方体对象，创建正方体的 6 个面。建面函数 `polygon` 以画线或者画面的绘制方式，围成正方体。

```

1.  bool drawLines = false;
2.
3.  void polygon(int a, int b, int c, int d, int face)
4.  {
5.
6.      /* draw a polygon via list of vertices */
7.
8.      if (drawLines) {
9.          glColor3f(1.0, 1.0, 1.0);
10.         glBegin(GL_LINE_LOOP);
11.         glVertex3fv(vertices[a]);
12.         glVertex3fv(vertices[b]);
13.         glVertex3fv(vertices[c]);

```

```

14.         glVertex3fv(vertices[d]);
15.         glEnd();
16.     }
17.     else {
18.         glBegin(GL_POLYGON);
19.
20.         glColor3fv(colors[a]);
21.         glVertex3fv(vertices[a]);
22.
23.         glColor3fv(colors[b]);
24.         glVertex3fv(vertices[b]);
25.
26.         glColor3fv(colors[c]);
27.         glVertex3fv(vertices[c]);
28.
29.         glColor3fv(colors[d]);
30.         glVertex3fv(vertices[d]);
31.
32.         glEnd();
33.     }
34. }
35.
36. void colorcube(void)
37. {
38.     /* map vertices to faces */
39.     polygon(1, 0, 3, 2, 0);
40.     polygon(3, 7, 6, 2, 1);
41.     polygon(7, 3, 0, 4, 2);
42.     polygon(2, 6, 5, 1, 3);
43.     polygon(4, 5, 6, 7, 4);
44.     polygon(5, 4, 0, 1, 5);
45. }

```

绑定到视口的显示函数对正方体图形进行几何变换操作。

在 `display` 函数中，首先清理之前的缓冲区，之后对默认的模式视图矩阵模式 `GL_MODELVIEW` 进行计算，将模型视图在 `z` 轴方向后退到裁剪空间可以观察到整个正方体的位置。

函数 `glPushMatrix` 用于将当前矩阵结果压入栈中，在进行一系列其他计算后，再使用 `glPopMatrix` 将之前的结果读取出来。对所有矩阵操作使用这两个函数包裹，并加载单位矩阵，可以防止矩阵操作的累积效应。

平移变换使用三角函数和时间步进来模拟正方体的跳动，对计算得出的

deltaHeight 添加到 glTranslatef 中的 y 轴上，然后将矩阵结果乘以对象模型的变换矩阵，最后把值存储回变换矩阵。旋转和缩放变换可以仿照该模式自行设计。

在计算得出最终的变换矩阵 objectXform 之后，用当前模型视图矩阵（世界坐标系 z 轴负方向平移 5 单位，即正方体中心）乘以变换矩阵，在得到的新坐标系下绘制正方体。

```

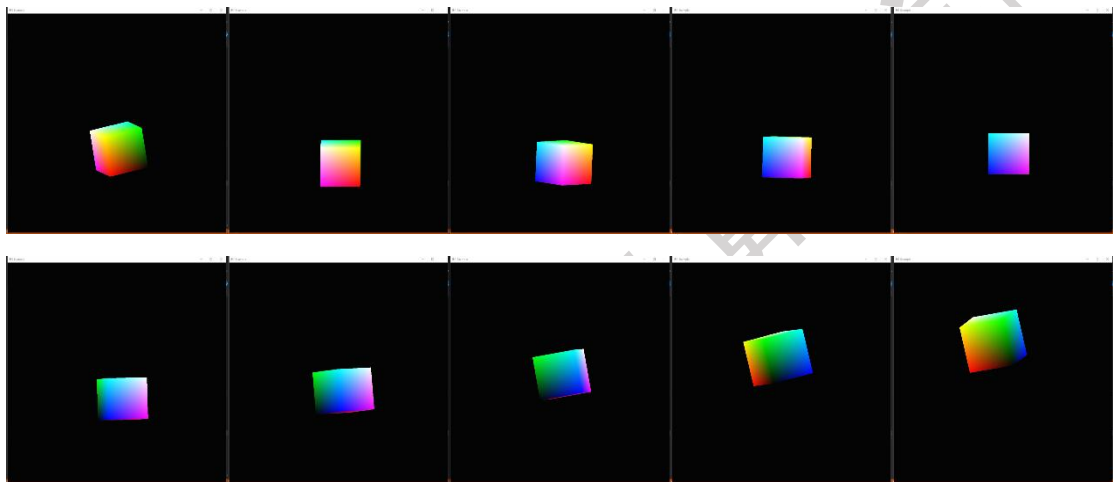
1.  float time = 0.0;
2.  float deltaTime = 0.01;
3.
4.  void display(void)
5.  {
6.      glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
7.
8.      /* view transform */
9.      glLoadIdentity();
10.     glTranslatef(0.0, 0.0, viewxform_z);
11.
12.     //////////////////////////////////////
13.     // 平移
14.     glPushMatrix();
15.     glLoadIdentity();
16.     GLfloat deltaHeight = (GLfloat) ( sin(time + deltaTime) - sin(time)
) * 2; //使用三角函数需要添加头文件 math.h
17.     glTranslatef(0.0, deltaHeight, 0.0);
18.     glMultMatrixf((GLfloat *)objectXform);
19.     glGetFloatv(GL_MODELVIEW_MATRIX, (GLfloat *)objectXform);
20.     glPopMatrix();
21.
22.     // 旋转 ...
23.     // 缩放 ...
24.     //////////////////////////////////////
25.     // 绘制立方体
26.     glPushMatrix();
27.     glMultMatrixf((GLfloat *)objectXform);
28.     colorcube();
29.     glPopMatrix();
30.     glFlush();
31.     glutSwapBuffers();
32.
33.     Sleep(deltaTime * 1000); //使用 Sleep 函数需要添加头文件 Windows.h

```

```
34.     time += deltaTime;  
35.     glutPostRedisplay();  
36. }
```

最后使用 `glutSwapBuffers` 函数对双缓冲区 `GL_DOUBLE` 程序刷新显示窗口。令程序延迟 `deltaTime` 秒⁹，让时间步进 `deltaTime`，并使用 `glutPostRedisplay` 函数通知程序重新绘制，即再次调用显示函数。

实验结果参考



⁹ 需要引用 `Windows.h` 头文件。其他方法如使用 `glutTimerFunc` 定时器绑定回调函数。动画可参考教材 12.10 节。

实验六 图形用户交互

实验目的

熟悉 OpenGL 用户接口函数，使用 OpenGL 函数实现鼠标和键盘的用户输入处理。

实验内容

变换模式

在前一节的基础上，参考教材 20.6 节和 20.7 节，学习使用 OpenGL 交互输入函数和菜单功能。

基于前面实验的部分内容，添加额外的用户输入处理模块。将代码中的时间和动画部分剔除，对平移、旋转和缩放三种变换使用变换模式变量(TransMode)判断分支。使用 `angle` 和 `axis[3]` 变量存储变化的值，动态的切换变换模式并修改变化值，程序再次调用显示函数时（使用 `glutPostRedisplay` 函数）就可以按输入处理图形变换。

```
1.  enum TransMode {
2.      Translate,
3.      Rotate,
4.      Scale
5.  };
6.  TransMode transModeSelected = Rotate;
7.  float angle = 0.0, axis[3];
8.
9.  void display(void)
10. {
11.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
12.
13.     /* view transform */
14.     glLoadIdentity();
15.     glTranslatef(0.0, 0.0, viewxform_z);
16.
17.     //////////////////////////////////////
18.     if (transModeSelected == Translate)
```

```

19.     {
20.         // 平移
21.         glPushMatrix();
22.         glLoadIdentity();
23.         glTranslatef(axis[0], axis[1], axis[2]);
24.         glMultMatrixf((GLfloat *)objectXform);
25.         glGetFloatv(GL_MODELVIEW_MATRIX, (GLfloat *)objectXform);
26.         glPopMatrix();
27.     }
28.     else if (transModeSelected == Rotate)
29.     {
30.         // 旋转，此部分需要自己实现
31.     }
32.     else if (transModeSelected == Scale)
33.     {
34.         // 缩放，此部分需要自己实现
35.     }
36.     ////////////////////////////////////////
37.     glPushMatrix();
38.     glMultMatrixf((GLfloat *)objectXform);
39.     colorcube();
40.     glPopMatrix();
41.
42.     glFlush();
43.     glutSwapBuffers();
44. }

```

在 main 函数中设置鼠标函数，鼠标移动函数，键盘函数的绑定。

```

1. void mouseButton(int button, int state, int x, int y);
2. void mouseMotion(int x, int y);
3. void keyboard(unsigned char key, int x, int y);
4.
5. int main(int argc, char **argv)
6. {
7.     // ...
8.     glutMouseFunc(mouseButton);
9.     glutMotionFunc(mouseMotion);
10.    glutKeyboardFunc(keyboard);
11.    // ...

```

```
12. }
```

鼠标输入

函数 `mouseButton` 管理鼠标按键的响应事件。包含两个选择分支，第一个分支判断鼠标左键、中键和右键，第二个分支判断按下和弹起。一次完整的鼠标操作作为“按下 – 移动 – 弹起”，如果不移动鼠标就是一次“点击”。该函数转交给 `startMotion` 和 `stopMotion` 函数处理“按下”和“弹起”的动作。

```
1. void mouseButton(int button, int state, int x, int y)
2. {
3.     switch (button) {
4.         case GLUT_LEFT_BUTTON:
5.             break;
6.         default:
7.             return;
8.     }
9.     switch (state) {
10.        case GLUT_DOWN:
11.            startMotion(x, y);
12.            break;
13.        case GLUT_UP:
14.            stopMotion(x, y);
15.            break;
16.    }
17. }
```

鼠标按下动作储存初始坐标信息。

鼠标弹起动作打印变换后的对象变换矩阵 `objectXform`，数据会被打印到命令行窗口中。

```
1. int lastX, lastY;
2.
3. void startMotion(int x, int y)
4. {
5.     lastX = x;
6.     lastY = y;
7. }
```

```

8.
9.  void stopMotion(int x, int y)
10. {
11.     angle = 0.0f;
12.     memset(axis, 0, sizeof(axis));
13.     printf("objectXform:\n");
14.     for (int i = 0; i < 4; i++)
15.     {
16.         printf("\t");
17.         for (int j = 0; j < 4; j++)
18.         {
19.             printf("%.2f\t", objectXform[i][j]);
20.         }
21.         printf("\n");
22.     }
23. }

```

鼠标移动动作由 `glutMotionFunc` 的绑定函数 `mouseMotion` 实现。该函数计算鼠标移动后的新坐标和旧坐标的距离，然后对不同变换模式计算变化值储存到 `angle` 和 `axis[3]` 变量中，最后调用 `glutPostRedisplay` 重新显示。

在变化值的计算时，需要使用窗口长宽信息，由窗口缩放的绑定函数 `myReshape` 写进全局变量。

在计算 `dx` 和 `dy` 时，需要注意窗口坐标系和世界坐标系的 `y` 轴正负方向是相反的。代码中 `dx * (10.0f/winWidth)` 可以理解为，当 `dx` 的累积量达到整个窗口宽度时，`axis[0]` 取值 `10.0f`，即向 `x` 轴平移 10 单位长度。

旋转和缩放部分可以参考该模式自行实现。需注意旋转要额外设置 `angle`，缩放在计算时不加 1，在调用时加 1，因为在程序没有执行鼠标移动函数时，`axis` 默认值为全 0，映射到 `glScalef` 应该为全 1。

```

1.  int winWidth, winHeight;
2.
3.  void mouseMotion(int x, int y)
4.  {
5.      float dx = x - lastX;
6.      float dy = y - lastY;
7.      if (!dx && !dy) return;
8.
9.      if (transModeSelected == Translate) {
10.         axis[0] = dx * (10.0f/winWidth);
11.         axis[1] = -dy * (10.0f/winHeight);

```

```
12.     axis[2] = 0;
13. }
14. else if (transModeSelected == Rotate) {
15.     angle = 3.0;
16.     axis[0] = dy * (360.0f/winHeight);
17.     axis[1] = dx * (360.0f/winWidth);
18.     axis[2] = 0;
19. }
20. else if (transModeSelected == Scale) {
21.     axis[0] = dx * (4.0f/winWidth);
22.     axis[1] = -dy * (4.0f/winHeight);
23.     axis[2] = 0;    // 在调用时需要 +1
24. }
25.
26.     lastX = x;
27.     lastY = y;
28.     glutPostRedisplay();
29. }
30.
31. void myReshape(int w, int h)
32. {
33.     winWidth = w;
34.     winHeight = h;
35.     // ...
36. }
```

键盘输入

键盘绑定函数 `keyboard` 将按键信息转发到 `userEventAction` 函数，针对不同按键执行事件，该函数在之后的菜单映射时还会发挥作用。

```
1. void keyboard(unsigned char key, int x, int y)
2. {
3.     userEventAction(key);
4. }
```

函数 `userEventAction` 按 `key` 值映射执行事件，最后调用 `glutPostRedisplay` 重新绘制画面。示例代码注册了键盘上的如下按键：

- 数字键 0，重置对象的变换矩阵

- 按键 r, 切换到旋转模式
- 按键 t, 切换到平移模式
- 按键 s, 切换到缩放模式
- 数字键 1, 切换线框图/多边形
- 按键 ESC (ASCII 码值为 27), 退出程序

```
1. void userEventAction(int key) {
2.     switch (key) {
3.         case '0':
4.             angle = 0.0f;
5.             memset(axis, 0, sizeof(axis));
6.             memset(objectXform, 0, sizeof(objectXform)); // 变换矩阵重
置
7.             for (int i = 0; i < 4; i++) {
8.                 for (int j = 0; j < 4; j++) {
9.                     if (i == j) {
10.                        objectXform[i][j] = 1.0f;
11.                    }
12.                }
13.            }
14.            break;
15.        case 'r': // 旋转
16.            transModeSelected = Rotate;
17.            break;
18.        case 't': // 平移
19.            transModeSelected = Translate;
20.            break;
21.        case 's': // 缩放
22.            transModeSelected = Scale;
23.            break;
24.        case '1': // 线框图/多边形
25.            drawLines = !drawLines;
26.            break;
27.        case 27: // ESC 键 (ASCII: 27) 退出
28.            exit(0);
29.            break;
30.        default:
31.            break;
32.    }
33.    glutPostRedisplay(); // 重绘
34. }
```

菜单功能

程序使用 `glutCreateMenu` 绑定一个菜单映射函数，传入的函数接受一个 `int` 类型，作为选择分支的判断，示例代码复用了 `userEventAction` 中的键盘映射规则。

使用 `glutAddMenuEntry` 添加菜单项，第一个参数传入菜单界面显示的名称，第二个参数储存了该菜单项被选中后，调用绑定的 `selectMain` 菜单函数并将其传入。这里创建了一个 `menuEntryStruct` 数组结构，简化菜单项的添加操作。

最后使用 `glutAttachMenu` 把创建后的菜单绑定到鼠标右键。

添加菜单功能的代码可以放在 `init` 函数中，在程序初始化时调用。

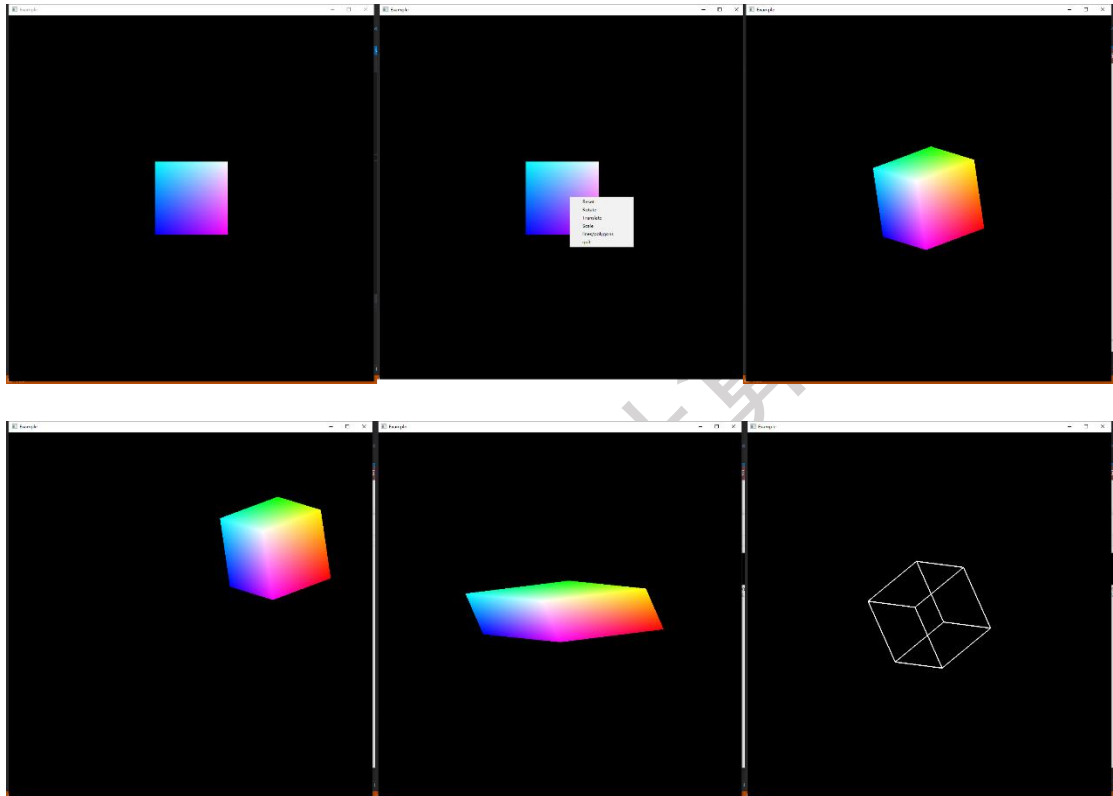
```
1.  struct menuEntryStruct {
2.      const char *label;
3.      char key;
4.  };
5.
6.  static menuEntryStruct mainMenu[] = {
7.      "Reset",          '0',
8.      "Rotate",         'r',
9.      "Translate",      't',
10.     "Scale",           's',
11.     "lines/polygons",  '1',
12.     "quit",            27, //ESC 键 (ASCII: 27)
13.  };
14.  int mainMenuEntries = sizeof(mainMenu) / sizeof(menuEntryStruct);
15.
16.  void selectMain(int choice)    // 序号->key 映射
17.  {
18.      userEventAction(mainMenu[choice].key);    // 调用通用动作解释函数
19.  }
20.
21.  void init(void)
22.  {
23.      glLineWidth(3.0);
24.      glEnable(GL_DEPTH_TEST); /* Enable hidden--surface--removal */
25.
26.      // 绑定菜单
27.      glutCreateMenu(selectMain); // 使用 selectMain 作为菜单调用函数
28.      for (int i = 0; i < mainMenuEntries; i++) {
29.          glutAddMenuEntry(mainMenu[i].label, i);
30.      }
```

```
31.
```

```
32.     glutAttachMenu(GLUT_RIGHT_BUTTON);           // 菜单绑定右键
```

```
33. }
```

实验结果参考



实验七 图形观察变换

实验目的

熟悉 OpenGL 观察变换函数, 使用 OpenGL 观察变换函数生成 3D 物体的不同观察图像。

实验内容

基本视图变换

参考教材 8.4 节和 10.10 节, 学习使用 OpenGL 观察变换函数。

观察变换使用 `gluLookAt` 模拟观察摄像机, 通过修改全局变量参数, 来达到变换视角的目的, 参数说明详见 10.10.1 节。使用 `glFrustum` 创建了透视投影的裁剪视椎体¹⁰。

```
1.  GLfloat xview = 0.0, yview = 0.0, zview = 0.0; // Viewing-
coordinate origin.
2.  GLfloat xref = 0.0, yref = 0.0, zref = -1.0; // Look-at point.
3.  GLfloat Vx = 0.0, Vy = 1.0, Vz = 0.0; // View-up vector.
4.  GLfloat viewxform_x=0.0, viewxform_y=0.0, viewxform_z=-5.0;
5.  void display(void)
6.  {
7.      glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
8.
9.      glMatrixMode(GL_PROJECTION);
10.     glLoadIdentity();
11.     glFrustum(-2.5, 2.5, -2.5, 2.5, mynear, myfar);
12.
13.     glMatrixMode(GL_MODELVIEW);
14.     glLoadIdentity();
15.     gluLookAt(xview, yview, zview, xref, yref, zref, Vx, Vy, Vz);
16.
17.     glTranslatef(viewxform_x, viewxform_y, viewxform_z);
18. }
```

¹⁰ 设置裁剪视椎体的部分应该放在 `reshape` 函数中, 根据窗口变化响应视椎体修改。此处放在 `display` 函数中, 便于理解视椎体和视线向量的相关性。

```
19.     // ...
20. }
```

为了便于观察，设置着色模式为 GL_FLAT，在正方体下方放置一个平台，该平台由 glutSolidCube 函数构建。

```
1.  void colorcube(void)
2.  {
3.      glShadeModel(GL_FLAT);
4.      // ...
5.  }
6.
7.  void display(void)
8.  {
9.      // ...
10.
11.     // 底部平面参考
12.     glPushMatrix();
13.     glTranslatef(0.0, -1.0, 0.0);
14.     glScalef(5.0, 0.1, 5.0);
15.     glColor3f(0.5, 0.5, 0.5);
16.     glutSolidCube(1.0);
17.     glPopMatrix();
18.
19.     glFlush();
20.     glutSwapBuffers();
21. }
```

在 userEventAction 函数中，添加下列事件响应：

- 数字键 2，前视图
- 数字键 4，左视图
- 数字键 6，右视图
- 数字键 8，后视图
- 数字键 5，顶视图

每种事件响应修改 gluLookAt 函数的调用参数，并在下一帧时切换视角。

```
1.  void userEventAction(int key) {
2.      switch (key) {
3.          case '2':     // 正视图
```

```
4.         xview = 0.0; yview = 0.0; zview = 0.0;
5.         xref = 0.0; yref = 0.0; zref = -5.0;
6.         Vx = 0.0; Vy = 1.0; Vz = 0.0;
7.         break;
8.     case '4':         // 左视图
9.         xview = -5.0; yview = 0.0; zview = -5.0;
10.        xref = 0.0; yref = 0.0; zref = -5.0;
11.        Vx = 0.0; Vy = 1.0; Vz = 0.0;
12.        break;
13.    case '6':         // 右视图
14.        xview = 5.0; yview = 0.0; zview = -5.0;
15.        xref = 0.0; yref = 0.0; zref = -5.0;
16.        Vx = 0.0; Vy = 1.0; Vz = 0.0;
17.        break;
18.    case '8':         // 后视图
19.        xview = 0.0; yview = 0.0; zview = -10.0;
20.        xref = 0.0; yref = 0.0; zref = -5.0;
21.        Vx = 0.0; Vy = 1.0; Vz = 0.0;
22.        break;
23.    case '5':         // 顶视图
24.        xview = 0.0; yview = 5.0; zview = -5.0;
25.        xref = 0.0; yref = 0.0; zref = -5.0;
26.        Vx = 0.0; Vy = 0.0; Vz = -1.0;
27.        break;
28.    }
29.    glutPostRedisplay();    // 重绘
30. }
```

在菜单结构中同样添加这些项，使用右键菜单可以执行相同的操作。

至此，程序可以完成基本的观察变换功能。

摄像机漫游

为了实现 3D 摄像机自由漫游的功能，我们只需要动态地更改 `gluLookAt` 函数的设定参数。本实验使用 WASD 控制摄像机前后左右偏移，使用 QE 控制摄像机升降，使用鼠标右键控制摄像机旋转。

添加 `rotateCamera` 布尔值，规定鼠标左键控制物体，`rotateCamera` 为 `false`，鼠标右键旋转摄像机，`rotateCamera` 为 `true`。

```
1.     bool rotateCamera = false;
```

```

2.
3.     void mouseButton(int button, int state, int x, int y)
4.     {
5.         switch (button) {
6.             case GLUT_LEFT_BUTTON:
7.                 rotateCamera = false;
8.                 break;
9.             case GLUT_RIGHT_BUTTON:
10.                rotateCamera = true;
11.                break;
12.            default:
13.                return;
14.        }
15.        switch (state) {
16.            case GLUT_DOWN:
17.                startMotion(x, y);
18.                break;
19.            case GLUT_UP:
20.                stopMotion(x, y);
21.                break;
22.        }
23.    }

```

把绑定到菜单的按键改为鼠标中键。

```

1.     glutAttachMenu(GLUT_MIDDLE_BUTTON);           // 菜单绑定

```

在 mouseMotion 函数中，对 rotateCamera 为 true 的情况，旋转摄像机。

VRight 表示摄像机局部坐标系的向右向量，其值由视线向量（xref - xview, yref - yview, zref - zview）和向上向量（Vx, Vy, Vz）的向量积求出。

局部变量 thetaX 表示俯仰角，thetaY 表示偏航角。

局部变量 px, py, pz 储存视线向量。

通过 quatRotate 四元数旋转，计算得出视线向量终点（因为视线向量起点位于参考点中心）经过绕向右向量旋转 thetaX，绕世界向上向量旋转 thetaY 后的最终点坐标。因为只是旋转，所以参考点（xview, yview, zview）不需要更改，只更新视点坐标（xref, yref, zref）。

最后对摄像机向上向量重复同样的计算，更新（Vx, Vy, Vz）。而摄像机向右向量可以在每次需要使用时重新计算。

```

1.  #define M_PI      3.14159265358979323846  // pi
2.
3.  GLfloat VRightx = (yref - yview) * Vz - (zref - zview) * Vy;
4.  GLfloat VRighty = (zref - zview) * Vx - (xref - xview) * Vz;
5.  GLfloat VRightz = (xref - xview) * Vy - (yref - yview) * Vx;
6.
7.  void mouseMotion(int x, int y)
8.  {
9.      float dx = x - lastX;
10.     float dy = y - lastY;
11.     if (!dx && !dy) return;
12.
13.     if (rotateCamera) {
14.         // 以参考点为中心，绕局部 X 轴旋转俯仰角，绕平行世界 Y 轴旋转环
视：
15.         // 左乘局部向右向量(cross(v,vup))旋转，左乘世界 Y 轴向量(0,1,0)
旋转
16.         VRightx = (yref - yview) * Vz - (zref - zview) * Vy;
17.         VRighty = (zref - zview) * Vx - (xref - xview) * Vz;
18.         VRightz = (xref - xview) * Vy - (yref - yview) * Vx;
19.         // 四元数旋转，旋转角由 dx 和 dy 确定
20.         GLfloat thetaX = -dy * (M_PI / winHeight);
21.         GLfloat thetaY = -dx * (M_PI / winWidth);
22.         GLfloat px = xref - xview;
23.         GLfloat py = yref - yview;
24.         GLfloat pz = zref - zview;
25.         quatRotate(thetaX, VRightx, VRighty, VRightz, px, py, pz);
26.         quatRotate(thetaY, 0.0, 1.0, 0.0, px, py, pz);
27.
28.         xref = xview + px;
29.         yref = yview + py;
30.         zref = zview + pz;
31.
32.         // 计算 vup 轴
33.         quatRotate(thetaX, VRightx, VRighty, VRightz, Vx, Vy, Vz);
34.         quatRotate(thetaY, 0.0, 1.0, 0.0, Vx, Vy, Vz);
35.     }
36.     else {
37.         if (transModeSelected == Translate) {
38.             axis[0] = dx * (10.0f/winWidth);
39.             axis[1] = -dy * (10.0f/winHeight);

```

```

40.         axis[2] = 0;
41.     }
42.     else if (transModeSelected == Rotate) {
43.         angle = 3.0;
44.         axis[0] = dy * (360.0f/winHeight);
45.         axis[1] = dx * (360.0f/winWidth);
46.         axis[2] = 0;
47.     }
48.     else if (transModeSelected == Scale) {
49.         axis[0] = dx * (4.0f/winWidth);
50.         axis[1] = -dy * (4.0f/winHeight);
51.         axis[2] = 0;    // 在调用时需要 +1
52.     }
53. }
54.
55. lastX = x;
56. lastY = y;
57. glutPostRedisplay();
58. }

```

四元数旋转函数 `quatRotate` 如下图所示。相关推导见教材 9.2.3 节。点坐标 (px, py, pz) 三个参数使用引用传递，并在函数内部创建临时副本，根据旧坐标计算得出旋转后的新坐标，并直接对引用赋值。

```

1.  void quatRotate(
2.      GLfloat theta, GLfloat ux, GLfloat uy, GLfloat uz,
3.      GLfloat& px, GLfloat& py, GLfloat& pz
4.  )
5.  {
6.      GLfloat inpx = px;
7.      GLfloat inpy = py;
8.      GLfloat inpz = pz;
9.      px =
10.         ( ux * ux * (1 - cos(theta)) + cos(theta) ) * inpx +
11.         ( ux * uy * (1 - cos(theta)) - uz * sin(theta) ) * inpy +
12.         ( ux * uz * (1 - cos(theta)) + uy * sin(theta) ) * inpz ;
13.      py =
14.         ( uy * ux * (1 - cos(theta)) + uz * sin(theta) ) * inpx +
15.         ( uy * uy * (1 - cos(theta)) + cos(theta) ) * inpy +
16.         ( uy * uz * (1 - cos(theta)) - ux * sin(theta) ) * inpz ;
17.      pz =
18.         ( uz * ux * (1 - cos(theta)) - uy * sin(theta) ) * inpx +

```

```

19.         ( uz * uy * (1 - cos(theta)) + ux * sin(theta) ) * inpy +
20.         ( uz * uz * (1 - cos(theta)) + cos(theta) ) * inpz ;
21.     }

```

在 `userEventAction` 函数中添加 WASD 和 QE 按键的事件映射，注意不要和之前的缩放操作的按键重复，可以把缩放按键改为 Y。按键区分大小写，示例代码均使用小写。

所有按键操作都会更新参考点坐标 (`xview, yview, zview`) 和视点坐标 (`xref, yref, zref`)。向前和向后沿摄像机视线向量 (`xref - xview, yref - yview, zref - zview`) 增加或减少平移量；向左和向右沿摄像机向右向量 (`VRightx, VRighty, VRightz`)，该向量需要在使用时更新值；摄像机升降沿世界向上向量 (`0, 1, 0`)。

```

1.  void userEventAction(int key) {
2.      switch (key) {
3.          case 'w':
4.              xview += 0.1 * (xref - xview);
5.              xref += 0.1 * (xref - xview);
6.              yview += 0.1 * (yref - yview);
7.              yref += 0.1 * (yref - yview);
8.              zview += 0.1 * (zref - zview);
9.              zref += 0.1 * (zref - zview);
10.         break;
11.         case 'a':
12.             VRightx = (yref - yview) * Vz - (zref - zview) * Vy;
13.             VRighty = (zref - zview) * Vx - (xref - xview) * Vz;
14.             VRightz = (xref - xview) * Vy - (yref - yview) * Vx;
15.             xview -= 0.1 * VRightx;
16.             xref -= 0.1 * VRightx;
17.             yview -= 0.1 * VRighty;
18.             yref -= 0.1 * VRighty;
19.             zview -= 0.1 * VRightz;
20.             zref -= 0.1 * VRightz;
21.         break;
22.         case 's':
23.             xview -= 0.1 * (xref - xview);
24.             xref -= 0.1 * (xref - xview);
25.             yview -= 0.1 * (yref - yview);
26.             yref -= 0.1 * (yref - yview);
27.             zview -= 0.1 * (zref - zview);
28.             zref -= 0.1 * (zref - zview);
29.         break;

```

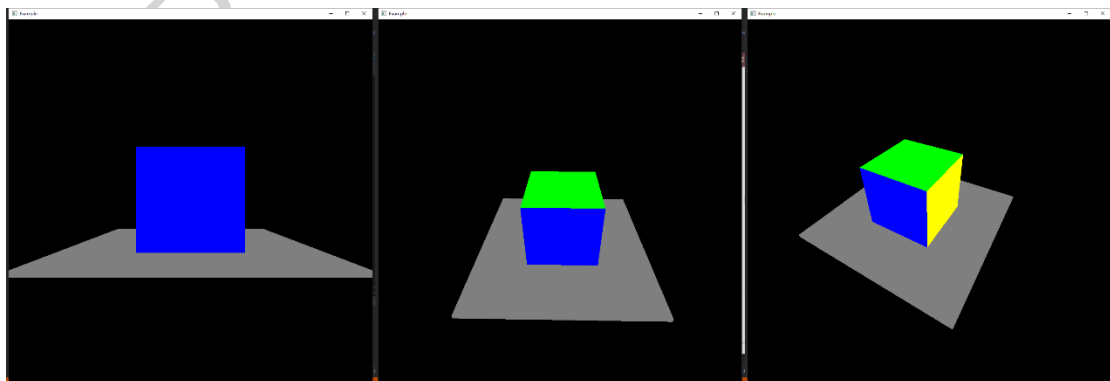
```

30.     case 'd':
31.         VRightx = (yref - yview) * Vz - (zref - zview) * Vy;
32.         VRighty = (zref - zview) * Vx - (xref - xview) * Vz;
33.         VRightz = (xref - xview) * Vy - (yref - yview) * Vx;
34.         xview += 0.1 * VRightx;
35.         xref += 0.1 * VRightx;
36.         yview += 0.1 * VRighty;
37.         yref += 0.1 * VRighty;
38.         zview += 0.1 * VRightz;
39.         zref += 0.1 * VRightz;
40.         break;
41.     case 'q':
42.         yview -= 0.1 * 1.0;
43.         yref -= 0.1 * 1.0;
44.         break;
45.     case 'e':
46.         yview += 0.1 * 1.0;
47.         yref += 0.1 * 1.0;
48.         break;
49.     }
50.     glutPostRedisplay();    // 重绘
51. }

```

其实，图形几何建模、几何变换、用户交互、观察变换，即前面几个实验内容可以合并为一个大实验，但出于篇幅代码长度的考虑，最终决定将实验步骤分解。读者可以将这些实验拼接，然后实现更多高阶功能，比如渲染器。

实验结果参考



实验八 贝塞尔曲线

实验目的

巩固样条曲线-贝塞尔曲线构造原理，使用 OpenGL 函数实现贝塞尔曲线的绘制。

实验内容

参考教材 14.8 节，可以绘制一个简易的 Bezier 曲线，但控制点被硬编码，无法动态修改曲线形状。

完整实现一个支持动态修改的 Bezier 曲线程序，需要

1. 动态创建控制点（对象资源管理）
2. 选取和修改控制点（OpenGL 拾取）

该实验不要求读者严格按照实验指导书来编写，可以只实现教材中的简易版本；如果读者熟悉 C++ 特性，可参考附录二，是一个还不错的 C++ 编程练习机会。

GLObject.h 头文件

参考教材 14.8 节和 14.9 节，根据公式推导使用 OpenGL 画点函数绘制贝塞尔曲线。

创建 GLObject.h 头文件，在其中定义 GLObject 相关结构。该文件中的结构说明如下：

GLState	用于判断当前操作状态
GLColor	存储颜色值的结构体
GLTransform	存储变换值的结构体
GLWorld	自定义世界结构体，表示程序运行的世界环境，包含 GLPoint 指针数组，管理所有绘制点的选取及移动
GLPoint	存储用户选取的单个点的相关信息
GLBezierCtrPts	单个该结构体保存一组连续绘制的 GLPoint 指针数组，在同一个程序中可能存在多个该结构体

此外，宏定义程序中的部分硬编码数值。

```

1.  #define MAX_OBJECTS 10000
2.  #define DEFAULT_POINT_SIZE 10
3.  #define DEFAULT_LINE_WIDTH 5
4.  #define DEFAULT_BEZIER_CONTROL_POINTS_NUM 4
5.  #define MAX_BEZIER_CONTROL_POINTS_NUM 13
6.  #define DEFAULT_PICK_BUFFER_SIZE 32
7.  #define DEFAULT_PICK_WINDOW_SIZE 10
8.  #define MAX_BEZIER_LINE_NUM 5
9.  #define PI 3.14159265358979323846 // pi
10. #include<math.h>
11. #include<GL/glut.h>
12. enum GLState {
13.     MoveObject,
14.     MakePoint
15. };
16.
17.
18. struct GLColor
19. {
20.     GLfloat r = 0, g = 0, b = 0;
21. };
22.
23. struct GLTransform
24. {
25.     // 世界坐标, 轴旋转, 轴缩放
26.     GLdouble x = 0, y = 0, z = 0;
27.     GLdouble rx = 0, ry = 0, rz = 0;
28.     GLdouble sx = 1, sy = 1, sz = 1;
29. };

```

GLPoint 的定义如下表格所示，成员变量和成员函数解释如下：

id	GLPoint 对象唯一标识符
color	GLColor 结构颜色值
transform	GLTransform 结构变换值
movable	对象是否可以移动
visible	对象是否可见
parentWorld	所在世界对象指针
Draw	绘制函数
SetPosition	设置位置值的封装函数

```

1.  struct GLPoint {
2.      GLint id = 0;
3.      Color color ;
4.      GLTransform transform;
5.      bool movable = false;
6.      bool visible = true;
7.
8.      GLWorld* parentWorld;
9.      void SetPosition(GLdouble x, GLdouble y, GLdouble z) {
10.          transform.x = x;
11.          transform.y = y;
12.          transform.z = z;
13.      }
14.      void Draw(GLenum RenderMode = GL_RENDER) {
15.          // 需要在绘制前调用 SetPostion, 否则绘制在原点
16.          if (!visible) return;
17.          if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
18.          glColor3f(1, 0, 0);
19.          glPointSize(DEFAULT_POINT_SIZE);
20.          glBegin(GL_POINTS);
21.          glVertex3d(transform.x, transform.y, transform.z);
22.          glEnd();
23.
24.      }
25.  }

```

GLWorld 类的定义如下表格所示，成员变量和成员函数解释如下：

objects	GLPoint*数组，储存世界对象下所有 GLPoint 对象
count	用作 objects 数组的下标索引和对象计数
CreatPt	创建一个 GLPoint 对象，添加到 objects 数组，返回该类型指针
DeleteObject	根据 id 删除一个对象（需要自己实现）
Empty	清空所有对象（需要自己实现）
DrawObjects	循环调用所有对象的 Draw 方法，缺省渲染方式为 GL_RENDER，可选值还有 GL_SELECT
pickedObjectId	当前选取的对象 ID
PickObject	根据鼠标输入选取对象，获得对象 ID 储存到 pickedObjectId 变量

```

1.  struct GLWorld
2.  {
3.      GLPoint* objects[MAX_OBJECTS];
4.      GLint count = 0;
5.
6.      GLPoint* CreatePt(GLdouble x, GLdouble y, GLdouble z) {
7.          if (count < MAX_OBJECTS)
8.          {
9.              GLPoint* ret = (GLPoint*)malloc(sizeof(GLPoint));
10.             objects[count] = (GLPoint*)ret;
11.             objects[count]->parentWorld = this;
12.             objects[count]->id = count;
13.             objects[count]->SetPosition(x, y, z);
14.             ++count;
15.             return ret;
16.         }
17.         else return nullptr;
18.     }
19.
20.     //void DeleteObject(GLint id);
21.     //void Empty();
22.
23.     void DrawObjects(GLenum RenderMode = GL_RENDER) {
24.         for (int i = 0; i < count; ++i)
25.         {
26.             if (!objects[i]) return;
27.             objects[i]->Draw(RenderMode);
28.         }
29.
30.     }
31.
32.     GLint pickedObjectId = -1;
33.     void PickObject(GLint xMouse, GLint yMouse) {
34.         pickRects(GLUT_LEFT_BUTTON, GLUT_DOWN, xMouse, yMouse);
35.
36.     }
37.
38.     //void pickRects(GLint button, GLint action, GLint xMouse, GLint
yMouse);
39.     //void processPicks(GLint nPicks, GLuint pickBuffer[]);
40.     GLuint pickBuffer[DEFAULT_PICK_BUFFER_SIZE];
41.     GLint nPicks, vpArray[4];
42.     void processPicks(GLint nPicks, GLuint pickBuffer[])

```

```

43.     {
44.         // 储存获取到的最后一个 ID，即认为后绘制的图像在先绘制的图形之上
45.         if (nPicks == 0) pickedObjectId = -1;
46.         else
47.         {
48.             // 因为只使用一个栈顶元素，所以可以由公式计算得出最后一个图
形 ID
49.             int chunkId = 4 * (nPicks - 1);
50.             pickedObjectId = pickBuffer[chunkId + 3];
51.         }
52.         return;
53.     }
54. void pickRects(GLint button, GLint action, GLint xMouse, GLint
yMouse)
55.     {
56.         GLuint pickBuffer[DEFAULT_PICK_BUFFER_SIZE];
57.         GLint nPicks, vpArray[4];
58.
59.         if (button != GLUT_LEFT_BUTTON || action != GLUT_DOWN)
60.             return;
61.
62.         glSelectBuffer(DEFAULT_PICK_BUFFER_SIZE, pickBuffer); //
Designate pick buffer.
63.         glRenderMode(GL_SELECT); // Activate picking
operations.
64.         glInitNames(); // Initialize the object-
ID stack.
65.
66.         // 只使用一个栈顶元素，后续为图形命名使用 glLoadName(id) 替换
栈顶
67.         glPushName(MAX_OBJECTS);
68.
69.         /* Save current viewing matrix. */
70.         glMatrixMode(GL_PROJECTION);
71.         glPushMatrix();
72.         glLoadIdentity();
73.
74.         /* Obtain the parameters for the current viewport. Set up
75.         * a 5 x 5 pick window, and invert the input yMouse value
76.         * using the height of the viewport, which is the fourth
77.         * element of vpArray.
78.         */
79.         glGetIntegerv(GL_VIEWPORT, vpArray);
80.         gluPickMatrix(GLdouble(xMouse), GLdouble(yMouse),

```

```

81.          DEFAULT_PICK_WINDOW_SIZE, DEFAULT_PICK_WINDOW_SIZE,
vpArray);
82.
83.          gluOrtho2D(xwcMin, xwcMax, ywcMin, ywcMax);
84.          //rects (GL_SELECT);      // Process the rectangles in
selection mode.
85.          this->DrawObjects(GL_SELECT);
86.
87.          /* Restore original viewing matrix. */
88.          glMatrixMode(GL_PROJECTION);
89.          glPopMatrix();
90.
91.          glFlush();
92.
93.          /* Determine the number of picked objects and return to
the
94.          * normal rendering mode.
95.          */
96.          nPicks = glRenderMode(GL_RENDER);
97.          processPicks(nPicks, pickBuffer); // Process picked
objects.
98.
99.          glutPostRedisplay();
100.     }
101.
102. };
103. extern GLfloat xwcMin, xwcMax, ywcMin, ywcMax;

```

结构中两个函数用于执行 OpenGL 拾取操作，可参考教材 20.6.7 节实现。不同的地方在于：

- 这两个函数变为结构体成员函数
- pickRects 方法在使用 OpenGL 名称拾取栈时，只使用一个栈顶元素
- pickRects 方法需要指定拾取裁剪窗口，所以需要获得裁剪坐标
- pickRects 方法不绘制方块，而是调用 DrawObjects(GL_SELECT)
- processPicks 方法只需要为 pickedObjectId 赋值

GLBezierCtrlPts 结构体主要作用是对用户单次连续选取的贝塞尔曲线控制点进行管理以及根据对应的控制点绘制贝塞尔曲线。

贝塞尔曲线绘制参考教材 14.8.2 节实现，需注意在控制点数量较多时，系数在阶乘阶段可能会超出数据范围，导致曲线错误。

```

1.  struct GLBezierCtrlPts
2.  {
3.      GLWorld* parentWorld;
4.      GLPoint* ctrlPoints[MAX_BEZIER_CONTROL_POINTS_NUM];
5.      int count = 0;
6.      GLint nBezCurvePoints = 1000;
7.
8.      // Usage:
9.      // GLWorld::NewObject<GLBezier>()->MakePoint(x,y,z);
10.     /**/
11.     void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
12.         if (count >= MAX_BEZIER_CONTROL_POINTS_NUM) return;
13.         ctrlPoints[count] = parentWorld->CreatePt(x, y, z);
14.         //ctrlPoints[count]->SetPosition(x, y, z);           // 初始位置
15.         ctrlPoints[count]->movable = true;                  // 可以移动
16.         count++;
17.         //printf("mp \n");
18.     }
19.     void Draw(GLenum RenderMode = GL_RENDER) {
20.         // 需要在绘制前初始化控制点
21.         if (count == 0) return;
22.         for (int i = 0; i < count; ++i)
23.         {
24.             if (!ctrlPoints[i]) return;
25.         }
26.         //if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
27.         glColor3f(0.0, 0.0, 0.0);
28.         glPointSize(DEFAULT_LINE_WIDTH);
29.         bezier(ctrlPoints, nBezCurvePoints);
30.
31.     }
32.
33.     struct wcPt3D {
34.
35.         GLfloat x, y, z;
36.     };
37.     void plotPoint(wcPt3D bezCurvePt) {
38.         glBegin(GL_POINTS);
39.         glVertex2f(bezCurvePt.x, bezCurvePt.y);
40.         glEnd();
41.     }
42.     void binomialCoeffs(GLint n, GLint* C) {
43.         GLint k, j;
44.         for (k = 0; k <= n; k++)

```

```

45.         {
46.             C[k] = 1;
47.             for (j = n; j >= k + 1; j--)
48.                 C[k] *= j;
49.             for (j = n - k; j >= 2; j--)
50.                 C[k] /= j;
51.         }
52.     }
53.     void computeBezPt(GLfloat u, wcPt3D* bezPt, GLint nCtrlPts,
wcPt3D* ctrlPts, GLint* C) {
54.         GLint k, n = nCtrlPts - 1;
55.         GLfloat bezBlendFcn;
56.         bezPt->x = bezPt->y = 0.0;
57.         for (k = 0; k < nCtrlPts; k++) {
58.             bezBlendFcn = C[k] * pow(1 - u, n - k) * pow(u, k);
59.             bezPt->x += ctrlPts[k].x * bezBlendFcn;
60.             bezPt->y += ctrlPts[k].y * bezBlendFcn;
61.             //bezPt->z += ctrlPts[k].z * bezBlendFcn;
62.         }
63.     }
64.     void drawBezier(wcPt3D* ctrlPts, GLint nCtrlPts, GLint
nBezCurvePts) {
65.         wcPt3D bezCurvePt;
66.         GLfloat u;
67.         GLint* C, k;
68.         C = new GLint[nCtrlPts];
69.         binomialCoeffs(nCtrlPts - 1, C);
70.         for (k = 0; k <= nBezCurvePts; k++)
71.         {
72.             u = GLfloat(k) / GLfloat(nBezCurvePts);
73.             computeBezPt(u, &bezCurvePt, nCtrlPts, ctrlPts, C);
74.             plotPoint(bezCurvePt);
75.             //printf("%d/t", k);
76.         }
77.         delete[] C;
78.     }
79.     void bezier(GLPoint* ctrlPts[], GLint nBezCurvePts) {
80.
81.         wcPt3D innerCtrlPts[MAX_BEZIER_CONTROL_POINTS_NUM];
82.         for (int i = 0; i < count; ++i)
83.         {
84.             innerCtrlPts[i].x = (GLfloat)ctrlPts[i]->transform.x;
85.             innerCtrlPts[i].y = (GLfloat)ctrlPts[i]->transform.y;
86.             innerCtrlPts[i].z = (GLfloat)ctrlPts[i]->transform.z;

```



```

87.     }
88.     drawBezier(innerCtrlPts, count, nBezCurvePts);
89.
90.     // 绘制控制点连线
91.     if (ctrlPts[0] && ctrlPts[0]->visible == false) return;
92.     glColor3f(0.0, 1.0, 0.0);
93.     glLineWidth(2.0);
94.     glBegin(GL_LINE_STRIP);
95.     for (int i = 0; i < count; ++i)
96.     {
97.         glVertex3d(ctrlPts[i]->transform.x,
ctrlPts[i]->transform.y, ctrlPts[i]->transform.z);
98.     }
99.     glEnd();
100.    glColor3f(1.0, 0.0, 0.0);
101.
102.    for (int i = 0; i < count; ++i)
103.    {
104.        ctrlPoints[i]->Draw();
105.    }
106.
107. }
108. }
```

Bezier.cpp 源文件

在源文件顶部引用“GLObject.h”，并创建 MyWorld、MyBezier 全局变量，设置程序窗口长宽和裁剪窗口坐标。

```

1.  #include "GLObject.h"
2.
3.  GLWorld MyWorld;
4.  GLBezierCtrlPts MyBezier[MAX_BEZIER_LINE_NUM];
5.  GLState MyState = MoveObject;
6.  GLint LineNum = 0;
7.  /* Set initial size of the display window. */
8.  GLsizei winWidth = 900, winHeight = 600;
9.
10. /* Set size of world-coordinate clipping window. */
11. GLfloat xwcMin = -450.0, xwcMax = 450.0;
```

```
12. GLfloat ywcMin = -300.0, ywcMax = 300.0;
```

设置菜单绑定，支持右键新建贝塞尔曲线。即在右键菜单中选取 Bezier 曲线后，对全局世界对象调用模板函数 `MyWorld.NewObject<GLBezier>()` 构建一个 GLBezier 对象，之后进入 `GLState::MakePoint` 状态，为曲线添加控制点。

在贝塞尔曲线完成后，可以再次点击菜单项结束绘制。之后可以通过鼠标点选控制点进行变换。

```
1. struct menuEntryStruct {
2.     const char *label;
3.     char key;
4. };
5.
6. static menuEntryStruct mainMenu[] = {
7.     "Reset",          '0',
8.     "Create Bezier/Finish Bezier", '1',
9.     "Quit",           27, //ESC 键 (ASCII: 27)
10. };
11. int mainMenuEntries = sizeof(mainMenu) / sizeof(menuEntryStruct);
12.
13.
14. bool globalControlPointsVisible = true;
15.
16. void userEventAction(int key) {
17.     switch (key) {
18.     case '0':
19.         MyWorld.Empty();
20.         MyWorld.pickedObjectId = -1;
21.         MyState = MoveObject;
22.         break;
23.     case '1': // bezier
24.         if (MyState == MoveObject)
25.         {
26.             MyState = MakePoint;
27.             ++lineNum;
28.         }
29.         else if (MyState == MakePoint)
30.         {
31.             MyState = MoveObject;
32.         }
33.     }
```

```

34.         break;
35.     case 27:        // ESC 键 (ASCII: 27) 退出
36.         MyWorld.Empty();
37.         exit(0);
38.         break;
39.     default:
40.         break;
41. }
42. glutPostRedisplay(); // 重绘
43. }

```

显示函数中调用 `MyWorld.DrawObjects(GL_RENDER)` 将 `objects` 数组中动态生成的对象全部绘制。

```

1. void displayFcn (void)
2. {
3.     glClear (GL_COLOR_BUFFER_BIT); // Clear display window.
4.     for (int i = 1; i <= lineNum;i++)MyBezier[i].Draw();
5.     glFlush ( );
6. }

```

鼠标按键绑定函数中，支持左键按下拾取操作，并且在左键弹起时重置 `pickedObjectId` 变量值，如果当前状态为绘制控制点状态，在鼠标按下的位置调用 `MakePoint` 方法，弹起时刷新窗口。

```

1. void unProject(GLdouble winx, GLdouble winy, GLdouble winz, GLdouble
   *objx, GLdouble *objy, GLdouble *objz)
2. {
3.     // 从窗口到世界坐标系
4.     GLint viewport[4];
5.     GLdouble modelview[16];
6.     GLdouble projection[16];
7.     glGetDoublev(GL_MODELVIEW_MATRIX, modelview);
8.     glGetDoublev(GL_PROJECTION_MATRIX, projection);
9.     glGetIntegerv(GL_VIEWPORT, viewport);
10.    gluUnProject(winx, winy, winz, modelview, projection, viewport
, objx, objy, objz);
11. }
12.
13. void mouseButton(int button, int state, int x, int y)

```

```

14.  {
15.      switch (button) {
16.          case GLUT_LEFT_BUTTON:
17.              break;
18.          default:
19.              return;
20.      }
21.      switch (state) {
22.          case GLUT_DOWN:
23.              if (MyState == MoveObject)
24.              {
25.                  MyWorld.PickObject(x, y);
26.                  printf("MyWorld.pickedObjectId: %d \n", MyWorld.picked
27.                  ObjectId);
28.              }
29.              else if (MyState == MakePoint)
30.              {
31.                  if (GLObject* ptr = MyWorld.objects[MyWorld.pickedObje
32.                  ctId])
33.                  {
34.                      // 从窗口到世界坐标系
35.                      GLdouble winx = (GLdouble) x;
36.                      GLdouble winy = (GLdouble) winHeight - (GLdouble)
37.                      y;
38.                      GLdouble winz = 0;
39.                      GLdouble objx = 0;
40.                      GLdouble objy = 0;
41.                      GLdouble objz = 0;
42.                      unProject(winx, winy, winz, &objx, &objy, &objz);
43.                      MyBezier[lineNum].parentWorld = &MyWorld;
44.                      MyBezier[lineNum].MakePoint(objx, objy, objz);
45.                  }
46.              }
47.              break;
48.          case GLUT_UP:
49.              glutPostRedisplay();
50.              break;
51.      }
52.  }

```

鼠标滑动绑定函数中，对当前选取的对象 ID，检查是否可以移动，然后通

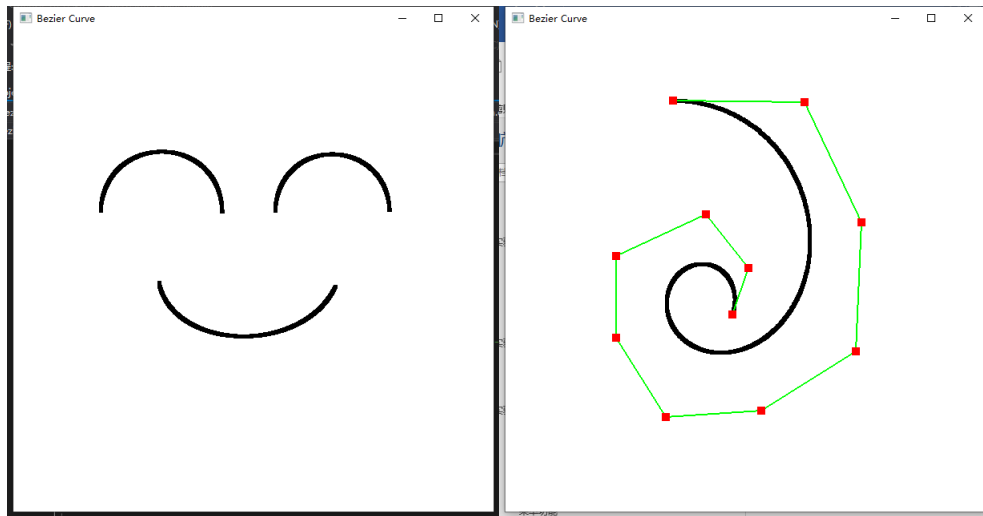
过 `objects` 数组获取对象指针。

使用 `gluUnProject` 将鼠标指针的屏幕坐标反投影回世界坐标系，将得到的世界坐标系值储存到临时变量，并对当前选取对象重新设置坐标位置。

最后使用 `glutPostRedisplay` 通知窗口重新绘制。

```
1.  void mouseMotion(int x, int y)
2.  {
3.      if ( MyWorld.pickedObjectId == -1 ) return;
4.      if ( ! MyWorld.objects[MyWorld.pickedObjectId]->movable ) return;
5.
6.      GLPoint* ptr = MyWorld.objects[MyWorld.pickedObjectId];
7.
8.      // 从窗口到世界坐标系
9.      GLdouble winx = (GLdouble) x;
10.     GLdouble winy = (GLdouble) winHeight - (GLdouble) y;
11.     GLdouble winz = 0;
12.     GLdouble objx = 0;
13.     GLdouble objy = 0;
14.     GLdouble objz = 0;
15.     unProject(winx, winy, winz, &objx, &objy, &objz);
16.
17.     ptr->SetPosition(objx, objy, objz);
18.
19.     glutPostRedisplay();
20. }
```

实验结果参考



实验九 分形图形

实验目的

学习各种分形图形构造方法，使用 OpenGL 函数实现分形图形的绘制。

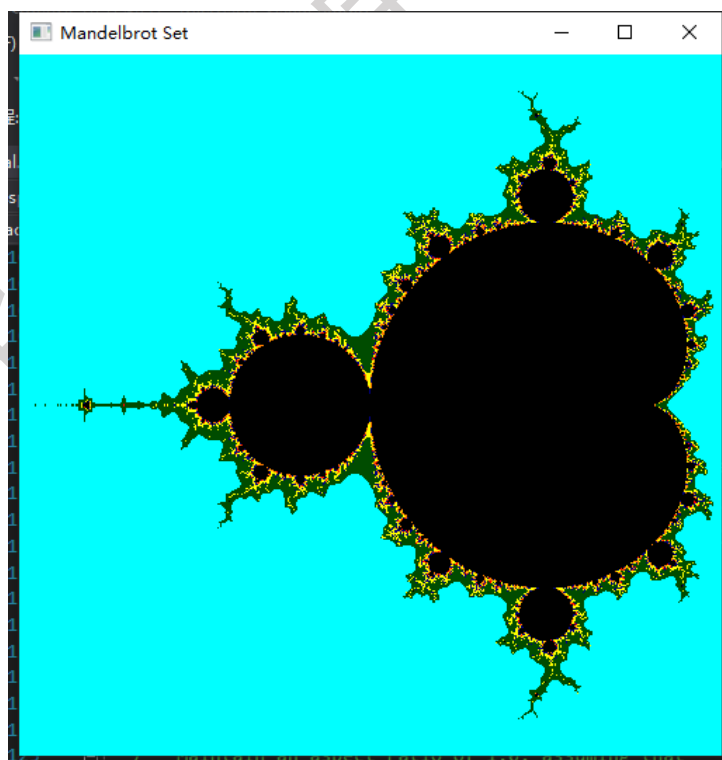
实验内容

Mandelbrot Set

参考教材 23.1 节，学习使用 OpenGL 函数绘制分形图形。

实验程序分形图形选择了自平方分形中的 Mandelbrot 集。

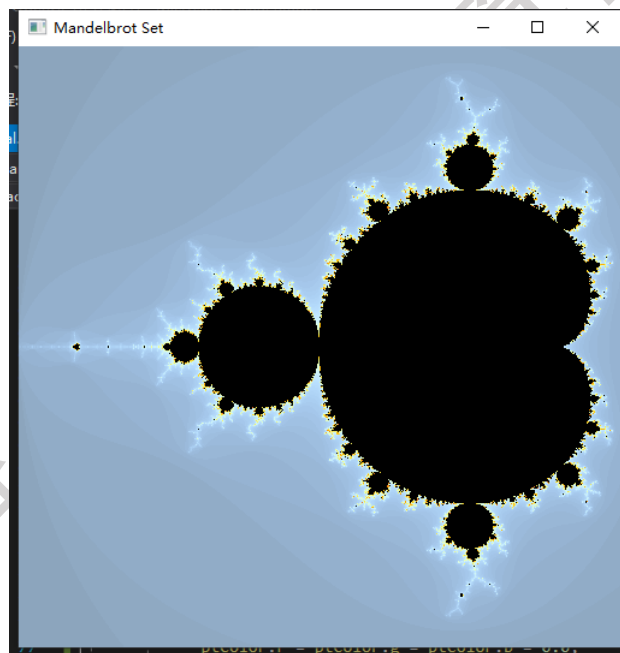
根据书中代码，我们可以很轻易的绘制出 Mandelbrot 集图形。使窗口聚焦复数平面上的收敛点集（图形中黑色区域），该集合内的复数点经过公式迭代到最大迭代次数后仍未发散（若模长大于 2 则判断为发散点）。对周围的发散点按照逃逸时间（即模长超过 2 所需迭代次数）映射为颜色值，对发散点上色，形成分形图形。为了便于实验，请将代码中的迭代次数设置为 100。



自由缩放

为了使图形颜色更舒适，可以自行设计颜色映射主题，如下图所示。

```
1.  if (iterCount >= maxIter)
2.      /* Set point color to black. */
3.      ptColor.r = ptColor.g = ptColor.b = 0.0;
4.  else
5.  {
6.      GLfloat value = (GLfloat) iterCount / maxIter;
7.
8.      ptColor.r = (GLfloat) ( sin( value * 2 + 0.1 ) / 2 + 0.5 );
9.      ptColor.g = (GLfloat) ( sin( value * 3 + 0.3 ) / 2 + 0.5 );
10.     ptColor.b = (GLfloat) ( sin( value * 5 + 0.5 ) / 2 + 0.5 );
11. }
```



实现自由缩放显示分形细节，只需要重新设置复数平面空间，并映射到裁剪窗口。

鼠标左键按下执行向目标点缩放事件，在按下之后绑定 `glutIdleFunc` 执行 `zoom` 函数，在鼠标左键弹起后取消执行该事件。在鼠标指针改变位置时，更新目标点。

```
1.  GLint newX = 0, newY = 0;
2.
3.  void mouseButton(int button, int state, int x, int y)
```



```
4.  {
5.      switch (button) {
6.          case GLUT_LEFT_BUTTON:
7.              break;
8.          case GLUT_RIGHT_BUTTON:
9.              // 重置
10.             xComplexMin = -2.00; xComplexMax = 0.50;
11.             yComplexMin = -1.25; yComplexMax = 1.25;
12.             complexWidth = xComplexMax - xComplexMin;
13.             complexHeight = yComplexMax - yComplexMin;
14.             glutPostRedisplay();
15.             return;
16.         default:
17.             return;
18.     }
19.     switch (state) {
20.         case GLUT_DOWN:
21.             newX = x;
22.             newY = y;
23.             glutIdleFunc(zoom);
24.             break;
25.         case GLUT_UP:
26.             newX = 0;
27.             newY = 0;
28.             glutIdleFunc(NULL);
29.             break;
30.     }
31. }
32.
33. void mouseMotion(int x, int y)
34. {
35.     newX = x;
36.     newY = y;
37. }
```

Zoom 函数修改复数空间坐标，向目标点靠拢，并通知窗口重新绘制。可以在这里设置每次缩放的幅度。该函数引用了自定义的 unProject 封装函数将鼠标坐标值映射到世界坐标系。

参数 winx, winy, winz 是当前鼠标点的窗口坐标，winz 始终为 0。

参数 objx, objy, objz 是将窗口坐标点反向投影回 OpenGL 世界坐标系得到的“对象”坐标值。

```

1.  void unProject(GLdouble winx, GLdouble winy, GLdouble winz, GLdouble
le *objx, GLdouble *objy, GLdouble *objz)
2.  {
3.      // 从窗口到世界坐标系
4.      GLint viewport[4];
5.      GLdouble modelview[16];
6.      GLdouble projection[16];
7.      glGetDoublev(GL_MODELVIEW_MATRIX, modelview);
8.      glGetDoublev(GL_PROJECTION_MATRIX, projection);
9.      glGetIntegerv(GL_VIEWPORT, viewport);
10.     gluUnProject(winx, winy, winz, modelview, projection, viewport
, objx, objy, objz);
11. }
12.
13. void zoom()
14. {
15.     // 向下缩放
16.     // 从窗口到世界坐标系
17.     GLdouble winx = (GLdouble) newX;
18.     GLdouble winy = (GLdouble) winHeight - (GLdouble) newY;
19.     GLdouble winz = 0;
20.     GLdouble objx = 0;
21.     GLdouble objy = 0;
22.     GLdouble objz = 0;
23.     unProject(winx, winy, winz, &objx, &objy, &objz);
24.     xComplexMin += (objx - xComplexMin) / 5;
25.     xComplexMax += (objx - xComplexMax) / 5;
26.     yComplexMin += (objy - yComplexMin) / 5;
27.     yComplexMax += (objy - yComplexMax) / 5;
28.     complexWidth = xComplexMax - xComplexMin;
29.     complexHeight = yComplexMax - yComplexMin;
30.     glutPostRedisplay();
31. }

```

在显示函数中，需要重新设置裁剪窗口。设置 `nx` 和 `ny` 改变需要计算的点的个数，设置 `maxIter` 更改最大迭代次数。

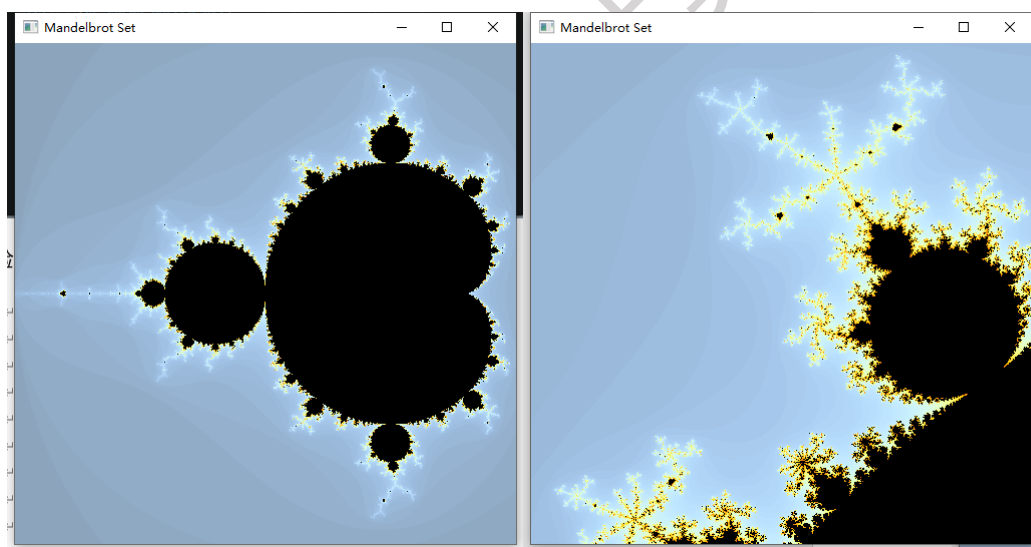
```

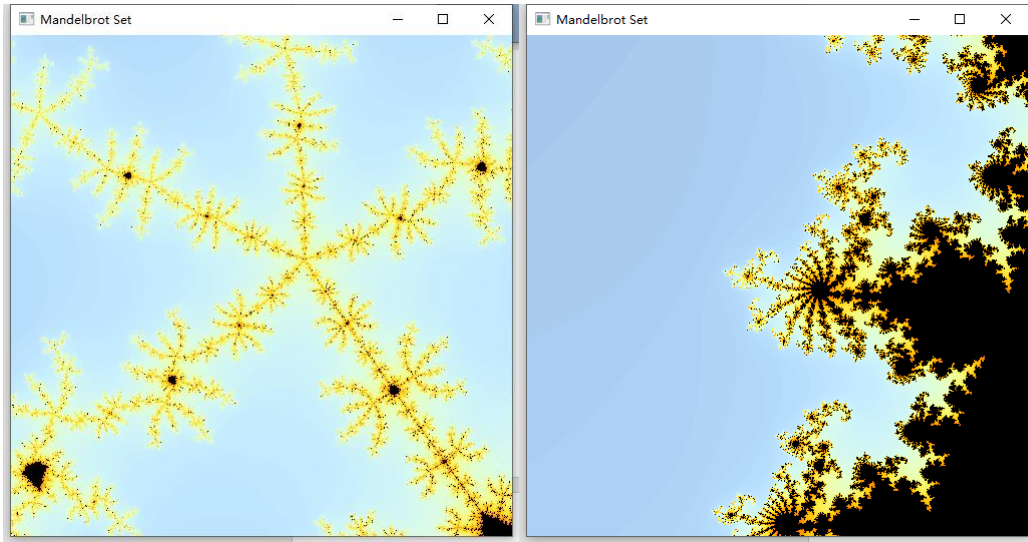
1.  void displayFcn (void)
2.  {
3.      /* Set number of x and y subdivisions and the max iterations.
*/

```

```
4.      GLint nx = winWidth, ny = winHeight, maxIter = 100;
5.
6.      glClear (GL_COLOR_BUFFER_BIT);    // Clear display window.
7.
8.      glMatrixMode (GL_PROJECTION);
9.      glLoadIdentity ( );
10.     gluOrtho2D (xComplexMin, xComplexMax, yComplexMin, yComplexMax
);
11.     glMatrixMode (GL_MODELVIEW);
12.
13.     mandelbrot (nx, ny, maxIter);
14.     glFlush ( );
15.
16.     printf("Zoom: %.2f \n", ( 2.5 / complexHeight ));
17. }
```

实验结果参考





实验十 导入模型文件

实验目的

了解常用 3D 模型文件格式，并支持程序导入和读取模型文件。

实验内容

该实验基于之前的实验一“立方体绘制”的整体代码框架，将立方体模型替换为其它外部模型。该实验采用 C++ 实现，附录 3 提供 C 语言实现供参考。

导入外部模型（OBJ）

OBJ 文件是一种常见的 3D 模型文件格式¹¹。本实验示例使用 Newell teaset¹² 的茶壶模型，读者可以替换其它模型。

OBJLoader 或者 OBJImporter 负责将 .obj 文件数据解析成我们需要使用的数据。在对象初始化时使用 `parseStream()` 函数逐行解析，将结果（顶点、法线、面、材质）保存到成员变量中。

除 `parseStream()` 外，`smoothNormals()` 函数用于生成平滑顶点法线，可参考这篇文章¹³。

```
1. #pragma once
2. #include <GL/glut.h>
3. #include <iostream>
4. #include <fstream>
5. #include <vector>
6. #include <string>
7. #include <sstream>
8. #include <unordered_map>
9. #include <algorithm>
10. using namespace std;
```

¹¹ OBJ: https://en.wikipedia.org/wiki/Wavefront_.obj_file

¹² Newell teaset: https://www.cs.utah.edu/~natevm/newell_teaset/newell_teaset.zip

¹³ Max Wagner, Generating Vertex Normals,
<https://web.archive.org/web/20130531101356/http://www.emeyex.com/site/tuts/VertexNormals.pdf>

```

11.
12. struct Vertex
13. {
14.     GLfloat x, y, z;
15. };
16.
17. struct Texture
18. {
19.     GLfloat x, y;
20. };
21.
22. struct Normal
23. {
24.     GLfloat x, y, z;
25. };
26.
27. struct Material
28. {
29.     GLfloat ambient[4] = {0.2, 0.2, 0.2, 1.0};
30.     GLfloat diffuse[4] = {0.8, 0.8, 0.8, 1.0};
31.     GLfloat specular[4] = {0.0, 0.0, 0.0, 1.0};
32.     GLfloat shininess = 65.0;
33. };
34.
35. struct Face
36. {
37.     GLint a, b, c;      // 顶点下标
38.     GLint ta, tb, tc;   // 顶点纹理下标
39.     GLint na, nb, nc;   // 顶点法线下标
40.     string mtl_name;    // 材质名
41. };
42.
43. class OBJImporter
44. {
45. public:
46.     OBJImporter(string path)
47.         : path(path)
48.     {
49.         ifstream ifs;
50.         ifs.open(path, ifstream::in);
51.         if (!ifs.is_open()) {
52.             cerr << "Can't open file.\n";
53.             return;
54.         }

```

```
55.     parseStream(ifs);
56.     ifs.close();
57.     smoothNormals();
58. }
59. ~OBJImporter() = default;
60.
61. string path;
62. vector<Vertex> vertexList;
63. vector<Texture> textureList;
64. vector<Normal> normalList;
65. vector<Face> faceList;
66. unordered_map<string, Material> materialTable;
67.
68. private:
69. void parseStream(std::ifstream& ifs) {
70.     string lineBuffer;
71.     string currentMaterialName;
72.     while (getline(ifs, lineBuffer).good())
73.     {
74.         if (lineBuffer.empty()) continue;
75.         switch (lineBuffer[0])
76.         {
77.             case '#': // comment
78.                 break;
79.             case 'm': // mtl:lib
80.                 readMaterial(lineBuffer);
81.                 break;
82.             case 'u': // usemtl
83.                 useMaterial(lineBuffer, currentMaterialName);
84.                 break;
85.             case 'v': // v, vt, vn
86.                 readVertex(lineBuffer);
87.                 break;
88.             case 'f': // face
89.                 readFace(lineBuffer, currentMaterialName);
90.                 break;
91.             default: // 未支持 group
92.                 break;
93.         }
94.     }
95. }
96. void readMaterial(string& lineBuffer);
97. void useMaterial(string& lineBuffer, string& currentMaterialName);
98. void readVertex(string& lineBuffer);
```

```

99.     void readFace(string& lineBuffer, string& currentMaterialName);
100.     void smoothNormals();
101. };

```

除 `parseStream()` 外，其他成员函数实现如下。可放在 `OBJImporter.cpp` 中。为了方便读取解析文件，使用了 `stringstream` 格式化读入。

```

1.  #include "OBJImporter.h"
2.
3.  void OBJImporter::readMaterial(string& lineBuffer)
4.  {
5.      stringstream lineStream(lineBuffer);
6.      string skip;
7.      string materialFilename;
8.      lineStream >> skip >> materialFilename;
9.      // 在 .obj 同路径下读取 .mtl 文件
10.     ifstream mtl;
11.     mtl.open(path.substr(0, path.rfind('/') + 1) + materialFilename, ifstream::in);
12.     if (!mtl.is_open()) {
13.         cerr << "Can't open file.\n";
14.         return;
15.     }
16.     // 解析 .mtl
17.     string lineBufferMtl;
18.     string currentMaterialName;
19.     while (getline(mtl, lineBufferMtl).good())
20.     {
21.         if (lineBuffer.empty()) continue;
22.         stringstream lineStreamMtl(lineBufferMtl);
23.         string skip;
24.         switch (lineBufferMtl[0])
25.         {
26.             case '#':
27.                 break;
28.             case 'n': // newmtl
29.                 lineStreamMtl >> skip >> currentMaterialName;
30.                 break;
31.             case 'K': // Ka Kd Ks
32.                 switch (lineBufferMtl[1])
33.                 {

```



```
34.         case 'a':    // ambient
35.             lineStreamMtl >> skip;
36.             for (int i = 0; i < 3; i++)
37.             {
38.                 lineStreamMtl >> materialTable[currentMaterialName].ambi
ent[i];
39.             }
40.             break;
41.         case 'd':    // diffuse
42.             lineStreamMtl >> skip;
43.             for (int i = 0; i < 3; i++)
44.             {
45.                 lineStreamMtl >> materialTable[currentMaterialName].diff
use[i];
46.             }
47.             break;
48.         case 's':    // specular
49.             lineStreamMtl >> skip;
50.             for (int i = 0; i < 3; i++)
51.             {
52.                 lineStreamMtl >> materialTable[currentMaterialName].spec
ular[i];
53.             }
54.             break;
55.         default:
56.             break;
57.     }
58.     break;
59.     case 'N':    //shininess
60.         lineStreamMtl >> skip >> materialTable[currentMaterialName].shin
iness;
61.         /* wavefront shininess is from [0, 1000], so scale for OpenGL */
62.         materialTable[currentMaterialName].shininess /= 1000;
63.         materialTable[currentMaterialName].shininess *= 128;
64.         break;
65.     default:
66.         break;
67. }
68. }
69. mtl.close();
70. }
71.
72. void OBJImporter::useMaterial(string& lineBuffer, string& currentMaterialNam
e)
```

```

73. {
74.     stringstream lineStream(lineBuffer);
75.     string skip;
76.     lineStream >> skip >> currentMaterialName;
77. }
78.
79. void OBJImporter::readVertex(string& lineBuffer)
80. {
81.     stringstream lineStream(lineBuffer);
82.     string skip;
83.     switch (lineBuffer[1])
84.     {
85.     case 'v':    // v 0 0 0
86.     {
87.         Vertex temp;
88.         lineStream >> skip >> temp.x >> temp.y >> temp.z;
89.         vertexList.push_back(temp);
90.     }
91.         break;
92.     case 't':    // vt 0 0    // 未支持纹理
93.     {
94.         //float x, y;
95.         //lineStream >> skip >> x >> y;
96.         //textureList.push_back(x, y);
97.     }
98.         break;
99.     case 'n':    // vn 0 0 0
100.    {
101.        Normal temp;
102.        lineStream >> skip >> temp.x >> temp.y >> temp.z;
103.        normalList.push_back(temp);
104.    }
105.        break;
106.    default:
107.        break;
108.    }
109. }
110.
111. void OBJImporter::readFace(string& lineBuffer, string& currentMaterialName)
112. {
113.     // can be one of %d, %d//%d, %d/%d, %d/%d/%d
114.     int flag = 0;
115.     for (int i = 0; i < lineBuffer.size(); i++)
116.     {

```

```
117.         if (lineBuffer[i] == '/')
118.         {
119.             if (i + 1 < lineBuffer.size() && lineBuffer[i+1] == '/')
120.             {
121.                 flag = 1;    // "v//n"
122.             }
123.             else
124.             {
125.                 for (int j = i + 1; j < lineBuffer.size(); j++)
126.                 {
127.                     if (lineBuffer[j] == ' ')
128.                     {
129.                         flag = 2;    // "v/t"
130.                         break;
131.                     }
132.                     if (lineBuffer[j] == '/')
133.                     {
134.                         flag = 3;    // "v/t/n"
135.                         break;
136.                     }
137.                 }
138.             }
139.             break;
140.         }
141.     }
142.
143.     // 替换 '/' 为 ' ', 便于处理
144.     for (auto& c : lineBuffer)
145.     {
146.         if (c == '/')
147.         {
148.             c = ' ';
149.         }
150.     }
151.
152.     stringstream lineStream(lineBuffer);
153.     string skip;
154.     lineStream >> skip;
155.
156.     switch (flag)
157.     {
158.     case 0:    // f v v v (v)
159.     {
160.         int a, b, c, d;
```

```

161.         lineStream >> a >> b >> c;
162.         Face temp = {a-1, b-1, c-
163.             1, 0, 0, 0, 0, 0, 0, currentMaterialName}; // 处理下标偏移
164.         faceList.push_back(temp);
165.         // 只支持三角形面和四边形面，四边形面当做两个三角形面处理
166.         if (lineStream.good())
167.         {
168.             lineStream >> d;
169.             Face temp = {a-1, c-1, d-
170.                 1, 0, 0, 0, 0, 0, 0, currentMaterialName};
171.             faceList.push_back(temp);
172.         }
173.         break;
174.     case 1:    // f v//n v//n v//n (v//n)
175.     {
176.         int a, b, c, d;
177.         int an, bn, cn, dn;
178.         lineStream >> a >> an >> b >> bn >> c >> cn;
179.         Face temp = {a-1, b-1, c-1, 0, 0, 0, an-1, bn-1, cn-
180.             1, currentMaterialName};
181.         faceList.push_back(temp);
182.         if (lineStream.good())
183.         {
184.             lineStream >> d >> dn;
185.             Face temp = {a-1, c-1, d-1, 0, 0, 0, an-1, cn-1, dn-
186.                 1, currentMaterialName};
187.             faceList.push_back(temp);
188.         }
189.         break;
190.     case 2:    // f v/t v/t v/t (v/t)
191.     {
192.         // 未支持 vt
193.         int a, b, c, d;
194.         lineStream >> a >> b >> c;
195.         Face temp = {a-1, b-1, c-
196.             1, 0, 0, 0, 0, 0, 0, currentMaterialName}; // 处理下标偏移
197.         faceList.push_back(temp);
198.         if (lineStream.good())
199.         {
200.             lineStream >> d;
201.             Face temp = {a-1, c-1, d-
202.                 1, 0, 0, 0, 0, 0, 0, currentMaterialName};

```

```

199.         faceList.push_back(temp);
200.     }
201. }
202.     break;
203. case 3:     // f v/t/n v/t/n v/t/n (v/t/n)
204. {
205.     // 未支持 vt
206.     int a, b, c, d;
207.     int an, bn, cn, dn;
208.     int skip;
209.     lineStream >> a >> skip >> an >> b >> skip >> bn >> c >> skip >> cn
    ;
210.     Face temp = {a-1, b-1, c-1, 0, 0, 0, an-1, bn-1, cn-
    1, currentMaterialName};
211.     faceList.push_back(temp);
212.     if (lineStream.good())
213.     {
214.         lineStream >> d >> skip >> dn;
215.         Face temp = {a-1, c-1, d-1, 0, 0, 0, an-1, cn-1, dn-
    1, currentMaterialName};
216.         faceList.push_back(temp);
217.     }
218. }
219.     break;
220. default:
221.     break;
222. }
223. }
224.
225. void OBJImporter::smoothNormals()
226. {
227.     // 生成面法线
228.     vector<Normal> faceNormalList;
229.     faceNormalList.resize(faceList.size());
230.     for (int i = 0; i < faceList.size(); i++)
231.     {
232.         Face& f = faceList[i];
233.         Normal& fn = faceNormalList[i];
234.         // 叉积
235.         Normal Vab = {
236.             vertexList[f.b].x - vertexList[f.a].x,
237.             vertexList[f.b].y - vertexList[f.a].y,
238.             vertexList[f.b].z - vertexList[f.a].z
239.         };

```

```

240.         Normal Vac = {
241.             vertexList[f.c].x - vertexList[f.a].x,
242.             vertexList[f.c].y - vertexList[f.a].y,
243.             vertexList[f.c].z - vertexList[f.a].z
244.         };
245.         fn = {
246.             Vab.y * Vac.z - Vab.z * Vac.y,
247.             Vab.z * Vac.x - Vab.x * Vac.z,
248.             Vab.x * Vac.y - Vab.y * Vac.x
249.         };
250.         // 正则化
251.         GLfloat l = (GLfloat)sqrt(fn.x * fn.x + fn.y * fn.y + fn.z * fn.z);
252.         fn = {
253.             fn.x / l,
254.             fn.y / l,
255.             fn.z / l
256.         };
257.     }
258.
259.     // 不需要原有的法线
260.     if (!normalList.empty())
261.     {
262.         normalList.clear();
263.     }
264.
265.     // 为每个顶点添加成员面（所有包含这个顶点的面）
266.     vector<vector<int>> members;
267.     members.resize(vertexList.size());
268.     for (int i = 0; i < faceList.size(); i++)
269.     {
270.         members[faceList[i].a].push_back(i);
271.         members[faceList[i].b].push_back(i);
272.         members[faceList[i].c].push_back(i);
273.     }
274.
275.     // 生成平滑顶点法线
276.     for (int i = 0; i < vertexList.size(); i++)
277.     {
278.         Normal avg = {0, 0, 0};
279.         Normal fn1 = {0, 0, 0};
280.         for (auto& findex : members[i])
281.         {
282.             // 只有第一次或者与第一个面的夹角小于 90 度才添加该面法线
283.             Normal& fn = faceNormalList[findex];

```

```

284.         if (fn1.x * fn1.x + fn1.y * fn1.y + fn1.z * fn1.z == 0)
285.         {
286.             fn1 = {fn.x, fn.y, fn.z};
287.             avg.x += fn.x;
288.             avg.y += fn.y;
289.             avg.z += fn.z;
290.         }
291.         else if (fn1.x * fn.x + fn1.y * fn.y + fn1.z * fn.z)
292.         {
293.             avg.x += fn.x;
294.             avg.y += fn.y;
295.             avg.z += fn.z;
296.         }
297.     }
298.     GLfloat l = (GLfloat)sqrt(avg.x * avg.x + avg.y * avg.y + avg.z * a
vg.z);
299.     avg.x /= l;
300.     avg.y /= l;
301.     avg.z /= l;
302.     normalList.push_back(avg);
303. }
304. }

```

使用读取到的数据，可以绘制模型对象，在原来绘制立方体的基础上，更改了如下函数内容。OpenGL 光照函数参考教材 17.11 节。

对于不同初始大小的模型，示例程序并没有做统一化，其实也并不难，但是会导致代码片段过长。所以这里只是对特定的茶壶模型做平移和旋转操作，以放置在视窗中心。

```

1. void triangle(OBJImporter& obj, Face& f)
2. {
3.     auto& vertexList = obj.vertexList;
4.     auto& normalList = obj.normalList;
5.     auto& materialTable = obj.materialTable;
6.     if (drawLines) {
7.         glColor3f(1.0, 1.0, 1.0);
8.         glBegin(GL_LINE_LOOP);
9.             glVertex3f(vertexList[f.a].x, vertexList[f.a].y, vertexList[f.a]
.z);
10.            glVertex3f(vertexList[f.b].x, vertexList[f.b].y, vertexList[f.b]
.z);

```

```

11.         glVertex3f(vertexList[f.c].x, vertexList[f.c].y, vertexList[f.c]
    .z);
12.         glEnd();
13.     }
14.     else {
15.         glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, materialTable[f.mtl_name
    ].ambient);
16.         glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, materialTable[f.mtl_name
    ].diffuse);
17.         glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, materialTable[f.mtl_name
    ].specular);
18.         glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, materialTable[f.mtl_name
    ].shininess);
19.
20.         glBegin(GL_TRIANGLES);
21.
22.         //glNormal3f(normalList[f.na].x, normalList[f.na].y, normalList[f.na
    ].z);
23.         glNormal3f(normalList[f.a].x, normalList[f.a].y, normalList[f.a].z);
24.         glVertex3f(vertexList[f.a].x, vertexList[f.a].y, vertexList[f.a].z);
25.
26.         //glNormal3f(normalList[f.nb].x, normalList[f.nb].y, normalList[f.nb
    ].z);
27.         glNormal3f(normalList[f.b].x, normalList[f.b].y, normalList[f.b].z);
28.         glVertex3f(vertexList[f.b].x, vertexList[f.b].y, vertexList[f.b].z);
29.
30.         //glNormal3f(normalList[f.nc].x, normalList[f.nc].y, normalList[f.nc
    ].z);
31.         glNormal3f(normalList[f.c].x, normalList[f.c].y, normalList[f.c].z);
32.         glVertex3f(vertexList[f.c].x, vertexList[f.c].y, vertexList[f.c].z);
33.
34.         glEnd();
35.     }
36. }
37.
38. void drawteapot()
39. {
40.     static OBJImporter teapot("./newell_teaset/teapot.obj");
41.
42.     for (auto& f : teapot.faceList)
43.     {
44.         triangle(teapot, f);
45.     }
46. }

```



```
47.
48. void display(void)
49. {
50.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
51.
52.     /* view transform */
53.     glLoadIdentity();
54.     glTranslatef(0.0, 0.0, viewxform_z);
55.
56.     /* Enable a single OpenGL light. */
57.     glEnable(GL_NORMALIZE);
58.     static const GLfloat light_position[] = { 0, 0, 1, 0 };
59.     glLightfv(GL_LIGHT0, GL_POSITION, light_position);
60.     glEnable(GL_LIGHT0);
61.     glEnable(GL_LIGHTING);
62.     glEnable(GL_DEPTH_TEST);
63.     glTranslatef(0, -2, 0);
64.     glRotatef(90, 0, 1, 0);
65.     drawteapot();
66.
67.     glutSwapBuffers();
68. }
```

至此，程序可以加载显示 OBJ 模型。对应实验结果参考中的图一。

导入外部模型（PLY）

斯坦福兔子模型（Stanford Bunny）是图形学领域中的一个经典模型，其提供 PLY 格式模型文件下载¹⁴。实验使用 PLYImporter 类管理 PLY 模型导入，读者可以参考设计出其他三维对象文件格式的导入工具，如 OBJ 格式。

根据上面的实验，我们只需要将模型文件解析出顶点数据和面片数据，构建 vertices 顶点数组，并将顶点数组按照面片数据组合为 polygon，即可在 GLUT 程序中导入外部模型。

PLY 模型结构如下所示，453 个顶点数据，包含坐标和其他数据，948 个面片，首项的 3 表明由 3 个点围成三角形面片，后续的 3 项为顶点数组的序号。

¹⁴ The Stanford 3D Scanning Repository: <http://graphics.stanford.edu/data/3Dscanrep/>

```

1. ply
2. format ascii 1.0
3. comment zipper output
4. element vertex 453
5. property float x
6. property float y
7. property float z
8. property float confidence
9. property float intensity
10. element face 948
11. property list uchar int vertex_indices
12. end_header
13. -0.0312216 0.126304 0.00514924 0.850855 0.5
14. -0.0446774 0.131204 0.00570479 0.900159 0.5
15. -0.0683011 0.144828 0.0413688 0.398443 0.5
16. -0.00600095 0.130398 0.0178986 0.85268 0.5
17. -0.0173568 0.127613 0.00526885 0.675938 0.5
18. ....省略.....
19. ....共 453 个顶点.....
20. 3 164 94 98
21. 3 224 335 49
22. 3 331 350 376
23. 3 124 122 237
24. 3 61 89 69
25. ....省略.....
26. ....共 948 个面片.....

```

由此 PLY 模型文件写出 PLYImporter.h 文件，用于定义 PLYImporter 类解析导入数据。

首先引入需要用到的库和数据结构。Vertex 类储存顶点坐标和顶点颜色。Face 类储存三角形面片的组成顶点的序号。

```

1. #pragma once
2. #include <GL/glut.h>
3. #include <cstdlib>
4. #include <cstring>
5. #include <fstream>
6. #include <iostream>
7.
8. struct Vertex
9. {

```

```
10.     GLfloat x, y, z, r, g, b;
11. };
12.
13. struct Face
14. {
15.     GLint a, b, c;
16. };
```

PLYImporter 类包含如下 public 成员。

构造函数打开 filename，然后进入 parseStream 函数解析输入文件流，用于构造两个主要的成员属性，即 vertexPointer 和 facePointer。

GetVertexPointer 以及之后的几个方法，用于返回 private 属性句柄。

```
1. class PLYImporter
2. {
3. public:
4.     PLYImporter(const char* filename) {
5.         std::ifstream ifs;
6.         ifs.open(filename, std::ifstream::in);
7.         if (!ifs.is_open()) {
8.             std::cout << "open wrong" << std::endl;
9.             return;
10.        }
11.        parseStream(ifs);
12.        ifs.close();
13.    }
14.    ~PLYImporter() {
15.        delete[] vertexPointer;
16.        delete[] facePointer;
17.    }
18.    Vertex* GetVertexPointer() {
19.        return vertexPointer;
20.    }
21.    Face* GetFacePointer()
22.    {
23.        return facePointer;
24.    }
25.    GLint GetVertexNum() {
26.        return vertexNum;
27.    }
28.    GLint GetFaceNum() {
```

```

29.         return faceNum;
30.     }
31. private:
32.     Vertex* vertexPointer; // 指向顶点表
33.     Face* facePointer;     // 指向面片表
34.     GLint vertexNum = 0;
35.     GLint faceNum = 0;

```

私有成员方法 `parseStream` 借助 `getToken` 获取读入行每个 word 的 `start` 和 `end` 坐标, 使用 `matchToken` 比较其是否与 `keyword` 相同。

首先解析 PLY 文件的 `header` 部分, 匹配到 `element vertex xxx` 和 `element face yyy` 所在行, 根据数值分配指定大小的内存空间。

之后进入 `body` 部分, 把所有顶点数据拷贝进 `vertexPointer` 指向的空间, 面片数据拷贝到 `facePointer` 指向的空间。

因为斯坦福兔子模型没有提供顶点颜色或者顶点法线数据, 顶点数据只包含顶点坐标、置信度和强度, 因此这里暂时使用置信度作为 RGB 颜色灰度值。

在 `PLYImporter` 对象创建后, 其成员属性已经就绪。

```

1. void parseStream(std::ifstream& ifs) {
2.     char lineBuffer[256];
3.     // parse header
4.     while (ifs.getline(lineBuffer, 256).good())
5.     {
6.         int start = 0;
7.         int end = 0;
8.         if (getToken(lineBuffer, 256, start, end))
9.         {
10.            if (matchToken(lineBuffer, start, end, "element")
11.                && getToken(lineBuffer, 256, start, end))
12.            {
13.                // match element vertex xxx
14.                if (matchToken(lineBuffer, start, end, "vertex")
15.                    && getToken(lineBuffer, 256, start, end))
16.                {
17.                    // set xxx
18.                    vertexNum = tokenToInt(lineBuffer, start, end);
19.
20.                    // 分配空间
21.                    vertexPointer = new Vertex[vertexNum];

```

```
22.         }
23.
24.         // match element face yyy
25.         if (matchToken(lineBuffer, start, end, "face")
26.             && getToken(lineBuffer, 256, start, end))
27.         {
28.             // set yyy
29.             faceNum = tokenToInt(lineBuffer, start, end);
30.
31.             // 分配空间
32.             facePointer = new Face[faceNum];
33.         }
34.     }
35.
36.     if (matchToken(lineBuffer, start, end, "end_header"))
37.     {
38.         break;
39.     }
40. }
41. }
42. // parse body
43. int vertexCount = 0;
44. int faceCount = 0;
45. while (ifs.getline(lineBuffer, 256).good())
46. {
47.     int start = 0;
48.     int end = 0;
49.     if (vertexCount < vertexNum)
50.     {
51.         getToken(lineBuffer, 256, start, end);
52.         vertexPointer[vertexCount].x = tokenToFloat(lineBuffer, start, end);
53.         getToken(lineBuffer, 256, start, end);
54.         vertexPointer[vertexCount].y = tokenToFloat(lineBuffer, start, end);
55.         getToken(lineBuffer, 256, start, end);
56.         vertexPointer[vertexCount].z = tokenToFloat(lineBuffer, start, end);
57.         //getToken(lineBuffer, 256, start, end);
58.         getToken(lineBuffer, 256, start, end);
59.         vertexPointer[vertexCount].r = tokenToFloat(lineBuffer, start, end);
60.         vertexPointer[vertexCount].g = vertexPointer[vertexCount].r;
61.         vertexPointer[vertexCount].b = vertexPointer[vertexCount].r;
```

```

62.         vertexCount++;
63.     }
64.     else if (faceCount < faceNum)
65.     {
66.         getToken(lineBuffer, 256, start, end); // 去掉开头的 3
67.         getToken(lineBuffer, 256, start, end);
68.         facePointer[faceCount].a = tokenToInt(lineBuffer, start, end);
69.         getToken(lineBuffer, 256, start, end);
70.         facePointer[faceCount].b = tokenToInt(lineBuffer, start, end);
71.         getToken(lineBuffer, 256, start, end);
72.         facePointer[faceCount].c = tokenToInt(lineBuffer, start, end);
73.         faceCount++;
74.     }
75. }
76. }
77. bool getToken(const char* lineBuffer, int length, int& start, int& end) {
78.     if (lineBuffer[end] == '\0') return false;
79.     for (int i = end; lineBuffer[i] <= length; i++)
80.     {
81.         if (lineBuffer[i] != ' ')
82.         {
83.             if (lineBuffer[i] == '\0' || lineBuffer[i] == EOF)
84.             {
85.                 return false;
86.             }
87.             else
88.             {
89.                 start = i;
90.                 break;
91.             }
92.         }
93.     }
94.     for (int i = start; lineBuffer[i] <= length; i++)
95.     {
96.         if (lineBuffer[i] == ' ' || lineBuffer[i] == '\0' || lineBuffer[i] =
= EOF)
97.         {
98.             end = i;
99.             break;
100.        }
101.    }
102.    return true;
103. }

```

```

104. bool matchToken(const char* lineBuffer, int start, int end, const char* key
    word) {
105.     return memcmp(lineBuffer + start, keyword, end - start) == 0;
106. }
107. int tokenToInt(const char* lineBuffer, int start, int end) {
108.     char token[32];
109.     memcpy(token, lineBuffer + start, end - start);
110.     return atoi(token);
111. }
112. double tokenToFloat(const char* lineBuffer, int start, int end) {
113.     char token[32];
114.     memcpy(token, lineBuffer + start, end - start);
115.     return atof(token);
116. }

```

根据绘制立方体的方法，稍作修改就可以得到绘制兔子模型的方法。将 display 函数中的 drawcube 替换为 drawbunny 即可。

```

1. void triangle(Vertex* vertexPointer, int a, int b, int c)
2. {
3.     if (drawLines) {
4.         glColor3f(1.0, 1.0, 1.0);
5.         glBegin(GL_LINE_LOOP);
6.             glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer
[a].z);
7.             glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer
[b].z);
8.             glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer
[c].z);
9.         glEnd();
10.    }
11.    else {
12.        glBegin(GL_TRIANGLES);
13.
14.        glColor3f(vertexPointer[a].r, vertexPointer[a].g, vertexPointer[a].b
);
15.        glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer[a].
z);
16.
17.        glColor3f(vertexPointer[b].r, vertexPointer[b].g, vertexPointer[b].b
);

```

```

18.         glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer[b].
           z);
19.
20.         glColor3f(vertexPointer[c].r, vertexPointer[c].g, vertexPointer[c].b
           );
21.         glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer[c].
           z);
22.
23.         glEnd();
24.     }
25. }
26.
27. void drawbunny()
28. {
29.     static PLYImporter bunny("./bunny/reconstruction/bun_zipper.ply");
30.     Vertex* vertexPointer = bunny.GetVertexPointer();
31.     Face* facePointer = bunny.GetFacePointer();
32.     for (int i = 0; i < bunny.GetFaceNum(); i++)
33.     {
34.         triangle(vertexPointer, facePointer[i].a, facePointer[i].b, facePoin
           ter[i].c);
35.     }
36. }

```

将模型视图移到视野中，程序可以显示出兔子模型的大致轮廓，但表面过于斑驳，这是因为我们将置信度作为颜色值来渲染，实际上模型不包含顶点颜色值。如果需要更真实的显示效果，需要计算顶点的法线值，并给予光照，计算阴影。



OpenGL 光照函数参考教材 17.11 节。修改之~~后~~的代码如下。

绘制三角形面片时，通过面片的两条边计算得出垂直于面片的法线向量，并设置到三个顶点；不再设置顶点颜色，只使用默认的白色。

在 display 函数绘制兔子对象前，开启法线正则化，统一所用到的法线为单位向量；设置一个来自 (1,1,1) 方向的远距离平行光 LIGHT_0；启用 LIGHT_0，启用 LIGHTING 模式。

```
1. void triangle(Vertex* vertexPointer, int a, int b, int c)
2. {
3.     if (drawLines) {
4.         //...
5.     }
6.     else {
7.         glBegin(GL_TRIANGLES);
8.
9.         GLfloat Vab[3] = {
10.             vertexPointer[b].x - vertexPointer[a].x,
11.             vertexPointer[b].y - vertexPointer[a].y,
12.             vertexPointer[b].z - vertexPointer[a].z
13.         };
14.         GLfloat Vac[3] = {
15.             vertexPointer[c].x - vertexPointer[a].x,
16.             vertexPointer[c].y - vertexPointer[a].y,
17.             vertexPointer[c].z - vertexPointer[a].z
18.         };
19.
20.         glNormal3f(
21.             Vab[1] * Vac[2] - Vab[2] * Vac[1],
22.             Vab[2] * Vac[0] - Vab[0] * Vac[2],
23.             Vab[0] * Vac[1] - Vab[1] * Vac[0]
24.         );
25.
26.         //glColor3f(vertexPointer[a].r, vertexPointer[a].g, vertexPointer[a]
27.         .b);
28.         glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer[a].
29.         z);
30.
31.         //glColor3f(vertexPointer[b].r, vertexPointer[b].g, vertexPointer[b]
32.         .b);
33.         glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer[b].
34.         z);
```

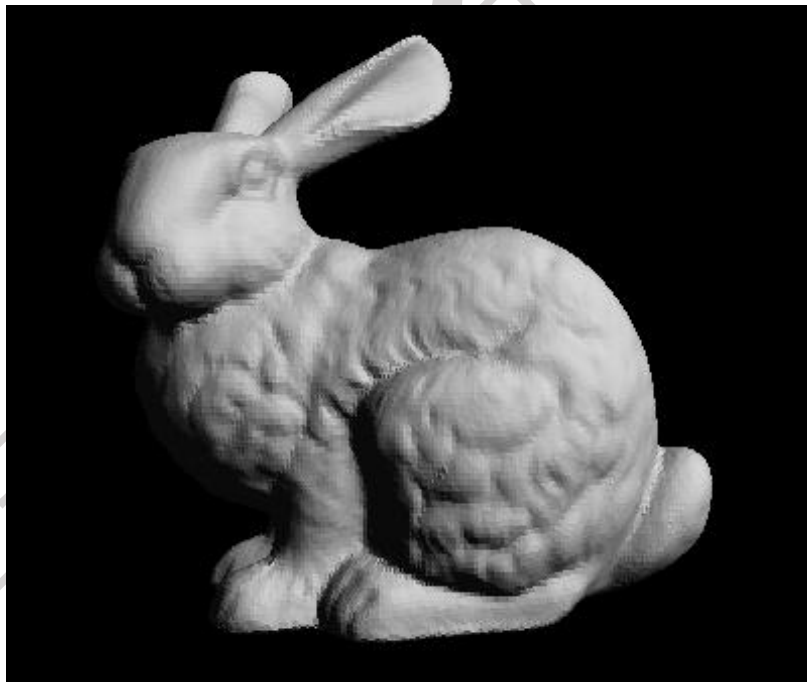
```

31.
32.     //glColor3f(vertexPointer[c].r, vertexPointer[c].g, vertexPointer[c]
    .b);
33.     glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer[c].
    z);
34.
35.     glEnd();
36. }
37. }
38.
39. // ...
40.
41. void display(void)
42. {
43.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
44.
45.     /* view transform */
46.     glLoadIdentity();
47.     glTranslatef(0.0, -2, -5);
48.     glScalef(15, 15, 15);
49.
50.
51.     /* Enable a single OpenGL light. */
52.     glEnable(GL_NORMALIZE);
53.     GLfloat light_position[] = {1.0, 1.0, 1.0, 0.0};
54.     glLightfv(GL_LIGHT0, GL_POSITION, light_position);
55.     glEnable(GL_LIGHT0);
56.     glEnable(GL_LIGHTING);
57.
58.     //glutSolidTeapot(1.0);
59.
60.     //drawcube();
61.     drawbunny();
62.
63.     glutSwapBuffers();
64.     //glutPostRedisplay();
65. }

```

最后，程序可以得到实验结果参考中的图二的结果。

实验结果参考



实验十一 绘图软件

实验目的

使用 OpenGL 函数实现 2D 绘图软件，支持直线绘制，圆形绘制，曲线绘制，区域填充等功能。

实验内容

基于贝塞尔曲线实验，我们可以轻松扩展 GLObject.h 头文件，并实现功能细节，将其扩展为简单绘图软件。

GLLine 和 GLCircle 派生自 GLObject，其中 GLLine 使用 2 个控制点作为起点和终点绘制线段，GLCircle 使用 2 个控制点作为圆心和圆上一点绘制圆形。内部实现可以借由 OpenGL 绘制函数，或通过算法由点生成。所有控制点都可以修改位置，从而修改原始图形。

```
1.  class GLLine : public GLObject
2.  {
3.  public:
4.      GLPoint* ctrlPoints[DEFAULT_LINE_CONTROL_POINTS_NUM];
5.      int count = 0;
6.
7.      virtual bool IsFinished() {
8.          return count >= DEFAULT_LINE_CONTROL_POINTS_NUM ;
9.      }
10.
11.     virtual void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
12.         if (count >= DEFAULT_LINE_CONTROL_POINTS_NUM) return;
13.         ctrlPoints[count] = parentWorld->NewObject<GLPoint>();
14.         ctrlPoints[count]->SetPosition(x, y, z);           // 初始位置
15.         ctrlPoints[count]->movable = true;                 // 可以移动
16.         count++;
17.     }
18.
19.     virtual void Draw(GLenum RenderMode = GL_RENDER) {
20.         // 需要在绘制前初始化控制点
21.         if (count == 0) return;
22.         for (int i = 0; i < count; ++i)
```

```

23.     {
24.         if (!ctrlPoints[i]) return;
25.     }
26.     if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
27.     glColor3f(color.r, color.g, color.b);
28.     glLineWidth(DEFAULT_LINE_WIDTH);
29.     glBegin(GL_LINES);
30.     for (int i = 0; i < count; ++i)
31.     {
32.         GLPoint* ptr = ctrlPoints[i];
33.         glVertex3d(ptr->transform.x, ptr->transform.y, ptr->transform.z);
34.     }
35.     glEnd();
36. }
37. };

```

```

1.  class GLCircle : public GLObject
2.  {
3.  public:
4.      GLPoint* ctrlPoints[DEFAULT_CIRCLE_CONTROL_POINTS_NUM];
5.      int count = 0;
6.
7.      virtual bool IsFinished() {
8.          return count >= DEFAULT_CIRCLE_CONTROL_POINTS_NUM ;
9.      }
10.
11.     virtual void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
12.         if (count >= DEFAULT_CIRCLE_CONTROL_POINTS_NUM) return;
13.         ctrlPoints[count] = parentWorld->NewObject<GLPoint>();
14.         ctrlPoints[count]->SetPosition(x, y, z);           // 初始位置
15.         ctrlPoints[count]->movable = true;                // 可以移动
16.         count++;
17.     }
18.
19.     virtual void Draw(GLenum RenderMode = GL_RENDER) {
20.         // 需要在绘制前初始化控制点
21.         if (count == 0) return;
22.         if (count != 2) return;
23.         for (int i = 0; i < count; ++i)
24.         {
25.             if (!ctrlPoints[i]) return;
26.         }

```

```

27.         if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
28.         glColor3f(color.r, color.g, color.b);
29.         glLineWidth(DEFAULT_LINE_WIDTH);
30.         Circle();
31.     }
32. };

```

GLPaint 同样派生自 GLObject，但行为与上述两者不同。在程序动态创建 GLPaint 对象时，需要初始化该对象坐标和颜色值，从而将初始坐标作为填充种子点，将颜色值作为填充色进行填充计算。填充算法采用扫描线 4 连通方法，从而避免对栈内存的大量消耗。

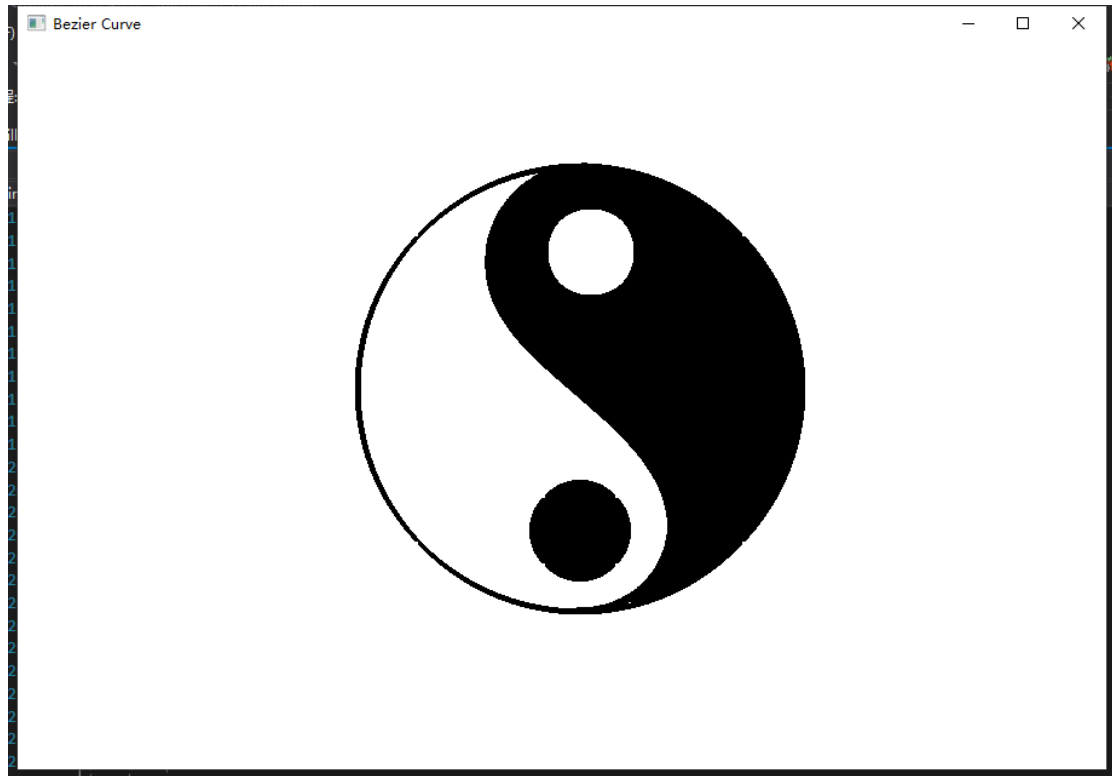
```

1.  class GLPaint : public GLObject
2.  {
3.      public:
4.          virtual void Draw(GLenum RenderMode = GL_RENDER) {
5.              // 需要在绘制前起始位置和颜色
6.              // SetPosition & SetColor
7.              if (finished) return;
8.              if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
9.              glColor3f(color.r, color.g, color.b);
10.             glPointSize(MIN_PAINT_POINT_SIZE);
11.             FloodFill();
12.             finished = true;
13.         }
14. };

```

实验示例作为绘图软件仍有许多工作需要进一步完善。如：图形填充后，填充部分应当写入背景，而不应该被每帧绘制或者清空；动态的获取填充颜色值，实现更加直观的用户接口（调色板）；从 GLObject 扩展出更多图形，如三角形，长方形，以及笔刷。

实验结果参考



附录 1 实验三 填充算法的 C++实现

实验目的

熟悉 OpenGL 图形绘制管线，使用 OpenGL 画点函数实现边界填充算法和泛滥填充算法。

实验内容

图形填充

参考教材 6.13 节，实现对不规则边界区域的两种填充算法。

由于书中并没有规定“颜色”的数据结构和操作方法，在这里给出一种实现方案。

```
1. struct Color
2. {
3.     GLfloat r, g, b;
4.
5.     Color() = default;
6.     Color(GLfloat inR, GLfloat inG, GLfloat inB)
7.         : r(inR), g(inG), b(inB)
8.     {}
9.
10.    bool operator==(Color otherColor)
11.    {
12.        return r == otherColor.r && g == otherColor.g && b == otherColor.b;
13.    }
14.    bool operator!=(Color otherColor)
15.    {
16.        return !(*this == otherColor);
17.    }
18. };
19.
20. void getPixel(GLint x, GLint y, Color& ptColor)
21. {
22.     glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, (GLvoid*)&ptColor);
23. }
```


Color 结构保存 RGB 三个浮点数，并提供构造函数和比较方法。

使用 `getPixel` 封装的 `glReadPixels` 获取某一像素位置的颜色，并将数据写回需要保存的位置。

后续改进此函数可能会导致栈溢出错误，问题根源在于

3. 世界坐标偏移和屏幕像素偏移不同，计算连通时容易越过填充边界。
4. 连通递归算法本身消耗大量栈空间，即使算法逻辑正确，在大面积填充时容易超出程序保留的可用栈空间¹⁵。

然后应当修改两种算法函数中的数据类型。

```
1. void boundaryFill4 (GLint x, GLint y, Color fillColor, Color borderColor)
2. {
3.     Color interiorColor;
4. }
```

为了暂时保证 `getPixel` 函数能正常运行，需要让裁剪空间坐标系和窗口屏幕像素坐标系重合，且程序窗口不支持缩放。

```
1. void init (void)
2. {
3.     glClearColor (0.0, 0.0, 0.0, 0.0);
4.     gluOrtho2D (0.0, 600.0, 0.0, 300.0);
5. }
6.
7. void main (int argc, char** argv)
8. {
9.     glutInitWindowSize (600, 300);
10. }
```

然后设置自定义的形状用于算法的颜色填充，这部分放在 `renderScene` 函数中。

边界填充算法只需要绘制出填充边界，然后给出边界内一点。

```
1. void renderScene(void)
2. {
```

¹⁵ 在 VS 项目属性 - 链接器 - 系统 - 堆栈保留大小设定了程序的可用栈空间，默认为 1MB。

```
3.     glColor3f (1, 1, 1);
4.     glBegin(GL_LINE_LOOP);
5.         glVertex2i(50, 100);
6.         glVertex2i(150, 100);
7.         glVertex2i(75, 200);
8.         glVertex2i(175, 200);
9.     glEnd();
10.    glFlush();
11.
12.    glColor3f (1, 0, 0);
13.    boundaryFill4(100, 101, Color(1, 0, 0), Color(1, 1, 1));
14.    glFlush();
15. }
```

泛滥填充算法可以使用多种不同颜色的边界，对内部单一颜色重新上色。示例代码绘制了三个不同颜色的三角形，中心围出的区域用白色填充。

```
1. void renderScene(void)
2. {
3.     glColor3f (1, 0, 0);
4.     glBegin(GL_POLYGON);
5.         glVertex2i(225, 100);
6.         glVertex2i(360, 100);
7.         glVertex2i(300, 70);
8.     glEnd();
9.     glColor3f (0, 1, 0);
10.    glBegin(GL_POLYGON);
11.        glVertex2i(310, 175);
12.        glVertex2i(250, 100);
13.        glVertex2i(200, 125);
14.    glEnd();
15.    glColor3f (0, 0, 1);
16.    glBegin(GL_POLYGON);
17.        glVertex2i(280, 150);
18.        glVertex2i(350, 100);
19.        glVertex2i(375, 125);
20.    glEnd();
21.    glFlush();
22.
23.    glColor3f (1, 1, 1);
24.    floodFill4(300, 101, Color(1, 1, 1), Color(0, 0, 0));
25.    glFlush();
}
```

```
26. }
```

可以额外设置一个 OpenGL 自动填充的多边形图元作为对比。

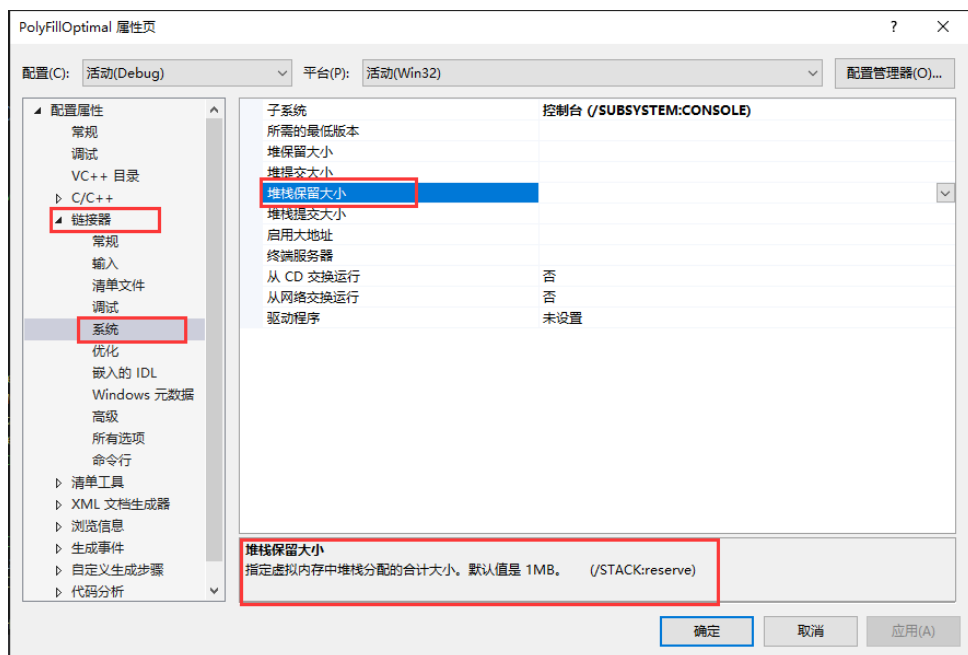
```
1. void renderScene(void)
2. {
3.     glColor3f (1, 1, 1);
4.     glBegin(GL_POLYGON);
5.         glVertex2i(450, 100);
6.         glVertex2i(550, 100);
7.         glVertex2i(475, 200);
8.         glVertex2i(575, 200);
9.     glEnd();
10. }
```

在填充算法内部，还可以添加 `glFlush` 调用，每次绘制点后刷新缓冲区，可以看到动态的填充过程。

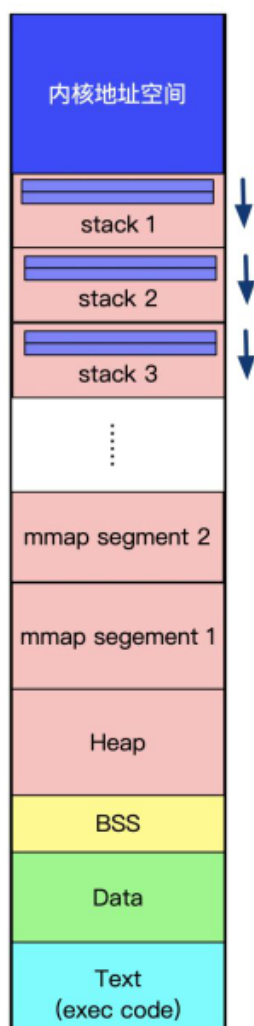
栈空间分析

为了说明教材 6.13 节中关于填充算法对栈空间的大量需求，对上述程序代码作如下改动。

已知 VC++ 项目程序的默认栈保留空间为 1MB，如图所示，程序在装入内存时会声明自己需求的栈空间大小，由系统进行分配。



内存模型如下图所示，程序线程栈空间由操作系统分配后，在栈空间内向下增长。模型上方为高地址，下方为低地址。



添加全局变量：

- stackMemSize 储存程序实际占用的最大栈内存空间
- stackMemBasePtr 储存程序的栈底基址
- memInfo 是调用 VirtualQuery 函数所需的储存结构，函数调用结束时内存信息被写入，该部分需引入 windows.h 头文件¹⁶

在 main 函数中添加一个局部变量 stackBase，以此作为虚拟内存查询函数 VirtualQuery 的寻址目标，将结果中的基址写入 stackMemBasePtr。

```

1. int stackMemSize = 0;
2. void* stackMemBasePtr = nullptr;
3. MEMORY_BASIC_INFORMATION memInfo;
4.
5. void main (int argc, char** argv)
6. {
7.     int stackBase = 0;
8.     if (VirtualQuery(&stackBase, &memInfo, sizeof(memInfo))) {
9.         stackMemBasePtr = memInfo.BaseAddress;
10.        printf("StackBase : 0x%p\n", memInfo.BaseAddress);
11.    }
12.    // ...
13. }
```

在算法内部借助 Color 结构进行寻址，使用内存栈底的高地址减去低地址，获取最大栈深度，将计算结果储存到 stackMemSize。

```

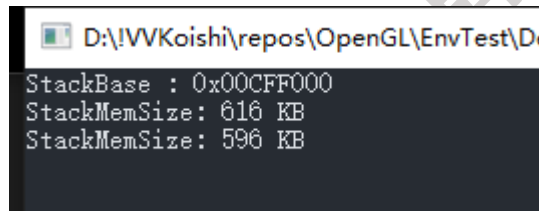
1. void boundaryFill4 (GLint x, GLint y, Color fillColor, Color borderColor)
2. {
3.     Color interiorColor;
4.
5.     if (VirtualQuery((LPCVOID)&interiorColor, &memInfo, sizeof(memInfo))) {
6.         int stackDeep = (int)stackMemBasePtr - (int)memInfo.BaseAddress;
7.         if (stackDeep > stackMemSize) {
8.             stackMemSize = stackDeep;
9.         }
10.    }
11.    // ...
12. }
```

¹⁶ 也可以直接使用取地址符号&来获取地址并计算，最终计算结果相近。

在填充算法调用前后，对 `stackMemSize` 进行初始化，并打印实际使用的栈空间大小。

```
1. stackMemSize = 0;
2.
3. glColor3f (1, 0, 0);
4. boundaryFill4(100, 101, Color(1, 0, 0), Color(1, 1, 1));
5. glFlush();
6.
7. printf("StackMemSize: %d KB \n", stackMemSize / 1024);
```

在命令行窗口可以看到，在示例代码的条件下，两种填充算法对栈空间的实际需求分别为 616KB 和 596KB，而程序默认栈大小为 1MB，因此当需要填充的面积较大时，算法很容易栈溢出（比如填充一个平行四边形，而不是交叉后形成的小三角形）。



The screenshot shows a command prompt window with the following text:

```
D:\VVKoishi\repos\OpenGL\EnvTest\De
StackBase : 0x00CFF000
StackMemSize: 616 KB
StackMemSize: 596 KB
```

扫描线 4 连通算法

为了对区域填充算法进行栈空间优化，使用扫描线方式填充水平像素区段，详见教材 6.13 节。

在扫描线方式的 4 连通区域填充算法中，维护 `pointStack` 填充起点栈，对栈尾元素点位置进行线段填充，在每次线段填充完成后，对该条线段上下连通的待填充区域的起点压入栈中，直到栈为空，填充算法结束。

```
1. struct Point
2. {
3.     GLdouble x;
4.     GLdouble y;
5. };
6.
7. void scanLineFilling(GLdouble x, GLdouble y, Color fillColor, Color interior
    Color)
8. {
```

```
9.     std::vector<Point> pointStack;
10.
11.     fillLine(x, y, fillColor, interiorColor, pointStack);
12.
13.     while (!pointStack.empty())
14.     {
15.         GLdouble popX = pointStack.back().x;
16.         GLdouble popY = pointStack.back().y;
17.         pointStack.pop_back();
18.         fillLine(popX, popY, fillColor, interiorColor, pointStack);
19.     }
20. }
```

函数 `fillLine` 负责填充该位置所在的线段，并且在填充完成后，将连通的待填充区域压入栈中。

```
1. void fillLine(GLdouble x, GLdouble y, Color fillColor, Color interiorColor,
2.               std::vector<Point>& pointStack)
3. {
4.     GLdouble startX = x;
5.     GLdouble startY = y;
6.
7.     GLdouble leftX = x;
8.     GLdouble rightX = x;
9.
10.    // 先向右后向左填充
11.    Color oldColor;
12.    getPixel(x, y, oldColor);
13.    while (oldColor == interiorColor && oldColor != fillColor)
14.    {
15.        setPixel(x, y);
16.        x = x + dx;
17.        getPixel(x, y, oldColor);
18.    }
19.    rightX = x - dx;
20.    x = startX - dx;
21.    getPixel(x, y, oldColor);
22.    while (oldColor == interiorColor && oldColor != fillColor)
23.    {
24.        setPixel(x, y);
25.        x = x - dx;
26.        getPixel(x, y, oldColor);
27.    }
```

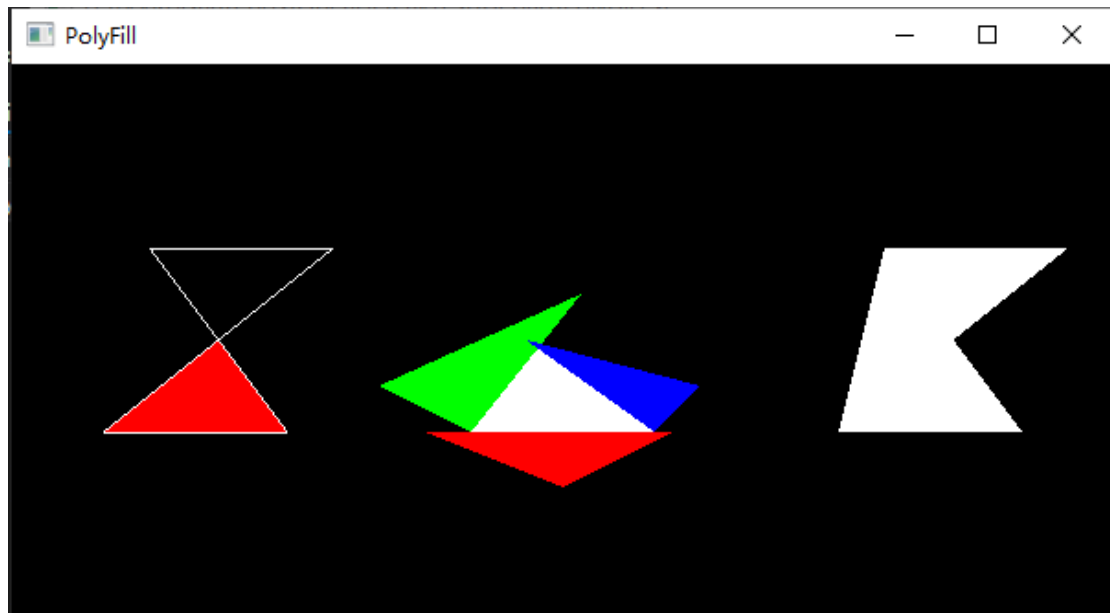
```

26.     }
27.     leftX = x + dx;
28.
29.     // 从左至右,先下后上, 读取 4 连通
30.     x = leftX;
31.     y = startY - dy;
32.     bool startContinue = false;
33.     getPixel(x, y, oldColor);
34.     while (x <= rightX)
35.     {
36.         if (!startContinue) {
37.             if (oldColor == interiorColor && oldColor != fillColor) {
38.                 pointStack.push_back({ x,y });
39.                 startContinue = true;
40.             }
41.         }
42.         else {
43.             if (oldColor != interiorColor || oldColor == fillColor) startContinue = false;
44.         }
45.         x = x + dx;
46.         getPixel(x, y, oldColor);
47.     }
48.
49.     x = leftX;
50.     y = startY + dy;
51.     startContinue = false;
52.     getPixel(x, y, oldColor);
53.     while (x <= rightX)
54.     {
55.         if (!startContinue) {
56.             if (oldColor == interiorColor && oldColor != fillColor) {
57.                 pointStack.push_back({ x,y });
58.                 startContinue = true;
59.             }
60.         }
61.         else {
62.             if (oldColor != interiorColor || oldColor == fillColor) startContinue = false;
63.         }
64.         x = x + dx;
65.         getPixel(x, y, oldColor);
66.     }

```


67. }

实验结果参考



附录 2 实验八 贝塞尔曲线的 C++实现

实验目的

巩固样条曲线-贝塞尔曲线构造原理，使用 OpenGL 函数实现贝塞尔曲线的绘制。

实验内容

参考教材 14.8 节，可以绘制一个简易的 Bezier 曲线，但控制点被硬编码，无法动态修改曲线形状。

完整实现一个支持动态修改的 Bezier 曲线程序，需要

3. 动态创建控制点（对象资源管理）
4. 选取和修改控制点（OpenGL 拾取）

该实验不要求读者严格按照实验指导书来编写，可以只实现教材中的简易版本；如果读者熟悉 C++ 特性，那么本实验会是一个还不错的练习机会。

GLObject.h 头文件

参考教材 14.8 节和 14.9 节，根据公式推导使用 OpenGL 画点函数绘制贝塞尔曲线。

创建 GLObject.h 头文件，在其中定义 GLObject 相关类。该文件中的类型说明如下：

GLState	用于判断当前操作状态
GLColor	存储颜色值的结构体
GLTransform	存储变换值的结构体
GLObject	自定义对象类型，表示程序中的常用对象，包含 GLColor 和 GLTransform 结构成员
GLWorld	自定义世界类型，表示程序运行的世界环境，包含 GLObject 指针数组，管理所有 GLObject 对象
GLPoint	派生自 GLObject，重写父类虚函数 Draw，绘制点
GLBezier	派生自 GLObject，重写父类虚函数 Draw，绘制贝塞尔曲线，初始化时创建 4 个控制点对象

此外，宏定义程序中的部分硬编码数值。

```

1. #define MAX_OBJECTS 10000
2. #define DEFAULT_POINT_SIZE 10
3. #define DEFAULT_LINE_WIDTH 5
4. #define DEFAULT_BEZIER_CONTROL_POINTS_NUM 4
5. #define MAX_BEZIER_CONTROL_POINTS_NUM 13
6. #define DEFAULT_PICK_BUFFER_SIZE 32
7. #define DEFAULT_PICK_WINDOW_SIZE 10
8. #define PI 3.14159265358979323846 // pi
9.
10. enum GLState {
11.     MoveObject,
12.     MakePoint
13. };
14.
15. struct GLColor;
16. struct GLTransform;
17.
18. class GObject;
19. class GLWorld;
20.
21. // GObject 派生类
22. class GLPoint; // 点
23. class GLBezier; // 贝塞尔曲线
24.
25.
26. struct GLColor
27. {
28.     GLfloat r = 0, g = 0, b = 0;
29. };
30.
31. struct GLTransform
32. {
33.     // 世界坐标, 轴旋转, 轴缩放
34.     GLdouble x = 0, y = 0, z = 0;
35.     GLdouble rx = 0, ry = 0, rz = 0;
36.     GLdouble sx = 1, sy = 1, sz = 1;
37. };

```

GObject 类的定义如下图所示。成员变量和成员方法解释如下：

id GObject 类对象唯一标识符

color	GLColor 结构颜色值
transform	GLTransform 结构变换值
movable	对象是否可以移动
visible	对象是否可见
parentWorld	所在世界对象指针
Draw	绘制函数，由派生类重写
SetPosition	设置位置值的封装函数

```

1. class GObject
2. {
3. public:
4.     GLint id = 0;
5.     GLColor color;
6.     GLTransform transform;
7.     bool movable = false;
8.     bool visible = true;
9.
10.    GLWorld* parentWorld;
11.
12.    virtual void Draw(GLenum RenderMode = GL_RENDER) {}
13.
14.    virtual void MakePoint(GLdouble x, GLdouble y, GLdouble z) {}
15.
16.    void SetPosition(GLdouble x, GLdouble y, GLdouble z) {
17.        transform.x = x;
18.        transform.y = y;
19.        transform.z = z;
20.    }
21. };

```

GLWorld 类的定义如下图所示。成员变量和成员方法解释如下：

objects	GObject*数组，储存世界对象下所有 GObject 对象
count	用作 objects 数组的下标索引和对象计数
GLWorld	构造函数和析构函数
~GLWorld	
NewObject	动态创建一个 GObject 类型，添加到 objects 数组，返回该类型指针

DeleteObject	根据 id 删除一个对象
Empty	清空所有对象
DrawObjects	循环调用所有对象的 Draw 方法，缺省渲染方式为 GL_RENDER，可选值还有 GL_SELECT
pickedObjectId	当前选取的对象 ID
PickObject	根据鼠标输入选取对象，获得对象 ID 储存到 pickedObjectId 变量

```

1. class GLWorld
2. {
3. public:
4.     GObject* objects[MAX_OBJECTS];
5.     GLint count = 0;
6.
7.     GLWorld() = default;
8.     ~GLWorld() {
9.         for (int i = count - 1; i >= 0; --i)
10.        {
11.            delete objects[i];
12.        }
13.    }
14.
15.    template<typename T>
16.    T* NewObject() {
17.        if (count < MAX_OBJECTS)
18.        {
19.            T* ret = new T;
20.            objects[count] = (GObject*) ret;
21.            objects[count]->parentWorld = this;
22.            objects[count]->id = count;
23.            ++count;
24.            return ret;
25.        }
26.        else return nullptr;
27.    }
28.
29.    void DeleteObject(GLint id);
30.    void Empty();
31.
32.    void DrawObjects(GLenum RenderMode = GL_RENDER) {
33.        for (int i = 0; i < count; ++i)
34.        {

```

```

35.         objects[i]->Draw(RenderMode);
36.     }
37. }
38.
39. GLint pickedObjectId = -1;
40. void PickObject(GLint xMouse, GLint yMouse) {
41.     pickRects(GLUT_LEFT_BUTTON, GLUT_DOWN, xMouse, yMouse);
42. }
43. private:
44.     void pickRects (GLint button, GLint action, GLint xMouse, GLint yMouse);
45.     void processPicks (GLint nPicks, GLuint pickBuffer [ ]);
46. };

```

类中两个私有方法用于执行 OpenGL 拾取操作，可参考教材 20.6.7 节实现。不同的地方在于：

- 这两个函数变为类成员方法
- pickRects 方法在使用 OpenGL 名称拾取栈时，只使用一个栈顶元素
- pickRects 方法需要指定拾取裁剪窗口，所以需要获得裁剪坐标
- pickRects 方法不绘制方块，而是调用 DrawObjects(GL_SELECT)
- processPicks 方法只需要为 pickedObjectId 赋值

```

1.     // 使用外部定义的世界坐标系裁剪空间，供设置选取时的裁剪窗口
2.     extern GLfloat xwcMin, xwcMax, ywcMin, ywcMax;
3.
4.     void GLWorld::pickRects (GLint button, GLint action, GLint xMouse,
5.                             GLint yMouse)
6.     {
7.         GLuint pickBuffer [DEFAULT_PICK_BUFFER_SIZE];
8.         GLint nPicks, vpArray [4];
9.         if (button != GLUT_LEFT_BUTTON || action != GLUT_DOWN)
10.            return;
11.
12.         glSelectBuffer (DEFAULT_PICK_BUFFER_SIZE, pickBuffer); // Designate pick buffer.
13.         glRenderMode (GL_SELECT); // Activate picking operations.
14.         glInitNames ( ); // Initialize the object-ID stack.
15.
16.         // 只使用一个栈顶元素，后续为图形命名使用 glLoadName(id) 替换栈顶

```

```
17.     glPushName(MAX_OBJECTS);
18.
19.     /* Save current viewing matrix. */
20.     glMatrixMode (GL_PROJECTION);
21.     glPushMatrix ( );
22.     glLoadIdentity ( );
23.
24.     /* Obtain the parameters for the current viewport. Set up
25.     * a 5 x 5 pick window, and invert the input yMouse value
26.     * using the height of the viewport, which is the fourth
27.     * element of vpArray.
28.     */
29.     glGetIntegerv (GL_VIEWPORT, vpArray);
30.     gluPickMatrix (GLdouble (xMouse), GLdouble (vpArray [3] - yMouse),
31.                    DEFAULT_PICK_WINDOW_SIZE, DEFAULT_PICK_WINDOW_
32.                    SIZE,
33.                    vpArray);
34.     gluOrtho2D (xwcMin, xwcMax, ywcMin, ywcMax);
35.     //rects (GL_SELECT);      // Process the rectangles in selecti
36.     this->DrawObjects(GL_SELECT);
37.
38.     /* Restore original viewing matrix. */
39.     glMatrixMode (GL_PROJECTION);
40.     glPopMatrix ( );
41.
42.     glFlush ( );
43.
44.     /* Determine the number of picked objects and return to the
45.     * normal rendering mode.
46.     */
47.     nPicks = glRenderMode (GL_RENDER);
48.     processPicks (nPicks, pickBuffer); // Process picked objects.
49.
50.     glutPostRedisplay ( );
51. }
52.
53. void GLWorld::processPicks (GLint nPicks, GLuint pickBuffer [ ])
54. {
55.     // 储存获取到的最后一个 ID, 即认为后绘制的图像在先绘制的图形之上
56.     if (nPicks == 0) pickedObjectId = -1;
```

```

57.     else
58.     {
59.         // 因为只使用一个栈顶元素，所以可以由公式计算得出最后一个图形
ID
60.         int chunkId = 4 * (nPicks - 1);
61.         pickedObjectId = pickBuffer[chunkId + 3];
62.     }
63.     return;
64. }

```

GLPoint 类 public 继承自 GObject，用于生成贝塞尔曲线控制点。Draw 方法在 GL_RENDER 和 GL_SELECT 模式下，只多一行 glLoadName(id) 替换栈顶元素的代码。

```

1.  class GLPoint : public GObject
2.  {
3.  public:
4.      GLPoint() {
5.          color.r = 1;
6.      }
7.
8.      virtual void Draw(GLenum RenderMode = GL_RENDER) {
9.          // 需要在绘制前调用 SetPostion，否则绘制在 origin
10.         if (!visible) return;
11.         if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
12.         glColor3f(color.r, color.g, color.b);
13.         glPointSize(DEFAULT_POINT_SIZE);
14.         glBegin(GL_POINTS);
15.         glVertex3d(transform.x, transform.y, transform.z);
16.         glEnd();
17.     }
18. };

```

GLBezier 类与绘制点类相似，在调用时可以使用 GLWorld::NewObject <GLBezier> () -> Init() 完成 4 控制点的贝塞尔曲线对象创建和初始化，也可以在创建空 GLBezier 对象后，使用 MakePoint 方法添加控制点。

贝塞尔曲线绘制参考教材 14.8.2 节实现，需注意在控制点数量较多时，系数在阶乘阶段可能会超出数据范围，导致曲线错误。


```

class GLBezier : public GObject
{
public:
    GLPoint* ctrlPoints[MAX_BEZIER_CONTROL_POINTS_NUM];
    int count = 0;
    GLint nBezCurvePoints = 1000;

    // Usage:
    // GLWorld::NewObject<GLBezier>()->MakePoint(x,y,z);
    virtual void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
        if (count >= MAX_BEZIER_CONTROL_POINTS_NUM) return;
        ctrlPoints[count] = parentWorld->NewObject<GLPoint>();
        ctrlPoints[count]->SetPosition(x, y, z);          // 初始位置
        ctrlPoints[count]->movable = true;                // 可以移动
        count++;
    }

    virtual void Draw(GLenum RenderMode = GL_RENDER) {
        // 需要在绘制前初始化控制点
        if (count == 0) return;
        for (int i = 0; i < count; ++i)
        {
            if (!ctrlPoints[i]) return;
        }
        if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
        glColor3f(color.r, color.g, color.b);
        glPointSize(DEFAULT_LINE_WIDTH);
        bezier(ctrlPoints, nBezCurvePoints);
    }
    // ...
};

```

上述代码中的 `bezier(ctrlPoint, nBezCurvePoints)` 实现为私有成员函数，实质上就是教材中 14.8.2 节代码的封装。

```

1. class GLBezier : public GObject
2. {
3.     // ...
4. private:
5.     class wcPt3D {
6.     public:

```

```

7.         GLfloat x, y, z;
8.     };
9.     void plotPoint (wcPt3D bezCurvePt);
10.    void binomialCoeffs (GLint n, GLint * C);
11.    void computeBezPt (GLfloat u, wcPt3D * bezPt, GLint nCtrlPts, wcPt3D * c
    trlPts, GLint * C);
12.    void bezier (wcPt3D * ctrlPts, GLint nCtrlPts, GLint nBezCurvePts);
13.    void bezier(GLPoint* ctrlPts[], GLint nBezCurvePts) {
14.        // 函数重载转发
15.        wcPt3D innerCtrlPts[MAX_BEZIER_CONTROL_POINTS_NUM];
16.        for (int i = 0; i < count; ++i)
17.        {
18.            innerCtrlPts[i].x = (GLfloat) ctrlPts[i]->transform.x;
19.            innerCtrlPts[i].y = (GLfloat) ctrlPts[i]->transform.y;
20.            innerCtrlPts[i].z = (GLfloat) ctrlPts[i]->transform.z;
21.        }
22.        bezier(innerCtrlPts, count, nBezCurvePts);
23.
24.        // 绘制控制点连线
25.        if (ctrlPts[0] && ctrlPts[0]->visible == false) return;
26.        glColor3f(0.0, 1.0, 0.0);
27.        glLineWidth(2.0);
28.        glBegin(GL_LINE_STRIP);
29.        for (int i = 0; i < count; ++i)
30.        {
31.            glVertex3d(ctrlPts[i]->transform.x, ctrlPts[i]->transform.y, ctr
    lPts[i]->transform.z);
32.        }
33.        glEnd();
34.    }
35. };

```

Bezier.cpp 源文件

在源文件顶部引用“GLObject.h”，并创建 MyWorld 全局变量，设置程序窗口长宽和裁剪窗口坐标。

```

1. #include "GLObject.h"
2.
3. GLWorld MyWorld;

```

```

4.
5. GLState MyState = MoveObject;
6.
7. /* Set initial size of the display window. */
8. GLsizei winWidth = 900, winHeight = 600;
9.
10. /* Set size of world-coordinate clipping window. */
11. GLfloat xwcMin = -450.0, xwcMax = 450.0;
12. GLfloat ywcMin = -300.0, ywcMax = 300.0;

```

设置菜单绑定，支持右键新建贝塞尔曲线。即在右键菜单中选取 Bezier 曲线后，对全局世界对象调用模板函数 `MyWorld.NewObject<GLBezier>()` 构建一个 GLBezier 对象，之后进入 `GLState::MakePoint` 状态，为曲线添加控制点。

在贝塞尔曲线完成后，可以再次点击菜单项结束绘制。之后可以通过鼠标点选控制点进行变换。

```

1. struct menuEntryStruct {
2.     const char *label;
3.     char key;
4. };
5.
6. static menuEntryStruct mainMenu[] = {
7.     "Reset",          '0',
8.     "Create Bezier/Finish Bezier", '1',
9.     "Quit",           27, //ESC 键 (ASCII: 27)
10. };
11. int mainMenuEntries = sizeof(mainMenu) / sizeof(menuEntryStruct);
12.
13.
14. bool globalControlPointsVisible = true;
15.
16. void userEventAction(int key) {
17.     switch (key) {
18.     case '0':
19.         MyWorld.Empty();
20.         MyWorld.pickedObjectId = -1;
21.         MyState = MoveObject;
22.         break;
23.     case '1': // bezier
24.         if (MyState == MoveObject)
25.             {

```

```

26.         GLBezier* bezPtr = MyWorld.NewObject<GLBezier>();
27.         MyState = MakePoint;
28.         MyWorld.pickedObjectId = bezPtr->id;
29.     }
30.     else if (MyState == MakePoint)
31.     {
32.         MyState = MoveObject;
33.         MyWorld.pickedObjectId = -1;
34.     }
35.     break;
36. case 27:         // ESC 键 (ASCII: 27) 退出
37.     MyWorld.Empty();
38.     exit(0);
39.     break;
40. default:
41.     break;
42. }
43. glutPostRedisplay();    // 重绘
44. }

```

显示函数中调用 `MyWorld.DrawObjects(GL_RENDER)` 将 `objects` 数组中动态生成的对象全部绘制。

```

1. void displayFcn (void)
2. {
3.     glClear (GL_COLOR_BUFFER_BIT);    // Clear display window.
4.     MyWorld.DrawObjects(GL_RENDER);
5.     glFlush ( );
6. }

```

鼠标按键绑定函数中，支持左键按下拾取操作，并且在左键弹起时重置 `pickedObjectId` 变量值，如果当前状态为绘制控制点状态，在鼠标按下的位置调用 `MakePoint` 方法，弹起时刷新窗口。

```

1. void unProject(GLdouble winx, GLdouble winy, GLdouble winz, GLdouble *objx,
   GLdouble *objy, GLdouble *objz)
2. {
3.     // 从窗口到世界坐标系
4.     GLint viewport[4];
5.     GLdouble modelview[16];

```

```
6.     GLdouble projection[16];
7.     glGetDoublev(GL_MODELVIEW_MATRIX, modelview);
8.     glGetDoublev(GL_PROJECTION_MATRIX, projection);
9.     glGetIntegerv(GL_VIEWPORT, viewport);
10.    gluUnProject(winx, winy, winz, modelview, projection, viewport, objx, ob
        jy, objz);
11. }
12.
13. void mouseButton(int button, int state, int x, int y)
14. {
15.     switch (button) {
16.     case GLUT_LEFT_BUTTON:
17.         break;
18.     default:
19.         return;
20.     }
21.     switch (state) {
22.     case GLUT_DOWN:
23.         if (MyState == MoveObject)
24.         {
25.             MyWorld.PickObject(x, y);
26.             printf("MyWorld.pickedObjectId: %d \n", MyWorld.pickedObjectId);
27.         }
28.         else if (MyState == MakePoint)
29.         {
30.             if (GLObject* ptr = MyWorld.objects[MyWorld.pickedObjectId])
31.             {
32.                 // 从窗口到世界坐标系
33.                 GLdouble winx = (GLdouble) x;
34.                 GLdouble winy = (GLdouble) winHeight - (GLdouble) y;
35.                 GLdouble winz = 0;
36.                 GLdouble objx = 0;
37.                 GLdouble objy = 0;
38.                 GLdouble objz = 0;
39.                 unProject(winx, winy, winz, &objx, &objy, &objz);
40.                 ptr->MakePoint(objx, objy, objz);
41.             }
42.         }
43.         break;
44.     case GLUT_UP:
45.         if (MyState == MoveObject)
46.         {
47.             MyWorld.pickedObjectId = -1;
```

```
48.     }
49.     glutPostRedisplay();
50.     break;
51. }
52. }
```

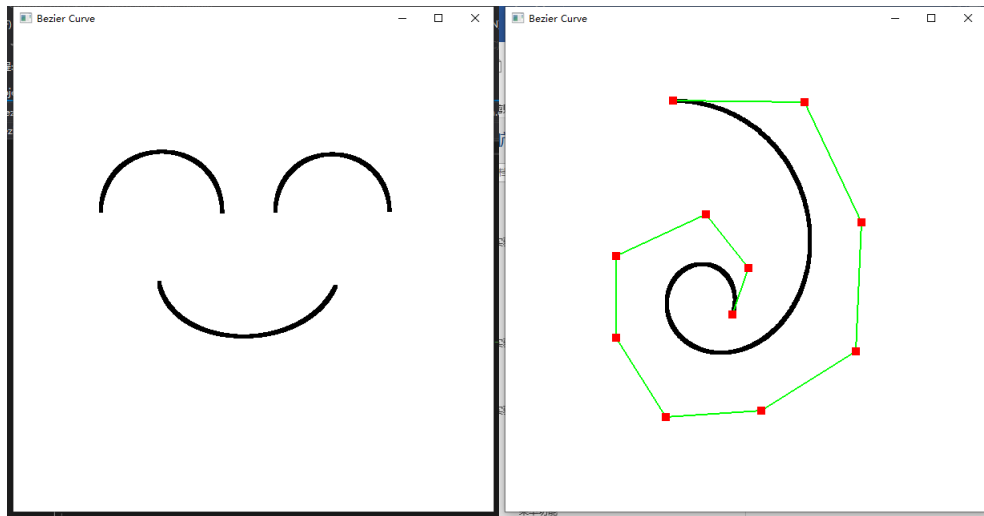
鼠标滑动绑定函数中，对当前选取的对象 ID，检查是否可以移动，然后通过 objects 数组获取对象指针。

使用 gluUnProject 将鼠标指针的屏幕坐标反投影回世界坐标系，将得到的世界坐标系值储存到临时变量，并对当前选取对象重新设置坐标位置。

最后使用 glutPostRedisplay 通知窗口重新绘制。

```
1. void mouseMotion(int x, int y)
2. {
3.     if ( MyWorld.pickedObjectId == -1 ) return;
4.     if ( ! MyWorld.objects[MyWorld.pickedObjectId]->movable ) return;
5.
6.     GObject* ptr = MyWorld.objects[MyWorld.pickedObjectId];
7.
8.     // 从窗口到世界坐标系
9.     GLdouble winx = (GLdouble) x;
10.    GLdouble winy = (GLdouble) winHeight - (GLdouble) y;
11.    GLdouble winz = 0;
12.    GLdouble objx = 0;
13.    GLdouble objy = 0;
14.    GLdouble objz = 0;
15.    unProject(winx, winy, winz, &objx, &objy, &objz);
16.
17.    ptr->SetPosition(objx, objy, objz);
18.
19.    glutPostRedisplay();
20. }
```

实验结果参考



附录 3 实验十 C 语言实现

实验目的

了解常用 3D 模型文件格式，并支持程序导入和读取模型文件。

实验内容

该实验基于之前的实验一“立方体绘制”的整体代码框架，将立方体模型替换为其它外部模型。

导入外部模型（OBJ）

OBJ 文件是一种常见的 3D 模型文件格式¹⁷。本实验示例使用 Newell teaset¹⁸ 的茶壶模型，读者可以替换其它模型。

OBJLoader 或者 OBJImporter 负责将.obj 文件数据解析成我们需要使用的数据。在对象初始化时使用 `parseStream()` 函数逐行解析，将结果（顶点、法线、面、材质）保存到成员变量中。

除 `parseStream()` 外，`smoothNormals()` 函数用于生成平滑顶点法线，可参考这篇文章¹⁹。其他成员函数实现如下，为了方便读取解析文件，使用了 `stringstream` 格式化读入。

```
1. #pragma once
2. #include <GL/glut.h>
3. #include <iostream>
4. #include <fstream>
5. #include <vector>
6. #include <string>
7. #include <sstream>
8. #include <unordered_map>
9. #include <algorithm>
10. using namespace std;
```

¹⁷ OBJ: https://en.wikipedia.org/wiki/Wavefront_.obj_file

¹⁸ Newell teaset: https://www.cs.utah.edu/~natevm/newell_teaset/newell_teaset.zip

¹⁹ Max Wagner, Generating Vertex Normals,

<https://web.archive.org/web/20130531101356/http://www.emeyex.com/site/tuts/VertexNormals.pdf>


```
11.
12. struct Vertex
13. {
14.     GLfloat x, y, z;
15. };
16.
17. struct Texture
18. {
19.     GLfloat x, y;
20. };
21.
22. struct Normal
23. {
24.     GLfloat x, y, z;
25. };
26.
27. struct Material
28. {
29.     GLfloat ambient[4] = {0.2, 0.2, 0.2, 1.0};
30.     GLfloat diffuse[4] = {0.8, 0.8, 0.8, 1.0};
31.     GLfloat specular[4] = {0.0, 0.0, 0.0, 1.0};
32.     GLfloat shininess = 65.0;
33. };
34.
35. struct Face
36. {
37.     GLint a, b, c;      // 顶点下标
38.     GLint ta, tb, tc;   // 顶点纹理下标
39.     GLint na, nb, nc;   // 顶点法线下标
40.     string mtl_name;    // 材质名
41. };
42.
43. struct OBJImporter
44. {
45.     OBJImporter(string path)
46.         : path(path)
47.     {
48.         ifstream ifs;
49.         ifs.open(path, ifstream::in);
50.         if (!ifs.is_open()) {
51.             cerr << "Can't open file.\n";
52.             return;
53.         }
54.         parseStream(ifs);
```

```

55.         ifs.close();
56.         smoothNormals();
57.     }
58.     ~OBJImporter() = default;
59.
60.     string path;
61.     vector<Vertex> vertexList;
62.     vector<Texture> textureList;
63.     vector<Normal> normalList;
64.     vector<Face> faceList;
65.     unordered_map<string, Material> materialTable;
66.
67.     void parseStream(std::ifstream& ifs) {
68.         string lineBuffer;
69.         string currentMaterialName;
70.         while (getline(ifs, lineBuffer).good())
71.         {
72.             if (lineBuffer.empty()) continue;
73.             switch (lineBuffer[0])
74.             {
75.                 case '#':    // comment
76.                     break;
77.                 case 'm':    // mtlLib
78.                     readMaterial(lineBuffer);
79.                     break;
80.                 case 'u':    // usemtl
81.                     useMaterial(lineBuffer, currentMaterialName);
82.                     break;
83.                 case 'v':    // v, vt, vn
84.                     readVertex(lineBuffer);
85.                     break;
86.                 case 'f':    // face
87.                     readFace(lineBuffer, currentMaterialName);
88.                     break;
89.                 default:     // 未支持 group
90.                     break;
91.             }
92.         }
93.     }
94.     void readMaterial(string& lineBuffer) {
95.         stringstream lineStream(lineBuffer);
96.         string skip;
97.         string materialFilename;
98.         lineStream >> skip >> materialFilename;

```

```
99.      // 在 .obj 同路径下读取 .mtl 文件
100.      ifstream mtl;
101.      mtl.open(path.substr(0, path.rfind('/') + 1) + materialFilename,
        ifstream::in);
102.      if (!mtl.is_open()) {
103.          cerr << "Can't open file.\n";
104.          return;
105.
106.      }
107.      // 解析 .mtl
108.      string lineBufferMtl;
109.      string currentMaterialName;
110.      while (getline(mtl, lineBufferMtl).good())
111.      {
112.          if (lineBuffer.empty()) continue;
113.          stringstream lineStreamMtl(lineBufferMtl);
114.          string skip;
115.          switch (lineBufferMtl[0])
116.          {
117.              case '#':
118.                  break;
119.              case 'n': // newmtl
120.                  lineStreamMtl >> skip >> currentMaterialName;
121.                  break;
122.              case 'K': // Ka Kd Ks
123.                  switch (lineBufferMtl[1])
124.                  {
125.                      case 'a': // ambient
126.                          lineStreamMtl >> skip;
127.                          for (int i = 0; i < 3; i++)
128.                          {
129.                              lineStreamMtl >>
materialTable[currentMaterialName].ambient[i];
130.                          }
131.                          break;
132.                      case 'd': // diffuse
133.                          lineStreamMtl >> skip;
134.                          for (int i = 0; i < 3; i++)
135.                          {
136.                              lineStreamMtl >>
materialTable[currentMaterialName].diffuse[i];
137.                          }
138.                          break;
139.                      case 's': // specular
```

```

140.             lineStreamMtl >> skip;
141.             for (int i = 0; i < 3; i++)
142.             {
143.                 lineStreamMtl >>
materialTable[currentMaterialName].specular[i];
144.             }
145.             break;
146.         default:
147.             break;
148.     }
149.     break;
150.     case 'N': //shininess
151.         lineStreamMtl >> skip >>
materialTable[currentMaterialName].shininess;
152.         /* wavefront shininess is from [0, 1000], so scale for
OpenGL */
153.         materialTable[currentMaterialName].shininess /= 1000;
154.         materialTable[currentMaterialName].shininess *= 128;
155.         break;
156.     default:
157.         break;
158. }
159. }
160. mtl.close();
161. }
162. void useMaterial(string& lineBuffer, string& currentMaterialName) {
163.     stringstream lineStream(lineBuffer);
164.     string skip;
165.     lineStream >> skip >> currentMaterialName;
166. }
167. void readVertex(string& lineBuffer) {
168.     stringstream lineStream(lineBuffer);
169.     string skip;
170.     switch (lineBuffer[1])
171.     {
172.     case ' ': // v 0 0 0
173.     {
174.         Vertex temp;
175.         lineStream >> skip >> temp.x >> temp.y >> temp.z;
176.         vertexList.push_back(temp);
177.     }
178.     break;
179.     case 't': // vt 0 0 // 未支持纹理
180.     {

```

```
181.         //float x, y;
182.         //lineStream >> skip >> x >> y;
183.         //textureList.push_back(x, y);
184.     }
185.     break;
186.     case 'n':    // vn 0 0 0
187.     {
188.         Normal temp;
189.         lineStream >> skip >> temp.x >> temp.y >> temp.z;
190.         normalList.push_back(temp);
191.     }
192.     break;
193.     default:
194.         break;
195.     }
196. }
197. void readFace(string& lineBuffer, string& currentMaterialName) {
198.     // can be one of %d, %d//%d, %d/%d, %d/%d/%d
199.     int flag = 0;
200.     for (int i = 0; i < lineBuffer.size(); i++)
201.     {
202.         if (lineBuffer[i] == '/')
203.         {
204.             if (i + 1 < lineBuffer.size() && lineBuffer[i + 1] == '/')
205.             {
206.                 flag = 1;    // "v//n"
207.             }
208.             else
209.             {
210.                 for (int j = i + 1; j < lineBuffer.size(); j++)
211.                 {
212.                     if (lineBuffer[j] == ' ')
213.                     {
214.                         flag = 2;    // "v/t"
215.                         break;
216.                     }
217.                     if (lineBuffer[j] == '/')
218.                     {
219.                         flag = 3;    // "v/t/n"
220.                         break;
221.                     }
222.                 }
223.             }
224.             break;
```

```

225.         }
226.     }
227.
228.
229.     // 替换 '/' 为 ' ', 便于处理
230.     for (auto& c : lineBuffer)
231.     {
232.         if (c == '/')
233.         {
234.             c = ' ';
235.         }
236.     }
237.
238.     stringstream lineStream(lineBuffer);
239.     string skip;
240.     lineStream >> skip;
241.
242.     switch (flag)
243.     {
244.     case 0:    // f v v v (v)
245.     {
246.         int a, b, c, d;
247.         lineStream >> a >> b >> c;
248.         Face temp = { a - 1, b - 1, c - 1, 0, 0, 0, 0, 0, 0,
currentMaterialName }; // 处理下标偏移
249.         faceList.push_back(temp);
250.         // 只支持三角形面和四边形面，四边形面当做两个三角形面处理
251.         if (lineStream.good())
252.         {
253.             lineStream >> d;
254.             Face temp = { a - 1, c - 1, d - 1, 0, 0, 0, 0, 0, 0,
currentMaterialName };
255.             faceList.push_back(temp);
256.         }
257.     }
258.     break;
259.     case 1:    // f v//n v//n v//n (v//n)
260.     {
261.         int a, b, c, d;
262.         int an, bn, cn, dn;
263.         lineStream >> a >> an >> b >> bn >> c >> cn;
264.         Face temp = { a - 1, b - 1, c - 1, 0, 0, 0, an - 1, bn - 1, cn
- 1, currentMaterialName };
265.         faceList.push_back(temp);

```

```

266.         if (lineStream.good())
267.         {
268.             lineStream >> d >> dn;
269.             Face temp = { a - 1, c - 1, d - 1, 0, 0, 0, an - 1, cn -
1, dn - 1, currentMaterialName };
270.             faceList.push_back(temp);
271.         }
272.     }
273.     break;
274.     case 2:    // f v/t v/t v/t (v/t)
275.     {
276.         // 未支持 vt
277.         int a, b, c, d;
278.         lineStream >> a >> b >> c;
279.         Face temp = { a - 1, b - 1, c - 1, 0, 0, 0, 0, 0, 0,
currentMaterialName }; // 处理下标偏移
280.         faceList.push_back(temp);
281.         if (lineStream.good())
282.         {
283.             lineStream >> d;
284.             Face temp = { a - 1, c - 1, d - 1, 0, 0, 0, 0, 0, 0,
currentMaterialName };
285.             faceList.push_back(temp);
286.         }
287.     }
288.     break;
289.     case 3:    // f v/t/n v/t/n v/t/n (v/t/n)
290.     {
291.         // 未支持 vt
292.         int a, b, c, d;
293.         int an, bn, cn, dn;
294.         int skip;
295.         lineStream >> a >> skip >> an >> b >> skip >> bn >> c >>
skip >> cn;
296.         Face temp = { a - 1, b - 1, c - 1, 0, 0, 0, an - 1, bn - 1, cn
- 1, currentMaterialName };
297.         faceList.push_back(temp);
298.         if (lineStream.good())
299.         {
300.             lineStream >> d >> skip >> dn;
301.             Face temp = { a - 1, c - 1, d - 1, 0, 0, 0, an - 1, cn -
1, dn - 1, currentMaterialName };
302.             faceList.push_back(temp);
303.         }

```

```

304.         }
305.         break;
306.         default:
307.             break;
308.     }
309. }
310. void smoothNormals() {
311.     // 生成面法线
312.     vector<Normal> faceNormalList;
313.     faceNormalList.resize(faceList.size());
314.     for (int i = 0; i < faceList.size(); i++)
315.     {
316.         Face& f = faceList[i];
317.         Normal& fn = faceNormalList[i];
318.         // 叉积
319.         Normal Vab = {
320.             vertexList[f.b].x - vertexList[f.a].x,
321.             vertexList[f.b].y - vertexList[f.a].y,
322.             vertexList[f.b].z - vertexList[f.a].z
323.         };
324.         Normal Vac = {
325.             vertexList[f.c].x - vertexList[f.a].x,
326.             vertexList[f.c].y - vertexList[f.a].y,
327.             vertexList[f.c].z - vertexList[f.a].z
328.         };
329.         fn = {
330.             Vab.y * Vac.z - Vab.z * Vac.y,
331.             Vab.z * Vac.x - Vab.x * Vac.z,
332.             Vab.x * Vac.y - Vab.y * Vac.x
333.         };
334.         // 正则化
335.         GLfloat l = (GLfloat)sqrt(fn.x * fn.x + fn.y * fn.y + fn.z *
fn.z);
336.         fn = {
337.             fn.x / l,
338.             fn.y / l,
339.             fn.z / l
340.         };
341.     }
342.
343.     // 不需要原有的法线
344.     if (!normalList.empty())
345.     {
346.         normalList.clear();

```



```
347.     }
348.
349.     // 为每个顶点添加成员面（所有包含这个顶点的面）
350.     vector<vector<int>> members;
351.     members.resize(vertexList.size());
352.     for (int i = 0; i < faceList.size(); i++)
353.     {
354.         members[faceList[i].a].push_back(i);
355.         members[faceList[i].b].push_back(i);
356.         members[faceList[i].c].push_back(i);
357.     }
358.
359.     // 生成平滑顶点法线
360.     for (int i = 0; i < vertexList.size(); i++)
361.     {
362.         Normal avg = { 0, 0, 0 };
363.         Normal fn1 = { 0, 0, 0 };
364.         for (auto& findex : members[i])
365.         {
366.             // 只有第一次或者与第一个面的夹角小于 90 度才添加该面法线
367.             Normal& fn = faceNormalList[findex];
368.             if (fn1.x * fn1.x + fn1.y * fn1.y + fn1.z * fn1.z == 0)
369.             {
370.                 fn1 = { fn.x, fn.y, fn.z };
371.                 avg.x += fn.x;
372.                 avg.y += fn.y;
373.                 avg.z += fn.z;
374.             }
375.             else if (fn1.x * fn.x + fn1.y * fn.y + fn1.z * fn.z)
376.             {
377.                 avg.x += fn.x;
378.                 avg.y += fn.y;
379.                 avg.z += fn.z;
380.             }
381.         }
382.         GLfloat l = (GLfloat)sqrt(avg.x * avg.x + avg.y * avg.y +
            avg.z * avg.z);
383.         avg.x /= l;
384.         avg.y /= l;
385.         avg.z /= l;
386.         normalList.push_back(avg);
387.     }
388.     };
```

使用读取到的数据，可以绘制模型对象，在原来绘制立方体的基础上，更改了如下函数内容。OpenGL 光照函数参考教材 17.11 节。

对于不同初始大小的模型，示例程序并没有做统一化，其实也并不难，但是会导致代码片段过长。所以这里只是对特定的茶壶模型做平移和旋转操作，以放置在视窗中心。

```
1. void triangle(OBJImporter& obj, Face& f)
2. {
3.     auto& vertexList = obj.vertexList;
4.     auto& normalList = obj.normalList;
5.     auto& materialTable = obj.materialTable;
6.     if (drawLines) {
7.         glColor3f(1.0, 1.0, 1.0);
8.         glBegin(GL_LINE_LOOP);
9.             glVertex3f(vertexList[f.a].x, vertexList[f.a].y, vertexList[f.a]
.z);
10.             glVertex3f(vertexList[f.b].x, vertexList[f.b].y, vertexList[f.b]
.z);
11.             glVertex3f(vertexList[f.c].x, vertexList[f.c].y, vertexList[f.c]
.z);
12.         glEnd();
13.     }
14.     else {
15.         glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, materialTable[f.mtl_name
].ambient);
16.         glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, materialTable[f.mtl_name
].diffuse);
17.         glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, materialTable[f.mtl_nam
e].specular);
18.         glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, materialTable[f.mtl_nam
e].shininess);
19.
20.         glBegin(GL_TRIANGLES);
21.
22.         //glNormal3f(normalList[f.na].x, normalList[f.na].y, normalList[f.na
].z);
23.         glNormal3f(normalList[f.a].x, normalList[f.a].y, normalList[f.a].z);
24.         glVertex3f(vertexList[f.a].x, vertexList[f.a].y, vertexList[f.a].z);
25.
```

```
26.         //glNormal3f(normalList[f.nb].x, normalList[f.nb].y, normalList[f.nb
].z);
27.         glNormal3f(normalList[f.b].x, normalList[f.b].y, normalList[f.b].z);
28.         glVertex3f(vertexList[f.b].x, vertexList[f.b].y, vertexList[f.b].z);
29.
30.         //glNormal3f(normalList[f.nc].x, normalList[f.nc].y, normalList[f.nc
].z);
31.         glNormal3f(normalList[f.c].x, normalList[f.c].y, normalList[f.c].z);
32.         glVertex3f(vertexList[f.c].x, vertexList[f.c].y, vertexList[f.c].z);
33.
34.         glEnd();
35.     }
36. }
37.
38. void drawteapot()
39. {
40.     static OBJImporter teapot("./newell_teaset/teapot.obj");
41.
42.     for (auto& f : teapot.faceList)
43.     {
44.         triangle(teapot, f);
45.     }
46. }
47.
48. void display(void)
49. {
50.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
51.
52.     /* view transform */
53.     glLoadIdentity();
54.     glTranslatef(0.0, 0.0, viewxform_z);
55.
56.     /* Enable a single OpenGL light. */
57.     glEnable(GL_NORMALIZE);
58.     static const GLfloat light_position[] = { 0, 0, 1, 0 };
59.     glLightfv(GL_LIGHT0, GL_POSITION, light_position);
60.     glEnable(GL_LIGHT0);
61.     glEnable(GL_LIGHTING);
62.     glEnable(GL_DEPTH_TEST);
63.     glTranslatef(0, -2, 0);
64.     glRotatef(90, 0, 1, 0);
65.     drawteapot();
66.
67.     glutSwapBuffers();
```

至此，程序可以加载显示 OBJ 模型。对应实验结果参考中的图一。

导入外部模型（PLY）

斯坦福兔子模型（Stanford Bunny）是图形学领域中的一个经典模型，其提供 PLY 格式模型文件下载²⁰。实验使用 `PLYImporter` 类管理 PLY 模型导入，读者可以参考设计出其他三维对象文件格式的导入工具，如 OBJ 格式。

根据上面的实验，我们只需要将模型文件解析出顶点数据和面片数据，构建 `vertices` 顶点数组，并将顶点数组按照面片数据组合为 `polygon`，即可在 GLUT 程序中导入外部模型。

PLY 模型结构如下所示，453 个顶点数据，包含坐标和其他数据，948 个面片，首项的 3 表明由 3 个点围成三角形面片，后续的 3 项为顶点数组的序号。

```

1. ply
2. format ascii 1.0
3. comment zipper output
4. element vertex 453
5. property float x
6. property float y
7. property float z
8. property float confidence
9. property float intensity
10. element face 948
11. property list uchar int vertex_indices
12. end_header
13. -0.0312216 0.126304 0.00514924 0.850855 0.5
14. -0.0446774 0.131204 0.00570479 0.900159 0.5
15. -0.0683011 0.144828 0.0413688 0.398443 0.5
16. -0.00600095 0.130398 0.0178986 0.85268 0.5
17. -0.0173568 0.127613 0.00526885 0.675938 0.5
18. ....省略.....
19. ....共 453 个顶点.....
20. 3 164 94 98

```

²⁰ The Stanford 3D Scanning Repository: <http://graphics.stanford.edu/data/3Dscanrep/>

```

21. 3 224 335 49
22. 3 331 350 376
23. 3 124 122 237
24. 3 61 89 69
25. ....省略.....
26. ....共 948 个面片.....

```

由此 PLY 模型文件写出 PLYImporter.h 文件，用于定义 PLYImporter 类解析导入数据。

首先引入需要用到的库和数据结构。Vertex 类储存顶点坐标和顶点颜色。Face 类储存三角形面片的组成顶点的序号。

```

1. #pragma once
2. #include <GL/glut.h>
3. #include <cstdlib>
4. #include <cstring>
5. #include <fstream>
6. #include <iostream>
7.
8. struct Vertex
9. {
10.     GLfloat x, y, z, r, g, b;
11. };
12.
13. struct Face
14. {
15.     GLint a, b, c;
16. };

```

PLYImporter 结构体包含如下成员。

构造函数打开 filename，然后进入 parseStream 函数解析输入文件流，用于构造两个主要的成员属性，即 vertexPointer 和 facePointer。

GetVertexPointer 以及之后的几个函数，用于返回属性句柄。

```

1. struct PLYImporter
2. {
3.
4.     PLYImporter(const char* filename) {
5.         std::ifstream ifs;

```

```

6.         ifs.open(filename, std::ifstream::in);
7.         if (!ifs.is_open()) {
8.             std::cout << "open wrong" << std::endl;
9.             return;
10.        }
11.        parseStream(ifs);
12.        ifs.close();
13.    }
14.    ~PLYImporter() {
15.        delete[] vertexPointer;
16.        delete[] facePointer;
17.    }
18.    Vertex* GetVertexPointer() {
19.        return vertexPointer;
20.    }
21.    Face* GetFacePointer()
22.    {
23.        return facePointer;
24.    }
25.    GLint GetVertexNum() {
26.        return vertexNum;
27.    }
28.    GLint GetFaceNum() {
29.        return faceNum;
30.    }
31.
32.    Vertex* vertexPointer; // 指向顶点表
33.    Face* facePointer;     // 指向面片表
34.    GLint vertexNum = 0;
35.    GLint faceNum = 0;

```

成员函数 `parseStream` 借助 `getToken` 获取读入行中每个 word 的 start 和 end 坐标，使用 `matchToken` 比较其是否与 keyword 相同。

首先解析 PLY 文件的 header 部分，匹配到 `element vertex xxx` 和 `element face yyy` 所在行，根据数值分配指定大小的内存空间。

之后进入 body 部分，把所有顶点数据拷贝进 `vertexPointer` 指向的空间，面片数据拷贝到 `facePointer` 指向的空间。

因为斯坦福兔子模型没有提供顶点颜色或者顶点法线数据，顶点数据只包含顶点坐标、置信度和强度，因此这里暂时使用置信度作为 RGB 颜色灰度值。

在 `PLYImporter` 对象创建后，其成员属性已经就绪。

```
1. void parseStream(std::ifstream& ifs) {
2.     char lineBuffer[256];
3.     // parse header
4.     while (ifs.getline(lineBuffer, 256).good())
5.     {
6.         int start = 0;
7.         int end = 0;
8.         if (getToken(lineBuffer, 256, start, end))
9.         {
10.            if (matchToken(lineBuffer, start, end, "element")
11.                && getToken(lineBuffer, 256, start, end))
12.            {
13.                // match element vertex xxx
14.                if (matchToken(lineBuffer, start, end, "vertex")
15.                    && getToken(lineBuffer, 256, start, end))
16.                {
17.                    // set xxx
18.                    vertexNum = tokenToInt(lineBuffer, start, end);
19.
20.                    // 分配空间
21.                    vertexPointer = new Vertex[vertexNum];
22.                }
23.
24.                // match element face yyy
25.                if (matchToken(lineBuffer, start, end, "face")
26.                    && getToken(lineBuffer, 256, start, end))
27.                {
28.                    // set yyy
29.                    faceNum = tokenToInt(lineBuffer, start, end);
30.
31.                    // 分配空间
32.                    facePointer = new Face[faceNum];
33.                }
34.            }
35.
36.            if (matchToken(lineBuffer, start, end, "end_header"))
37.            {
38.                break;
39.            }
40.        }
41.    }
42.    // parse body
43.    int vertexCount = 0;
44.    int faceCount = 0;
```

```

45.     while (ifs.getline(lineBuffer, 256).good())
46.     {
47.         int start = 0;
48.         int end = 0;
49.         if (vertexCount < vertexNum)
50.         {
51.             getToken(lineBuffer, 256, start, end);
52.             vertexPointer[vertexCount].x = tokenToFloat(lineBuffer, start, end);
53.             getToken(lineBuffer, 256, start, end);
54.             vertexPointer[vertexCount].y = tokenToFloat(lineBuffer, start, end);
55.             getToken(lineBuffer, 256, start, end);
56.             vertexPointer[vertexCount].z = tokenToFloat(lineBuffer, start, end);
57.             //getToken(lineBuffer, 256, start, end);
58.             getToken(lineBuffer, 256, start, end);
59.             vertexPointer[vertexCount].r = tokenToFloat(lineBuffer, start, end);
60.             vertexPointer[vertexCount].g = vertexPointer[vertexCount].r;
61.             vertexPointer[vertexCount].b = vertexPointer[vertexCount].r;
62.             vertexCount++;
63.         }
64.         else if (faceCount < faceNum)
65.         {
66.             getToken(lineBuffer, 256, start, end); // 去掉开头的 3
67.             getToken(lineBuffer, 256, start, end);
68.             facePointer[faceCount].a = tokenToInt(lineBuffer, start, end);
69.             getToken(lineBuffer, 256, start, end);
70.             facePointer[faceCount].b = tokenToInt(lineBuffer, start, end);
71.             getToken(lineBuffer, 256, start, end);
72.             facePointer[faceCount].c = tokenToInt(lineBuffer, start, end);
73.             faceCount++;
74.         }
75.     }
76. }
77. bool getToken(const char* lineBuffer, int length, int& start, int& end) {
78.     if (lineBuffer[end] == '\0') return false;
79.     for (int i = end; lineBuffer[i] <= length; i++)
80.     {
81.         if (lineBuffer[i] != ' ')
82.         {
83.             if (lineBuffer[i] == '\0' || lineBuffer[i] == EOF)
84.             {

```



```

85.         return false;
86.     }
87.     else
88.     {
89.         start = i;
90.         break;
91.     }
92. }
93. }
94. for (int i = start; lineBuffer[i] <= length; i++)
95. {
96.     if (lineBuffer[i] == ' ' || lineBuffer[i] == '\0' || lineBuffer[i] =
    = EOF)
97.     {
98.         end = i;
99.         break;
100.    }
101. }
102. return true;
103. }
104. bool matchToken(const char* lineBuffer, int start, int end, const char* key
    word) {
105.     return memcmp(lineBuffer + start, keyword, end - start) == 0;
106. }
107. int tokenToInt(const char* lineBuffer, int start, int end) {
108.     char token[32];
109.     memcpy(token, lineBuffer + start, end - start);
110.     return atoi(token);
111. }
112. double tokenToFloat(const char* lineBuffer, int start, int end) {
113.     char token[32];
114.     memcpy(token, lineBuffer + start, end - start);
115.     return atof(token);
116. }

```

根据绘制立方体的方法，稍作修改就可以得到绘制兔子模型的方法。将 display 函数中的 drawcube 替换为 drawbunny 即可。

```

1. void triangle(Vertex* vertexPointer, int a, int b, int c)
2. {
3.     if (drawLines) {
4.         glColor3f(1.0, 1.0, 1.0);
5.         glBegin(GL_LINE_LOOP);

```

```

6.         glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer
   [a].z);
7.         glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer
   [b].z);
8.         glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer
   [c].z);
9.         glEnd();
10.    }
11.    else {
12.        glBegin(GL_TRIANGLES);
13.
14.        glColor3f(vertexPointer[a].r, vertexPointer[a].g, vertexPointer[a].b
   );
15.        glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer[a].
   z);
16.
17.        glColor3f(vertexPointer[b].r, vertexPointer[b].g, vertexPointer[b].b
   );
18.        glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer[b].
   z);
19.
20.        glColor3f(vertexPointer[c].r, vertexPointer[c].g, vertexPointer[c].b
   );
21.        glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer[c].
   z);
22.
23.        glEnd();
24.    }
25. }
26.
27. void drawbunny()
28. {
29.     static PLYImporter bunny("./bunny/reconstruction/bun_zipper.ply");
30.     Vertex* vertexPointer = bunny.GetVertexPointer();
31.     Face* facePointer = bunny.GetFacePointer();
32.     for (int i = 0; i < bunny.GetFaceNum(); i++)
33.     {
34.         triangle(vertexPointer, facePointer[i].a, facePointer[i].b, facePoin
   ter[i].c);
35.     }
36. }

```

将模型视图移到视野中，程序可以显示出兔子模型的大致轮廓，但表面过于

斑驳,这是因为我们将置信度作为颜色值来渲染,实际上模型不包含顶点颜色值。如果需要更真实的显示效果,需要计算顶点的法线值,并给予光照,计算阴影。



OpenGL 光照函数参考教材 17.11 节。修改之^后的代码如下。

绘制三角形面片时,通过面片的两条边计算得出垂直于面片的法线向量,并设置到三个顶点;不再设置顶点颜色,只使用默认的白色。

在 display 函数绘制兔子对象前,开启法线正则化,统一所用到的法线为单位向量;设置一个来自 (1,1,1) 方向的远距离平行光 LIGHT_0;启用 LIGHT_0,启用 LIGHTING 模式。

```
1. void triangle(Vertex* vertexPointer, int a, int b, int c)
2. {
3.     if (drawLines) {
4.         //...
5.     }
6.     else {
7.         glBegin(GL_TRIANGLES);
8.
9.         GLfloat Vab[3] = {
10.             vertexPointer[b].x - vertexPointer[a].x,
11.             vertexPointer[b].y - vertexPointer[a].y,
12.             vertexPointer[b].z - vertexPointer[a].z
13.         };
14.         GLfloat Vac[3] = {
15.             vertexPointer[c].x - vertexPointer[a].x,
16.             vertexPointer[c].y - vertexPointer[a].y,
17.             vertexPointer[c].z - vertexPointer[a].z
18.         };
19.
```

```

20.         glNormal3f(
21.             Vab[1] * Vac[2] - Vab[2] * Vac[1],
22.             Vab[2] * Vac[0] - Vab[0] * Vac[2],
23.             Vab[0] * Vac[1] - Vab[1] * Vac[0]
24.         );
25.
26.         //glColor3f(vertexPointer[a].r, vertexPointer[a].g, vertexPointer[a]
27.         .b);
28.         glVertex3f(vertexPointer[a].x, vertexPointer[a].y, vertexPointer[a].
29.         z);
30.
31.         //glColor3f(vertexPointer[b].r, vertexPointer[b].g, vertexPointer[b]
32.         .b);
33.         glVertex3f(vertexPointer[b].x, vertexPointer[b].y, vertexPointer[b].
34.         z);
35.
36.         //glColor3f(vertexPointer[c].r, vertexPointer[c].g, vertexPointer[c]
37.         .b);
38.         glVertex3f(vertexPointer[c].x, vertexPointer[c].y, vertexPointer[c].
39.         z);
40.
41.     glEnd();
42. }
43. }
44.
45. // ...
46.
47. void display(void)
48. {
49.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
50.
51.     /* view transform */
52.     glLoadIdentity();
53.     glTranslatef(0.0, -2, -5);
54.     glScalef(15, 15, 15);
55.
56.     /* Enable a single OpenGL light. */
57.     glEnable(GL_NORMALIZE);
58.     GLfloat light_position[] = {1.0, 1.0, 1.0, 0.0};
59.     glLightfv(GL_LIGHT0, GL_POSITION, light_position);
60.     glEnable(GL_LIGHT0);
61.     glEnable(GL_LIGHTING);
62.     glEnable(GL_DEPTH_TEST);

```

```
58.    //glutSolidTeapot(1.0);
59.
60.    //drawcube();
61.    drawbunny();
62.
63.    glutSwapBuffers();
64.    //glutPostRedisplay();
65. }
```

最后，程序可以得到实验十 中相同的结果。



附录 4 实验十一 C 语言实现

实验目的

使用 OpenGL 函数实现 2D 绘图软件，支持直线绘制，圆形绘制，曲线绘制，区域填充等功能。

实验内容

基于贝塞尔曲线实验，我们可以轻松扩展 GLObject.h 头文件，并实现功能细节，将其扩展为简单绘图软件。

GLLine 和 GLCircle 与 GLBezierCtrlPts 结构类似，其中 GLLine 使用多个控制点作为起点和终点绘制线段，GLCircle 使用 2 个控制点作为圆心和圆上一点绘制圆形。内部实现可以借由 OpenGL 绘制函数，或通过算法由点生成。所有控制点都可以修改位置，从而修改原始图形。

```
1. struct GLLine
2. {
3.     GLWorld* parentWorld;
4.     GLPoint* ctrlPoints[MAX_LINE_CONTROL_POINTS_NUM];
5.     int count = 0;
6.
7.
8.     void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
9.         if (count >= MAX_LINE_CONTROL_POINTS_NUM) return;
10.        ctrlPoints[count] = parentWorld->CreatePt(x, y, z);
11.        //ctrlPoints[count]->SetPosition(x, y, z);           // 初始位置
12.        ctrlPoints[count]->movable = true;                  // 可以移动
13.        count++;
14.        //printf("mp \n");
15.    }
16.    void Draw(GLenum RenderMode = GL_RENDER) {
17.        // 需要在绘制前初始化控制点
18.        if (count == 0) return;
19.        for (int i = 0; i < count; ++i)
20.        {
21.            if (!ctrlPoints[i]) return;
22.        }
23.        //if (RenderMode == GL_SELECT) glLoadName((GLuint)id);
```

```

24.     glColor3f(0.0, 0.0, 0.0);
25.     glPointSize(DEFAULT_LINE_WIDTH);
26.     glBegin(GL_LINE_STRIP);
27.     for (int i = 0; i < count; i++)
28.     {
29.         glVertex3f(ctrlPoints[i]->transform.x, ctrlPoints[i]->transform.y,
        ctrlPoints[i]->transform.z);
30.     }
31.     glEnd();
32.     glColor3f(0.0, 0.0, 1.0);
33.     glBegin(GL_POINTS);
34.     glPointSize(DEFAULT_POINT_SIZE);
35.     for (int i = 0; i < count; i++)
36.     {
37.         glVertex3f(ctrlPoints[i]->transform.x, ctrlPoints[i]->transform.y,
        ctrlPoints[i]->transform.z);
38.     }
39.     glEnd();
40. }
41. };

```

```

1.  struct GLCircle
2.  {
3.      GLPoint * ctrlPoints[DEFAULT_CIRCLE_CONTROL_POINTS_NUM];
4.      GLWorld* parentWorld;
5.      int count = 0;
6.
7.      bool IsFinished() {
8.          return count >= DEFAULT_CIRCLE_CONTROL_POINTS_NUM ;
9.      }
10. }
11.
12. void MakePoint(GLdouble x, GLdouble y, GLdouble z) {
13.     if (count >= DEFAULT_CIRCLE_CONTROL_POINTS_NUM) return;
14.
15.     ctrlPoints[count] = parentWorld->CreatePt(x, y, z);    // 初始位置
16.     ctrlPoints[count]->movable = true;                    // 可以移动
17.     count++;
18.
19. }
20. void drawCircle() {
21.     // 修改之前自己实现的画圆函数
22.     //.....
23. }

```

```
24. //.....
25. };
```

GLPaint 同样与 GLBezierCtrlPts 类似，但行为与上述两者不同。在程序动态创建 GLPaint 对象时，需要初始化该对象坐标和颜色值，从而将初始坐标作为填充种子点，将颜色值作为填充色进行填充计算。填充算法采用扫描线 4 连通方法，从而避免对栈内存的大量消耗。

```
1. struct Paint
2. {
3.     Color color;
4.     bool finished;
5.     void Draw(GLenum RenderMode = GL_RENDER) {
6.         // 需要在绘制前起始位置和颜色
7.         // SetPosition & SetColor
8.
9.         if (finished) return;
10.
11.         glColor3f(color.r, color.g, color.b);
12.         glPointSize(MIN_PAINT_POINT_SIZE);
13.         FloodFill(); // 使用之前自己实现的扫描线四联通填充算法
14.         finished = true;
15.
16.     }
17. };
```

实验示例作为绘图软件仍有许多工作需要进一步完善。如：图形填充后，填充部分应当写入背景，而不应该被每帧绘制或者清空；动态的获取填充颜色值，实现更加直观的用户接口（调色板）；从 GLObject 扩展出更多图形，如三角形，长方形，以及笔刷。可以考虑自行添加上述功能进行实现。